Tklib

Reference manual pages

2026-10-26

Contents

apps **5**
bitmap-editor 5
diagram-viewer 5
shtmlview 7
autoscroll **8**
autoscroll 8
canvas **9**
canvas_drag 9
canvas_ecircle 12
canvas_epoints 16
canvas_epolyline 22
canvas_equad 27
canvas_erectangle 31
canvas_gradient 35
canvas_highlight 37
canvas_mvg 38
canvas_pdf 39
canvas_snap 40
canvas_sqmap 40
canvas_tags 42
canvas_trlines 44
canvas_zoom 45
chatwidget **46**
chatwidget 46
controlwidget **48**
controlwidget 48
crosshair **56**
crosshair 56
ctext **58**
ctext 58
cursor **63**
cursor 63
datefield **63**
datefield 63
diagrams **65**
diagram 65
getstring **105**
tk_getString 105
history **106**
tklib_history 106
ico **107**
ico 107
ipentry **110**
ipentry 110

khim **113**
khim 113
map **115**
area-display 115
area-file 116
area-map-display 117
area-store-fs 117
area-store-mem 118
area-table-display 119
box-display 119
box-entry 120
box-file 121
box-map-display 121
box-store-fs 122
box-store-mem 123
box-table-display 123
display 124
map_overview 125
mark 126
point-file 127
point-map-display 127
point-store-fs 128
point-store-mem 129
point-table-display 129
provider-osm 130
track-display 131
track-entry 131
track-file 132
track-map-display 133
track-store-fs 133
track-store-mem 134
track-table-display 135
menubar **135**
menubar 135
notifywindow **146**
notifywindow 146
ntext **146**
ntext 146
ntextBindings 150
ntextIndent 153
ntextWordBreak 157
persistentSelection **160**
persistentSelection 160
plotchart **164**
plotchart 164
shtmlview **228**

shtmlview-doctools 228
shtmlview-mkdoc 229
shtmlview 231
swaplist 239
swaplist 239
tkpiechart 240
canvaslabel 240
pie 242
pieboxlabeler 246
pieperipherallabeler 247
tooltip 248
tipstack 248
tooltip 250
widget 253
widget 253
widget_calendar 255
widget_dateentry 256
widget_toolbar 258
widgetPlus 259
widgetPlus 259
widgetl 265
widget_listentry 265
widget_listsimple 271
widgetv 274
widget_validator 274
License 277
license license terms 277

apps

bitmap-editor

Keywords: bitmap-editor, bitmap, editor, image, pixel, xbm

Name

bitmap-editor - Editor for XBM images

Synopsis

bitmap-editor ?*xbmfile*?

Description

The application described by this document, **bitmap-editor**, is a simple editor for XBM based bitmap images. Written by Keith Vetter the original code can be found at http://wiki.tcl-lang.org/6298.

Command line

bitmap-editor ?*xbmfile*?

Invoked without argument the editor GUI will be opened and show a standard bitmap to edit. Invoked with an argument it is expected to be the path to a bitmap file in XBM format, and the contained bitmap is shown.

path *xbmfile* (in/out)

This argument specifies the path to a bitmap file in XBM format, whose contents is to shown and edited by the application.

Bugs, ideas, feedback

This document, and the application it describes, will undoubtedly contain bugs and other problems. Please report such in the category *bitmap* of the Tcllib SF Trackers. Please also report any ideas for enhancements you may have for either application and/or documentation.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *bitmap-editor* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Image processing

Copyright

```
Copyright (c)
```

diagram-viewer

Keywords: diagram-viewer, canvas, conversion, diagram, vector

Name

diagram-viewer - Lightweight Diagram Processor

Synopsis

diagram-viewer show *inputfile...*
diagram-viewer convert -o *output format inputfile...*

Description

The application described by this document, **diagram-viewer**, is a lightweight processor for tklib diagram files

diagram-viewer is based upon the package **diagram**. See it for examples and language reference.

Use cases

diagram-viewer was written with the following two use cases in mind.

1. Processing and display of one or more diagram files.
2. Batch conversion of one or more diagram files into raster image files.

Command line

diagram-viewer show *inputfile*...

This is the form for use case [1]. The application opens a gui showing the list of input files to the left, allowing the user to choose which of them to render to the canvas on the right.

diagram-viewer convert -o *output format inputfile*...

This is the form for use case [2]. The application converts the input files into raster image of the specified *format*.

path *output* (in)

This argument specifies where to write the generated image. It can be the path to a file or directory.

If the *output* does not exist then [file dirname $output] has to exist and must be a writable directory.

In case of multiple input files the generated image will be written to a file in the directory, and the name of that file will be derived from the *inputfile*, and *format*.

In case of a single input file the generated image will be written to the file.

(handle) *format* (in)

This argument specifies the image format to convert the diagrams into when processing the input. The application recognizes all formats supported by the **Img** package, i.e. for which it can load a package **img::format** (together with **canvas::snap**), and all formats for which it can load a package **canvas::format**. These are (currently) **mvg**, and **pdf**.

path *inputfile* (in)

This argument specifies the path to the diagram file to process. It has to exist, must be readable, and written in *diagram* format.

Bugs, ideas, feedback

This document, and the application it describes, will undoubtedly contain bugs and other

problems. Please report such in the category *diagram* of the Tcllib SF Trackers. Please also report any ideas for enhancements you may have for either application and/or documentation.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *diagram* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Documentation tools

Copyright

```
Copyright (c) 2010 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

shtmlview

Keywords: shtmlview, doctools, html, markdown, mkdoc, viewer

Name

shtmlview - Basic HTML and Markdown viewer application

Synopsis

shtmlview
shtmlview --help
shtmlview --version
shtmlview --render *text*
shtmlview --render -
shtmlview *filename*

Description

The application described by this document, **shtmlview**, is a simple viewer for HTML files. Further supported are documents written in *Markdown*, *doctools*, or *mkdoc* format, if various supporting packages are available. See section Beyond HTML.

Command line

shtmlview
: see below ...

shtmlview --help
: Prints usage information to standard error.

shtmlview --version
: Prints the application version to standard output.

shtmlview --render *text*
: Displays the provided text in a viewer window. Treats the text as Markdown except if it starts with a recognized HTML tag. In that case it assumes and renders as HTML.

shtmlview --render -
: As before, except it reads the text to render from standard input.

shtmlview *filename*
Reads and renders the named file in a viewer widget.

Beyond HTML

Documents in any other format than HTML require the presence of various supporting packages to enable the application to display them.

Markdown
- **Markdown**

doctools
- **shtmlview::doctools**
- **dtplite**

mkdoc
- **shtmlview::mkdoc**
- **mkdoc::mkdoc**

Bugs, ideas, feedback

This document, and the application it describes, will undoubtedly contain bugs and other problems. Please report such in the category *shtmlview* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either application and/or documentation.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *shtmlview* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Document display

Copyright

```
Copyright (c)
```

autoscroll

autoscroll

Keywords: autoscroll, scroll, scrollbar

Name

autoscroll - Provides for a scrollbar to automatically mapped and unmapped as needed

Synopsis

package require Tcl
package require autoscroll ?**1.1**?
::autoscroll::autoscroll *scrollbar*
::autoscroll::unautoscroll *scrollbar*

::autoscroll::wrap
::autoscroll::unwrap

Description

This package allows scrollbars to be mapped and unmapped as needed depending on the size and content of the scrollbars scrolled widget. The scrollbar must be managed by either pack or grid, other geometry managers are not supported.

When managed by pack, any geometry changes made in the scrollbars parent between the time a scrollbar is unmapped, and when it is mapped will be lost. It is an error to destroy any of the scrollbars siblings while the scrollbar is unmapped. When managed by grid, if anything becomes gridded in the same row and column the scrollbar occupied it will be replaced by the scrollbar when remapped.

This package may be used on any scrollbar-like widget as long as it supports the **set** subcommand in the same style as scrollbar. If the **set** subcommand is not used then this package will have no effect.

::autoscroll::autoscroll *scrollbar*
: Arranges for the already existing scrollbar **scrollbar** to be mapped and unmapped as needed.

::autoscroll::unautoscroll *scrollbar*
: Returns the named scrollbar to its original static state.

::autoscroll::wrap
: Arranges for all scrollbars created after this command is run to be automatically mapped and unmapped as needed.

::autoscroll::unwrap
: Turns off the automatic autoscrolling of all new scrollbars. Does not effect existing scrollbars

```
text .t -yscrollcommand ".scrolly set"
scrollbar .scrolly -orient v -command ".t yview"
pack .scrolly -side right -fill y
pack .t -side left -fill both -expand 1
::autoscroll::autoscroll .scrolly
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *autoscroll* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas

canvas_drag

Keywords: canvas_drag, canvas, dragging

Name

canvas::drag - Manage the dragging of canvas items or item groups

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::drag ?**0.1**?
::canvas::drag on *canvas tagOrId cmd option...*
::canvas::drag off *canvas tagOrId*
::canvas::drag item *canvas tagOrId option...*
::canvas::drag group *canvas tagOrId cmd option...*
{*}cmd start *canvas item*
{*}cmd move *canvas clientdata dx dy*
{*}cmd done *canvas clientdata*

Description

This package provides utility commands to setup and rmeove dragging of items or item groups on a canvas, hiding all complexity regarding bindings from the user.

API

::canvas::drag on *canvas tagOrId cmd option...*

This command initializes item dragging on the *canvas* widget, with the items used as drag handles identified by *tagOrId*. The command prefix *cmd*, invoked for drag start and movement, is responsible for the initialization and actual execution of the drag operation.

The signature of the command prefix is described later, in section Drag callback.

Similarly, the accepted options and their values are described in section Options

The result of the command is the empty string.

::canvas::drag off *canvas tagOrId*

This command removes any drag operation set on the items of canvas *canvas* identified by *tagOrId*.

The result of the command is the empty string.

::canvas::drag item *canvas tagOrId option...*

This is a convenience command wrapped around method **on** (see above) to drag single items of the *canvas* widget, identified by *tagOrId*.

It uses an internal standard callback for this.

The result of the command is the empty string.

::canvas::drag group *canvas tagOrId cmd option...*

This is a convenience command wrapped around method **on** (see above) to drag single items of the *canvas* widget, identified by *tagOrId*.

It uses an internal standard callback for this. The callback *cmd* specified has the same signature as the Drag callback, except that

1. The **move** method is not invoked.
2. The result of the **start** method *has to be* a canvas tag refering to the whole group of items to move. In other words, it must convert from drag handle (item id) to dragged groupt

(tag).

The result of the command is the empty string.

Drag callback

The drag callback is a command prefix invoked in the following two ways:

{*}cmd start *canvas item*
This form is invoked when has initiated dragging using drag handle identified by the canvas *item* id. The callback now has to perform anything necessary for its type of drag operation.

The result of the command can be anything. It is stored by the system as client information and passed unchanged to the movement callback for its use. In this manner the drag callback is able to maintain custom state from start to movement.

{*}cmd move *canvas clientdata dx dy*
This form is invoked when the mouse moved during a drag operation. It is invoked with the client data from the start callback (or the previous move callback) and the distances the mouse has traveled in horizontal and vertical directions.

The result of the command can be anything. It is stored by the system as client information and passed unchanged to the next movement callback for its use. In this manner the drag callback is able to maintain custom state from movement to movement.

{*}cmd done *canvas clientdata*
This form is invoked when the drag operation ends. It is invoked with the client data from the last movement callback (or start callback if there had been no motion).

The result of the command is ignored.

Options

The commands to create drag operations (**on**, **item**, and **group**) all accept the following options to configure the new drag.

-event *spec*
The value of this option specifies the mouse button used to initiate the drag operation, and the keyboard modifier, if any. Examples of specifications:

To initiate a drag operation by pressing mouse button 3 on a drag handle, use:

```
-event 3
```

This is the default as well, if the option is not specified.

To initiate a drag operation by pressing mouse button 2 on a drag handle while holding down the Control key, use:

```
-event Control-2
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems.

Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_ecircle

Keywords: canvas_ecircle, canvas, circle, editing

Name

canvas::edit::circle - Editing a circle on a canvas

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::circle ?**0.1**?
::canvas::edit circle *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName set *minx miny maxx maxy*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd *editorObj coordinates*

Description

This package provides a class whose instances handle editing a circle on a canvas. Instances can be configured with regard to the visual appearance of the markers denoting the circle's principal points. Note that instances do not store the edited circle themselves, but delegate this to a configurable object.

Interaction

The circle editor internally uses an instance of package **canvas::edit::points**, and inherits all the options and behaviours for it. The color options are additionally applied to the circle element of this editor

Beyond that it provides an option, **-circle-config**, to customize the circle item used to render the circle through its principal points. Note that it cannot overide the inherited color options.

During interaction instances execute a state machine where the user creates two points, at which point the system completes the circle and reports it. At that point the system prevents adding more points. Dragging the points is still possible, enabling resizing and moving the circle. Deleting any of the principal points deletes the entire circle, at which point the system is again ready for the creation of a new circle from two points.

Class API

::canvas::edit circle *objectName canvas options*...

This, the class command, creates and configures a new instance of a circle editor, named

objectName. The instance will be connected to the specified *canvas* widget.

The result of the command is the fully qualified name of the instance command.

The options accepted here, and their values, are explained in the section Options.

Instance API

Instances of the circle editors provide the following API:

objectName destroy

This method destroys the circle editor and releases all its internal resources.

Note that this operation does not destroy the items the editor managed on the attached canvas, nor the canvas itself.

The result of the method is an empty string.

objectName enable

This method activates editing of the circle on the canvas. This is the default after instance creation. A call is ignored if the editor is already active.

The result of the method is an empty string.

The complementary method is **disable**. The interogatory method for the current state is **active**.

objectName disable

This method disables editing of the circle on the canvas. A call is ignored if the editor is already disabled.

The result of the method is an empty string.

The complementary method is **enable**. The interogatory method for the current state is **active**.

objectName active

This method queries the editor state.

The result of the method is a boolean value, **true** if the editor is active, and **false** otherwise, i.e. disabled.

The methods to change the state are **enable** and **disable**.

objectName set *minx miny maxx maxy*

This method programmatically set the circle from the border coordinates. This writes over a previous (possibly partial) circle.

The result of the method is an empty string.

Note that this method goes through the whole set of callbacks invoked when the principal

points are created, i.e. **-create-cmd**, and, more importantly, **-data-cmd**.

This is the method through which to load a pre-existing circle into an editor instance.

objectName clear

This method programmatically removes the circle from the editor, making the way free for creating a new one.

The result of the method is an empty string.

Note that this method goes through the same callbacks invoked when the user interactively removes a principal point, i.e. **-data-cmd**.

Options

The class command accepts the following options

-active-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the mouse enters or leaves the circle or one of the principal points managed by the instance.

If not specified it defaults to an empty command which does nothing.

The signature of this command prefix is

activeCmd *editorObj id*

The *id* identifies the entered object.

An empty *id* indicates that the last entered object was left.

The value **circle** indicates that the circle's outline was entered.

Any other value is a code for one of the circle's principal points, i.e. **center**, and **edge**.

The result of this method is ignored.

-add-remove-point *eventspec*

The value of this option is an event specification (without bracketing angles) declaring which event will trigger adding and removing a point.

This option can only be set at construction time.

The default setup uses different events for adding and removing points, **ButtonPress-1** and **ButtonPress-2** respectively.

When using this option the same event is used for both operations. This is no problem because adding is bound as canvas-global event while removal is bound to the relevant canvas items.

-circle-config *list*

The value of this option is a list of options and their values to be applied to the internal circle item whenever it is regenerated after changes. Note that these options *cannot* override the

principal coloring of the circle as applied by the main options **-color** and **-hilit-color**.

If not specified it defaults to the empty list.

-color *colorspec*

The value of this option is the fill color for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **SkyBlue2**.

-create-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when it has to create a new principal point.

While this option can be set after construction, it is recommended to use this feature only as a means of inserting custom processing to be done at creation time which remembers and calls the previous value of the option.

If not specified it defaults to a command which will create a black-bordered blue circle of radius 3 centered on the location of the new point.

The signature of this command prefix is

createCmd *canvas x y*

The result of the command prefix *must* be a list of the canvas items it created to represent the marker. Note here that the visual representation of a "principal point" may consist of multiple canvas items in an arbitrary shape.

The returned list of items is allowed to be empty, and such is taken as signal that the callback vetoed the creation of the principal point.

-data-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the circle was edited in some way (created, modified, deleted). This is how the editor delegates the actual storage of the circle information to an outside object.

This option can only be set at construction time.

If not specified it defaults to an empty string and is ignored by the editor, i.e. not invoked.

The signature of this command prefix is

dataCmd *editorObj coordinates*

This callback is invoked when the circle was changed either interactively, or programmatically. See instance method **set** for the latter.

The *editorObj* identifies the instance invoking the callback, whereas *coordinates* is a list of 4 elements, containing the min x, and y, and the max x and y coordinates of the circle, in this order.

The result of this method is ignored.

-drag-point *eventspec*

The value of this option is an event specification (without bracketing angles) declaring which

event will trigger a drag action on points.

This option can only be set at construction time.

The default specification is **ButtonPress-3**.

-hilit-color ***colorspec***
The value of this option is the highlight color for the default item created when adding a point, and no **-highlight-cmd** is specified.

The default value is **red**.

-kind ***name***
The value of this option is the canvas item type for the default item created when adding a point, and no **-create-cmd** is specified. Only item types specified through a bounding box are suitable.

The default value is **oval**.

-radius ***int***
The value of this option is the radius for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **3**.

-tag ***string***
The value of this option is the name of the canvas tag with which to identify all items of all vertices managed by the editor.

This option can only be set at construction time.

If not specified it defaults to **CIRCLE**

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_epoints

Keywords: canvas_epoints, canvas, editing, point cloud, points

Name

canvas::edit::points - Editing a cloud of points on a canvas

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.3**?
::canvas::edit points *objectName canvas options...*
objectName destroy

objectName enable
objectName disable
objectName active
objectName add *x y*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd add *editorObj id x y*
dataCmd remove *editorObj id*
dataCmd move start *editorObj id*
dataCmd move delta *editorObj id x y dx dy*
dataCmd move done *editorObj id*

Description

This package provides a class whose instances handle editing a cloud of point markers on a canvas. Instances can be configured with regard to the visual appearance of markers. Note that instances do not store the edited points themselves, but delegate this to a configurable object.

Interaction

The default bindings enable the creation, removal and dragging of point (markers) using the mouse buttons 1, 2, and 3. Specifically:

1. Clicking anywhere with button 1 adds a new point.
2. Clicking on a point with button 2 removes that point.
3. Clicking on a point with button 3 starts a drag operation ending with the release of the button.

The option **-drag-point** can be used to change the button used to initiate drag operations. Note that the option value may not only contain the button number, but also modifier prefix separated from the button by a dash "**-**". The recognized modifiers are **Control**, **Shift**, and **Alt**.

The option **-add-remove-point** can be used to change the button used to add/remove points. This works because removal is an item binding, while adding is canvas global. The option recognizes the same syntax for button as **-drag-point**.

In the standard configuration the canvas item representing a point is a circle configured using the appearance options **-color**, **-hilit-color**, and **-radius**. Their defaults are **Skyblue2**, **red**, and **3**, repsectively.

The option **-kind** can be used to slightly customize the canvas item to any kind accepting the item options **-width**, **-outline**, **-fill**, and **-activefill**.

For more extensive customization its is necessary to configure a creation callback using option **-create-cmd**. The callback takes the canvas and x. y coordinates as arguments, in this order, and is expected to return a list of the canvas items representing the point.

Note the plural. With the callback it is possible to visualize a point using a group of items.

Further note that when the callback return an empty list no point is created. In other words, the callback can also be used to perform checks if a point at the given location is desirable, or not. This can be combined with the default creation behaviour as the option can be queried, and its default value is the callback for the default behaviour.

Keeping with the callback for a bit, the option **-active-cmd** sets a callback invoked whenever

the mouse is over one of the created points, i.e. active. This can be used to drive derived displays. For example highlighting the point's entry in a table or other widget.

The last option of importance is **-tag**. Its value is the name of the canvas tag used to mark all the items owned, i.e. created and managed by an. The default is **POINT**.

When attaching multiple instances of this behaviour to the same canvas each instance has to be configured with a unique tag, to prevent them from interfering with each other.

Class API

::canvas::edit points *objectName canvas options*...

This, the class command, creates and configures a new instance of a point cloud editor, named *objectName*. The instance will be connected to the specified *canvas* widget.

The result of the command is the fully qualified name of the instance command.

The options accepted here, and their values, are explained in the section Options.

Instance API

Instances of the point cloud editors provide the following API:

objectName destroy

This method destroys the point cloud editor and releases all its internal resources.

Note that this operation does not destroy the items of the point markers the editor managed on the attached canvas, nor the canvas itself.

The result of the method is an empty string.

objectName enable

This method activates editing of the point cloud on the canvas. This is the default after instance creation. A call is ignored if the editor is already active.

The result of the method is an empty string.

The complementary method is **disable**. The interogatory method for the current state is **active**.

objectName disable

This method disables editing of the point cloud on the canvas. A call is ignored if the editor is already disabled.

The result of the method is an empty string.

The complementary method is **enable**. The interogatory method for the current state is **active**.

objectName active

This method queries the editor state.

The result of the method is a boolean value, **true** if the editor is active, and **false** otherwise,

i.e. disabled.

The methods to change the state are **enable** and **disable**.

objectName add *x y*

This method programmatically creates a point at the specified location.

The result of the method is an empty string.

Note that this method goes through the whole set of callbacks invoked when the user interactively creates a point, i.e. **-create-cmd**, and, more importantly, **-data-cmd**.

This is the method through which to load pre-existing points into an editor instance.

objectName clear

This method programmatically removes all points from the editor.

The result of the method is an empty string.

Note that this method goes through the same callback invoked when the user interactively removes a point, i.e. **-data-cmd**.

Options

The class command accepts the following options

-active-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the mouse enters or leaves a point managed by the instance.

If not specified it defaults to an empty command which does nothing.

The signature of this command prefix is

activeCmd *editorObj id*

The *id* identifies the point within the editor.

An empty *id* indicates that the last entered point was left.

The result of this method is ignored.

-add-remove-point *eventspec*

The value of this option is an event specification (without bracketing angles) declaring which event will trigger adding and removing a point.

This option can only be set at construction time.

The default setup uses different events for adding and removing points, **ButtonPress-1** and **ButtonPress-2** respectively.

When using this option the same event is used for both operations. This is no problem because adding is bound as canvas-global event while removal is bound to the relevant

canvas items.

-color *colorspec*

The value of this option is the fill color for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **SkyBlue2**.

-create-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when it has to create a new point.

While this option can be set after construction, it is recommended to use this feature only as a means of inserting custom processing to be done at creation time which remembers and calls the previous value of the option.

If not specified it defaults to a command which will create a black-bordered blue circle of radius 3 centered on the location of the new point.

The signature of this command prefix is

createCmd *canvas x y*

The result of the command prefix *must* be a list of the canvas items it created to represent the marker. Note here that the visual representation of a "point" may consist of multiple canvas items in an arbitrary shape.

The returned list of items is allowed to be empty, and such is taken as signal that the callback vetoed the creation of the point.

-data-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when a point was edited in some way. This is how the editor delegates the actual storage of point information to an outside object.

This option can only be set at construction time.

If not specified it defaults to an empty string and is ignored by the editor, i.e. not invoked.

The signatures of this command prefix are

dataCmd add *editorObj id x y*

This callback is invoked when a new point was added to the instance, either interactively, or programmatically. See instance method **add** for the latter.

The *id* identifies the point within the editor and will be used by the two other callbacks to specify which point to modify.

The last two arguments *x* and *y* specify the location of the new point in canvas coordinates.

The result of this method is ignored.

dataCmd remove *editorObj id*

This callback is invoked when a point removed from the editor instance.

The *id* identifies the removed point within the editor.

The result of this method is ignored.

dataCmd move start *editorObj id*

This callback is invoked when the movement of a point in the editor instance has started.

The *id* identifies the point within the editor about to be moved.

The result of this method is ignored.

dataCmd move delta *editorObj id x y dx dy*

This callback is invoked when the point moved in the editor instance.

The *id* identifies the moved point within the editor, and the remaining arguments *x*, *y*, *dx*, and *dy* provide the new absolute location of the point, and full delta to the original location.

At the time of the calls the system is *not* committed to the move yet. Only after method **move done** was invoked and has accepted or rejected the last position will the editor update its internal data structures, either committing to the new location, or rolling the move back to the original one.

Given this the location data provided here should be saved only in temporary storage until then.

The result of this method is ignored.

dataCmd move done *editorObj id*

This callback is invoked when the movement of a point in the editor instance is complete.

The *id* identifies the moved point within the editor.

The result of this method must be a boolean value. If the method returns **false** the move is vetoed and rollbed back.

-drag-point *eventspec*

The value of this option is an event specification (without bracketing angles) declaring which event will trigger a drag action on points.

This option can only be set at construction time.

The default specification is **ButtonPress-3**.

-hilit-color *colorspec*

The value of this option is the highlight color for the default item created when adding a point, and no **-highlight-cmd** is specified.

The default value is **red**.

-kind *name*

The value of this option is the canvas item type for the default item created when adding a point, and no **-create-cmd** is specified. Only item types specified through a bounding box are

suitable.

The default value is **oval**.

-radius ***int***
The value of this option is the radius for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **3**.

-tag ***string***
The value of this option is the name of the canvas tag with which to identify all items of all points managed by the editor.

This option can only be set at construction time.

If not specified it defaults to **POINT**

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_epolyline

Keywords: canvas_epolyline, canvas, editing, polyline

Name

canvas::edit::polyline - Editing a polyline on a canvas

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::polyline ?**0.2**?
::canvas::edit polyline *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName add *x y*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd *editorObj coordinates*

Description

This package provides a class whose instances handle editing a single poly-line (or -gon) on a canvas. Instances can be configured with regard to the visual appearance of the markers

denoting the line's vertices, as well as if the line is to be closed or not, and further if a closed line has to be convex or not. Note that instances do not store the edited polyline themselves, but delegate this to a configurable object.

Interaction

The polyline editor internally uses an instance of package **canvas::edit::points**, and inherits all the options and behaviours for it. The color options are additionally applied to the line element of this editor

Beyond that it provides 2 options to configure the editing mode, and one option to customize the line/polygon item used to render the line through the points.

The main editing mode is controlled by option **-closed** for *line* (i.e. open) versus *polygon* (i.e. closed). The default is line/open mode.

In polygon/closed mode the option **-convex** controls if the editor enforces convexity of the polygon, or not. The default is no enforcement of convexity.

The option **-line-config** enables basic customization of the line/polygon item. Note that it cannot overide the inherited color options.

Class API

::canvas::edit polyline *objectName canvas options*...

This, the class command, creates and configures a new instance of a polyline editor, named *objectName*. The instance will be connected to the specified *canvas* widget.

The result of the command is the fully qualified name of the instance command.

The options accepted here, and their values, are explained in the section Options.

Instance API

Instances of the polyline editors provide the following API:

objectName destroy

This method destroys the polyline editor and releases all its internal resources.

Note that this operation does not destroy the items the editor managed on the attached canvas, nor the canvas itself.

The result of the method is an empty string.

objectName enable

This method activates editing of the polyline on the canvas. This is the default after instance creation. A call is ignored if the editor is already active.

The result of the method is an empty string.

The complementary method is **disable**. The interogatory method for the current state is **active**.

objectName disable

This method disables editing of the polyline on the canvas. A call is ignored if the editor is

already disabled.

The result of the method is an empty string.

The complementary method is **enable**. The interogatory method for the current state is **active**.

objectName active

This method queries the editor state.

The result of the method is a boolean value, **true** if the editor is active, and **false** otherwise, i.e. disabled.

The methods to change the state are **enable** and **disable**.

objectName add *x y*

This method programmatically adds a vertex at the specified location to the line.

The result of the method is an empty string.

Note that this method goes through the whole set of callbacks invoked when the user interactively creates a vertex, i.e. **-create-cmd**, and, more importantly, **-data-cmd**.

One important difference however. The new vertex is always added at the end of the line, whereas interactive creation uses heuristics to splice it into the line at a suitable location.

This is the method through which to load the vertices of a pre-existing line into an editor instance.

objectName clear

This method programmatically removes all vertices from the editor, essentially removing the whole line.

The result of the method is an empty string.

Note that this method goes through the same callbacks invoked when the user interactively removes a vertex, i.e. **-data-cmd**.

Options

The class command accepts the following options

-active-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the mouse enters or leaves the line or a point managed by the instance.

If not specified it defaults to an empty command which does nothing.

The signature of this command prefix is

activeCmd *editorObj id*

The *id* identifies the entered object.

An empty *id* indicates that the last entered object was left.

The value **line** indicates that the line was entered.

A numeric value **N** indicates that the **N**'th vertex of the line was entered, counting from **0**.

The result of this method is ignored.

-add-remove-point *eventspec*
The value of this option is an event specification (without bracketing angles) declaring which event will trigger adding and removing a point.

This option can only be set at construction time.

The default setup uses different events for adding and removing points, **ButtonPress-1** and **ButtonPress-2** respectively.

When using this option the same event is used for both operations. This is no problem because adding is bound as canvas-global event while removal is bound to the relevant canvas items.

-closed *boolean*
The value of this option is a boolean flag indicating if the edited polyline is closed, i.e. a loop, or not. *NOTE* that a proper loop requires at least 3 points. For less than that an open line is shown, or even just a point.

This option can only be set at construction time.

If not specified it defaults to **false**

-color *colorspec*
The value of this option is the fill color for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **SkyBlue2**.

-convex *boolean*
The value of this option is a boolean flag indicating if the edited polyline is strictly convex, or not. This option has meaning if and only if option **-closed** is configured to be **true**.

This option can only be set at construction time.

If not specified it defaults to **false**

-create-cmd *command-prefix*
The value of this option is a command prefix the editor will invoke when it has to create a new vertex.

While this option can be set after construction, it is recommended to use this feature only as a means of inserting custom processing to be done at creation time which remembers and calls

the previous value of the option.

If not specified it defaults to a command which will create a black-bordered blue circle of radius 3 centered on the location of the new point.

The signature of this command prefix is
createCmd *canvas x y*
The result of the command prefix *must* be a list of the canvas items it created to represent the marker. Note here that the visual representation of a "vertex" may consist of multiple canvas items in an arbitrary shape.

The returned list of items is allowed to be empty, and such is taken as signal that the callback vetoed the creation of the vertex.

-data-cmd *command-prefix*
The value of this option is a command prefix the editor will invoke when the line was edited in some way (vertex added, removed, moved). This is how the editor delegates the actual storage of the line information to an outside object.

This option can only be set at construction time.

If not specified it defaults to an empty string and is ignored by the editor, i.e. not invoked.

The signature of this command prefix is
dataCmd *editorObj coordinates*
This callback is invoked when the line was changed either interactively, or programmatically. See instance method **add** for the latter.

The *editorObj* identifies the instance invoking the callback, whereas *coordinates* is a list of vertex locations, with each location a pair of x- and y-coordinates.

The result of this method is ignored.

-drag-point *eventspec*
The value of this option is an event specification (without bracketing angles) declaring which event will trigger a drag action on points.

This option can only be set at construction time.

The default specification is **ButtonPress-3**.

-hilit-color *colorspec*
The value of this option is the highlight color for the default item created when adding a point, and no **-highlight-cmd** is specified.

The default value is **red**.

-kind *name*
The value of this option is the canvas item type for the default item created when adding a point, and no **-create-cmd** is specified. Only item types specified through a bounding box are suitable.

The default value is **oval**.

-line-config ***list***
: The value of this option is a list of options and their values to be applied to the internal line/polygon item whenever it is regenerated after changes. Note that these options *cannot* override the principal coloring of the line/polygon as applied by the main options **-color** and **-hilit-color**.

  If not specified it defaults to the empty list.

-radius ***int***
: The value of this option is the radius for the default item created when adding a point, and no **-create-cmd** is specified.

  The default value is **3**.

-tag ***string***
: The value of this option is the name of the canvas tag with which to identify all items of all vertices managed by the editor.

  This option can only be set at construction time.

  If not specified it defaults to **POLYLINE**

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_equad

Keywords: canvas_equad, canvas, concave, convex, editing, non-convex, quadrilateral

Name

canvas::edit::quadrilateral - Editing a quadrilateral on a canvas

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::quadrilateral ?**0.2**?
::canvas::edit quadrilateral *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName add *x y*
objectName clear
createCmd *canvas x y*

highlightCmd on *canvas item*
highlightCmd off *canvas state*
dataCmd *editorObj coordinates*

Description

This package provides a class whose instances handle the editing of a quadrilateral on a canvas. Instances can be configured with regard to the visual appearance (regular, and highlighted) of vertex markers. Note that instances do not store the edited quadrilateral themselves, but delegate this to a configurable object.

Class API

::canvas::edit quadrilateral *objectName canvas options*...

This, the class command, creates and configures a new instance of a quadrilateral editor, named *objectName*. The instance will be connected to the specified *canvas* widget.

The result of the command is the fully qualified name of the instance command.

The options accepted here, and their values, are explained in the section Options.

Instance API

Instances of the quadrilateral editors provide the following API:

objectName destroy

This method destroys the quadrilateral editor and releases all its internal resources.

Note that this operation does not destroy the items the editor managed on the attached canvas, nor the canvas itself.

The result of the method is an empty string.

objectName enable

This method activates editing of the quadrilateral on the canvas. This is the default after instance creation. A call is ignored if the editor is already active.

The result of the method is an empty string.

The complementary method is **disable**. The interogatory method for the current state is **active**.

objectName disable

This method disables editing of the quadrilateral on the canvas. A call is ignored if the editor is already disabled.

The result of the method is an empty string.

The complementary method is **enable**. The interogatory method for the current state is **active**.

objectName active

This method queries the editor state.

The result of the method is a boolean value, **true** if the editor is active, and **false** otherwise, i.e. disabled.

The methods to change the state are **enable** and **disable**.

objectName add *x y*

This method programmatically adds a vertex at the specified location to the quadrilateral.

The result of the method is an empty string.

Note that this method goes through the whole set of callbacks invoked when the user interactively creates a vertex, i.e. **-create-cmd**, and, more importantly, **-data-cmd**.

This is the method through which to load the vertices of a pre-existing quadrilateral into an editor instance.

Note that at most 4 vertices can be specified, and at least 4 must be specified for the quadrilateral to be complete.

objectName clear

This method programmatically unset the quadrilateral in the editor.

The result of the method is an empty string.

Note that this method goes through the same callback invoked when the user interactively removes a vertex, i.e. **-data-cmd**.

Options

The class command accepts the following options

-convex *boolean*

The value of this option is a boolean flag. When the flag is set the editor will accept only convex quadrilaterals and reject all operations which would result in a violation of convexity.

This option can only be set at construction time.

If not specified it defaults to **false**, i.e. acceptance of any quadrilateral.

-tag *string*

The value of this option is the name of the canvas tag with which to identify all items of all quadrilateral managed by the editor.

This option can only be set at construction time.

If not specified it defaults to **QUADRILATERAL**

-create-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when it has to create a new vertex.

This option can only be set at construction time.

If not specified it defaults to a command which will create a black-bordered blue circle of radius 3 centered on the location of the new point.

The signature of this command prefix is

createCmd *canvas x y*

The result of the command prefix *must* be a list of the canvas items it created to represent the marker. Note here that the visual representation of a "vertex" may consist of multiple canvas items in an arbitrary shape.

-highlight-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when it has to (un)highlight a vertex.

This option can only be set at construction time.

If not specified it defaults to a command which will re-color the item to highlight in red (and restores the color for unhighlighting).

The two signatures of this command prefix are

highlightCmd on *canvas item*

This method of the command prefix has to perform whatever is needed to highlight the vertex the *item* is a part of (remember the note above about quadrilateral allowed to be constructed from multiple canvas items).

The result of the command can be anything and will be passed as is as argument *state* to the **off** method.

highlightCmd off *canvas state*

This method is invoked to unhighlight a vertex described by the *state*, which is the unchanged result of the **on** method of the command prefix. The result of this method is ignored.

Note any interaction between dragging and highlighting of quadrilateral is handled within the editor, and that the callback bears no responsibility for doing such.

-data-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the quadrilateral was edited in some way. This is how the editor delegates the actual storage of quadrilateral information to an outside object.

This option can only be set at construction time.

If not specified it defaults to an empty string and is ignored by the editor, i.e. not invoked.

The signatures of this command prefix are

dataCmd *editorObj coordinates*

This callback is invoked when the quarilateral was changed either interactively, or programmatically. See instance method **add** for the latter.

The *editorObj* identifies the instance invoking the callback, whereas *coordinates* is a list of

vertex locations, with each location a pair of x- and y-coordinates.

The result of this method is ignored.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_erectangle

Keywords: canvas_erectangle, canvas, editing, rectangle

Name

canvas::edit::rectangle - Editing a rectangle on a canvas

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::edit::points ?**0.1**?
package require canvas::edit::rectangle ?**0.1**?
::canvas::edit rectangle *objectName canvas options...*
objectName destroy
objectName enable
objectName disable
objectName active
objectName set *minx miny maxx maxy*
objectName clear
activeCmd *editorObj id*
createCmd *canvas x y*
dataCmd *editorObj coordinates*

Description

This package provides a class whose instances handle editing a rectangle on a canvas. Instances can be configured with regard to the visual appearance of the markers denoting the rectangle's corners. Note that instances do not store the edited rectangle themselves, but delegate this to a configurable object.

Interaction

The rectangle editor internally uses an instance of package **canvas::edit::points**, and inherits all the options and behaviours for it. The color options are additionally applied to the rectangle element of this editor

Beyond that it provides an option, **-rect-config**, to customize the rectangle item used to render the rectangle through its corners. Note that it cannot overide the inherited color options.

During interaction instances execute a state machine where the user creates two points, at which point the system completes the rectangle with the missing corners and reports it. At that point the system prevents adding more points. Dragging the points is still possible, enabling

resizing and moving the rectangle. Deleting any corner deletes the entire rectangle, at which point the system is again ready for the creation of a new rectangle from two points.

Class API

::canvas::edit rectangle *objectName canvas options...*

This, the class command, creates and configures a new instance of a rectangle editor, named *objectName*. The instance will be connected to the specified *canvas* widget.

The result of the command is the fully qualified name of the instance command.

The options accepted here, and their values, are explained in the section Options.

Instance API

Instances of the rectangle editors provide the following API:

objectName destroy

This method destroys the rectangle editor and releases all its internal resources.

Note that this operation does not destroy the items the editor managed on the attached canvas, nor the canvas itself.

The result of the method is an empty string.

objectName enable

This method activates editing of the rectangle on the canvas. This is the default after instance creation. A call is ignored if the editor is already active.

The result of the method is an empty string.

The complementary method is **disable**. The interogatory method for the current state is **active**.

objectName disable

This method disables editing of the rectangle on the canvas. A call is ignored if the editor is already disabled.

The result of the method is an empty string.

The complementary method is **enable**. The interogatory method for the current state is **active**.

objectName active

This method queries the editor state.

The result of the method is a boolean value, **true** if the editor is active, and **false** otherwise, i.e. disabled.

The methods to change the state are **enable** and **disable**.

objectName set *minx miny maxx maxy*

This method programmatically set the rectangle from the border coordinates. This writes over

a previous (possibly partial) rectangle.

The result of the method is an empty string.

Note that this method goes through the whole set of callbacks invoked when corners are created, i.e. **-create-cmd**, and, more importantly, **-data-cmd**.

This is the method through which to load a pre-existing rectangle into an editor instance.

objectName clear

This method programmatically removes the rectangle from the editor, making the way free for creating a new one.

The result of the method is an empty string.

Note that this method goes through the same callbacks invoked when the user interactively removes a corner, i.e. **-data-cmd**.

Options

The class command accepts the following options

-active-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the mouse enters or leaves the rectangle or a corner managed by the instance.

If not specified it defaults to an empty command which does nothing.

The signature of this command prefix is

activeCmd *editorObj id*

The *id* identifies the entered object.

An empty *id* indicates that the last entered object was left.

The value **rect** indicates that the rectangle's outline was entered.

Any other value is a code for one of the rectangle's corner, i.e. **tl**, **tr**, **bl**, and **br**.

The result of this method is ignored.

-add-remove-point *eventspec*

The value of this option is an event specification (without bracketing angles) declaring which event will trigger adding and removing a point.

This option can only be set at construction time.

The default setup uses different events for adding and removing points, **ButtonPress-1** and **ButtonPress-2** respectively.

When using this option the same event is used for both operations. This is no problem

because adding is bound as canvas-global event while removal is bound to the relevant canvas items.

-color *colorspec*

The value of this option is the fill color for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **SkyBlue2**.

-create-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when it has to create a new corner.

While this option can be set after construction, it is recommended to use this feature only as a means of inserting custom processing to be done at creation time which remembers and calls the previous value of the option.

If not specified it defaults to a command which will create a black-bordered blue circle of radius 3 centered on the location of the new point.

The signature of this command prefix is

createCmd *canvas x y*

The result of the command prefix *must* be a list of the canvas items it created to represent the marker. Note here that the visual representation of a "corner" may consist of multiple canvas items in an arbitrary shape.

The returned list of items is allowed to be empty, and such is taken as signal that the callback vetoed the creation of the corner.

-data-cmd *command-prefix*

The value of this option is a command prefix the editor will invoke when the rectangle was edited in some way (created, modified, deleted). This is how the editor delegates the actual storage of the rectangle information to an outside object.

This option can only be set at construction time.

If not specified it defaults to an empty string and is ignored by the editor, i.e. not invoked.

The signature of this command prefix is

dataCmd *editorObj coordinates*

This callback is invoked when the rectangle was changed either interactively, or programmatically. See instance method **set** for the latter.

The *editorObj* identifies the instance invoking the callback, whereas *coordinates* is a list of 4 elements, containing the min x, and y, and the max x and y coordinates of the rectangle, in this order.

The result of this method is ignored.

-drag-point *eventspec*

The value of this option is an event specification (without bracketing angles) declaring which event will trigger a drag action on points.

This option can only be set at construction time.

The default specification is **ButtonPress-3**.

-hilit-color *colorspec*
The value of this option is the highlight color for the default item created when adding a point, and no **-highlight-cmd** is specified.

The default value is **red**.

-kind *name*
The value of this option is the canvas item type for the default item created when adding a point, and no **-create-cmd** is specified. Only item types specified through a bounding box are suitable.

The default value is **oval**.

-radius *int*
The value of this option is the radius for the default item created when adding a point, and no **-create-cmd** is specified.

The default value is **3**.

-rect-config *list*
The value of this option is a list of options and their values to be applied to the internal rectangle item whenever it is regenerated after changes. Note that these options *cannot* override the principal coloring of the rectangle as applied by the main options **-color** and **-hilit-color**.

If not specified it defaults to the empty list.

-tag *string*
The value of this option is the name of the canvas tag with which to identify all items of all vertices managed by the editor.

This option can only be set at construction time.

If not specified it defaults to **RECTANGLE**

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_gradient

Keywords: canvas_gradient, background, canvas, color gradient, gradients, image, widget background

Name

canvas::gradient - Canvas with a gradient background

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::gradient ?**0.2**?
::canvas::gradient *pathName* ?options?...

Description

This package provides a command to set background gradients on canvases.

NOTE! This package reserves the canvas tag **canvas::gradient** for its use.

API

::canvas::gradient *pathName* ?options?...

Creates a background gradient on the canvas *pathName*. An existing gradient is replaced by the new definition.

The set of available options is decribed below, in section Options.

Options

-direction *direction*

The value for this option specifies the *direction* of the gradient. If not specified the system defaults to **x**, a horizontal gradient. The package supports five directions:

x
for a horizontal gradient (default). Color changes along the x-axis, with the start color at the left.

y
for a vertical gradient. Color changes along the y-axis, with the start color at the top.

d1
for a diagonal gradient from left+top to bottom+right. Color changes along the diagonal. with the start color at left+top.

d2
for a diagonal gradient from left+bottom to top+right. Color changes along the diagonal. with the start color at left+bottom.

r
for a radial gradient around the center of the viewport Color changes from the center outward, with the start color at the center.

-color1 *color*

The value for this option specifies the starting color of the gradient. If not specified it defaults to **red**.

-color2 *color*

The value for this option specifies the ending color of the gradient. If not specified it defaults to **green**.

-type *type*

The value for this option specifies how the gradient is calculated. The package currently supports on ly one method, which is the default, naturally:
linear

for linear interpolation between the start and end colors (default).

Example

```
package require canvas::gradient
canvas .c
canvas::gradient .c -direction x -color1 yellow -color2 blue
pack .c -fill both -expand 1
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) 2013 Jarek Lewandowski, MaxJarek
Copyright (c) Origin http://wiki.tcl-lang.org/6100
Copyright (c) Origin http://wiki.tcl-lang.org/37242
Copyright (c) Origin http://wiki.tcl-lang.org/9079
```


canvas_highlight

Keywords: canvas_highlight, canvas, enter callback, highlighting, leave callback

Name

canvas::highlight - Manage the highlighting of canvas items or item groups

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::highlight ?**0.1**?
::canvas::highlight on *canvas tagOrId cmd*
::canvas::highlight off *canvas tagOrId*
{*}cmd on *canvas item*
{*}cmd off *canvas clientdata*

Description

This package provides utility commands for setting up and tearing down of highlights for canvas items or item groups, the latter identified by a tag.

API

::canvas::highlight on *canvas tagOrId cmd*

This command sets up a general highlight, with the items of canvas *canvas* to highlight in this manner identified by *tagOrId* and the *cmd* prefix providing the implementation, i.e. the how to perform the highlight.

The signature of the command prefix is described later, in section Highlight callback.

The result of the command is the empty string.

Limitations:

1. When a highlight is active no other highlight can be activated. This means that nested highlights are not possible.
2. The system may break if a highlight is removed from within its highlight callback.

::canvas::highlight off *canvas tagOrId*

This command removes any highlight set on the items of canvas *canvas* identified by *tagOrId*.

The result of the command is the empty string.

Highlight callback

The highlight callback is a command prefix invoked in the following two ways:

{*}cmd on *canvas item*

This form is invoked when the mouse has entered (one of) the item(s) the highlight was set up for. The callback now has to perform any reconfiguration necessary to highlight the item (group).

The result of the command can be anything. It is stored by the system as client information and passed unchanged to the un-highlight callback for its use. In this manner the highlight callback is able to maintain custom state from highlighting to un-highlighting.

Note that the callback does not have to maintain state, nor does it have to actually reconfigure the item (group). In the latter case the callback simply serves as easy enter/leave notification.

{*}cmd off *canvas clientdata*

This form is invoked when the mouse has left (one of) the item(s) of the currently active the highlight. The callback now has to perform any reconfiguration necessary to un-highlight the item (group).

The result of the command must be a boolean value with the usual value to be **true**. By returning **false** instead the callback can veto the removal of the highlight.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_mvg

Keywords: canvas_mvg, canvas, graphics, imagemagick, magick vector graphics, mvg, print screen, serialization, vector graphics

Name

canvas::mvg - Canvas to ImageMagick MVG vector format

Synopsis

package require Tcl 8.5

package require Tk 8.5
package require canvas::mvg ?**1**?
::canvas::mvg *pathName*

Description

This package provides a command to convert a canvas' contents to ImageMagick's MVG vector format.

API

::canvas::mvg *pathName*

Dump the contents of the canvas *pathName*. The result is a string in ImageMagick's MVG vector format.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) 2010 Wolf-Dieter Busch (http://wiki.tcl-lang.org/15505)
Copyright (c) 2010 Documentation, Andreas Kupries
```


canvas_pdf

Keywords: canvas_pdf, canvas, graphics, imagemagick, pdf, portable document format, print screen, serialization, vector graphics

Name

canvas::pdf - Canvas to PDF

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::pdf ?**1**?
package require pdf4tcl
::canvas::pdf *pathName*

Description

This package provides a command to convert a canvas' contents to Adobe's PDF vector format.

API

::canvas::pdf *pathName*

Dump the contents of the canvas *pathName*. The result is a string in Adobe's PDF.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright

```
Copyright (c) 2014 Andreas Kupries, Arjen Markus
Copyright (c) 2014 Documentation, Andreas Kupries
```

canvas_snap

Keywords: canvas_snap, canvas, image, photo, print screen, snapshot

Name

canvas::snap - Canvas snapshot to Tk photo image

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::snap ?**1.0.1**?
package require img::window
::canvas::snap *pathName*

Description

This package provides a command to take snapshots of arbitrary canvases.

API

::canvas::snap *pathName*

Takes a snapshot of the canvas *pathName*. The result is the name of a new Tk photo image containing the snapshot.

Note that this command has a side-effect. To avoid having white rectangles where other windows may overlap the canvas this command forces the toplevel window the canvas is in to the top of the stacking order.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright

```
Copyright (c) 2004 George Petasis (http://wiki.tcl-lang.org/1404)
Copyright (c) 2010 Documentation, Andreas Kupries
```

canvas_sqmap

Keywords: canvas_sqmap, canvas, cell, grid, image, map, square map, tile

Name

canvas::sqmap - Canvas with map background based on square tiles

Synopsis

package require Tcl 8.4

package require Tk 8.4
package require snit
package require uevent::onidle
package require cache::async
package require canvas::sqmap ?**0.3.1**?
::canvas::sqmap *pathName* ?*options*?
canvasName **image set** *cell image*
canvasName **image unset** *cell*
canvasName **flush**

Description

This package provides an extended canvas widget for the display of maps based on a set of square image tiles. The tiles are the background of the canvas, with all other canvas items added always shown in front of them. The number of tiles shown, tile size, and where to get the images to show are all configurable.

API

::canvas::sqmap *pathName* ?options?

Creates the canvas *pathName* and configures it. The new widget supports all of the options and methods of a regular canvas, plus the options and methods described below.

The result of the command is *pathName*.

Options

-grid-cell-width

The value for this option is a non-negative integer. It specifies the width of the cells the background is made up of.

-grid-cell-height

The value for this option is a non-negative integer. It specifies the height of the cells the background is made up of.

-grid-cell-command

The value for this option is a command prefix. It is invoked whenever the canvas needs the image for a specific cell of the background, with two additional arguments, the id of the cell, and a command prefix to invoke when the image is ready, or known to not exist.

The id of the cell is a 2-element list containing the row and column number of the cell, in this order. The result command prefix (named "$result" in the example below) has to be invoked with either two or three arguments, i.e.

```
    $result set   $cellid $image ; # image is known and ready
    $result unset $cellid        ; # image does not exist
```

This option may be left undefined, i.e. the canvas can operate without it. In that case the only images shown in grid cells are those explicitly set with the method **image set**, see the next section. All other grid cells will simply be empty.

-viewport-command

This option specifies a command prefix to invoke when the viewport of the canvas is changed,

to allow users keep track of where in the scroll-region we are at all times. This can be used, for example, to drive derivate displays, or to keep items in view by moving them as the viewport moves.

-image-on-load
The value for this option is an image. If specified the image is shown in a cell while the actual image for that cell is getting loaded through the callback specified by the **-grid-cell-command**.

-image-on-unset
The value for this option is an image. If specified the image is shown in a cell for which the callback specified by the **-grid-cell-command** reported that there is no actual image to be shown.

Methods

canvasName* image set *cell image
Invoking this method places the *image* into the specified *cell* of the background. The cell is given as a 2-element list containing row and column number, in this order.

Note that an image is allowed to be associated with and displayed in multiple cells of the canvas.

canvasName* image unset *cell
Invoking this method declares the specified *cell* of the background as empty, an existing image shown by this cell will be forgotten. The cell is given as a 2-element list containing row and column number, in this order.

***canvasName* flush**
Invoking this method forces the canvas to completely reload the images for all cells. Do not use this method if the canvas is operated without a **-grid-cell-command**, as in that case the canvas will simply forget all images without being able to reload them.

Image ownership

Note that the canvas *does not* take ownership of the images it shows in the background. In other words, when we say that the canvas forgets an image this means only that the association between a grid cell and shown image is broken. The image is *not* deleted. Managing the lifecycle of the images shown by the canvas is responsibility of the user of the canvas.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_tags

Keywords: canvas_tags, append tag, canvas, insert tag, remove tag, tags

Name

canvas::tag - Easier management of the tags on canvas items or item groups

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::tag ?**0.1**?
::canvas::tag append *canvas tagOrId tag...*
::canvas::tag prepend *canvas tagOrId tag...*
::canvas::tag append *canvas tagOrId index tag...*
::canvas::tag remove *canvas tagOrId tag...*
::canvas::tag match *canvas tagOrId pattern*

Description

This package provides utility commands for easier management of the tag lists associated with canvas items or item groups.

The problem with the existing canvas API is that a tag list can only be set as a whole, making adding and removing of tags from such lists relatively complex operations reading the current tag list, modifying it, and then writing the changed list back.

The commands provided by this package hide all this complexity from the user.

API

::canvas::tag append *canvas tagOrId tag...*
This command adds the tags *tag*... to the tag list for the items identified by the *tagOrId* in the *canvas* widget. The new tags are added at the end of the list.

The result of the command is the empty string.

::canvas::tag prepend *canvas tagOrId tag...*
This command adds the tags *tag*... to the tag list for the items identified by the *tagOrId* in the *canvas* widget. The new tags are added at the beginning of the list.

The result of the command is the empty string.

::canvas::tag append *canvas tagOrId index tag...*
This command adds the tags *tag*... to the tag list for the items identified by the *tagOrId* in the *canvas* widget. The new tags are inserted before the element at *index*.

index **0** refers to the beginning of the list.

index **end** refers to after the end of the list.

The result of the command is the empty string.

::canvas::tag remove *canvas tagOrId tag...*
This command removes the named tags *tag*... from the tag list for the items identified by the *tagOrId* in the *canvas* widget.

The result of the command is the empty string.

::canvas::tag match *canvas tagOrId pattern*
This command finds all tags for the items identified by the *tagOrId* in the *canvas* widget which match the glob *pattern*.

The result of the command is a list of the matching tags. Which may be empty.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_trlines

Keywords: canvas_trlines, canvas, crosshair, rubberband, tracking

Name

canvas::track::lines - Manage a group of rubber band lines

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require canvas::tag ?**0.1**?
::canvas::track lines *objectName canvas*
objectName destroy
objectName start *current p...*
objectName move *current*
objectName done

Description

This package provides a utility class managing the drawing of set of semi-crosshair (rubberband) lines.

Class API

::canvas::track lines *objectName canvas*

This, the class command, creates and configures a new instance of the tracker, named *objectName*. The instance will be connected to the specified *canvas* widget.

The result of the command is the fully qualified name of the instance command.

Instance API

Instances of this class provide the following API:

objectName destroy

This method destroys the instance and releases all its internal resources.

This operation does destroy the items representing the tracked lines. It does not destroy the attached canvas.

The result of the method is an empty string.

objectName start *current p...*

This method starts the tracking of a set of lines, one line per point *p*, which specifies the

destination end-point of each line. All lines will have *current* as a common end-point.

Note that a previously tracked set of lines is removed.

The result of the method is an empty string.

Each point is specified through a 2-element list containing its x- and y-coordinates, in this order.

objectName move *current*
This method updates the shared end-point of all rubberbands and redraws them.

The result of the method is an empty string.

The point is specified through a 2-element list containing its x- and y-coordinates, in this order.

objectName done
This method ends the tracking of the current set of lines and removes them from the canvas.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

canvas_zoom

Keywords: canvas_zoom, zoom

Name

canvas::zoom - Zoom control for canvas::sqmap

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require snit
package require uevent::onidle
package require canvas::zoom ?**0.2.1**?
::canvas::zoom *pathName* ?*options*?

Description

This package provides a widget to enable the user of a map display to control the zoom level.

API

::canvas::zoom *pathName* ?options?
Creates the zoom control widget *pathName* and configures it. The methods and options supported by the new widget are described in the following sections.

The result of the command is *pathName*.

Options

-orient
: The value for this option is either **vertical**, or **horizontal**, specifying the orientation of the major axis of the widget. The default is **vertical**.

-levels
: The value for this option is a non-negative integer. It specifies the number of zoom levels to support.

-variable
: The value for this option is the name of a global or namespaced variable which is connected with the widget. changes to the zoom level made the widget are propagated to this variable, and in turn changes to the variable are imported into the widget.

-command
: This option specifies a command prefix. This callback will be invoked whenever the zoom level is changed. It is called with two additional arguments, the zoom control widget, and the new zoom level, in this order.

Methods

The widget supports no methods beyond the standard (**configure**, **cget**, etc.).

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *canvas* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

chatwidget

chatwidget

Keywords: chatwidget, chat, composite widget, irc, mega-widget, widget

Name

chatwidget - Provides a multi-paned view suitable for display of chat room or irc channel information

Synopsis

package require Tk 8.5
package require chatwidget ?**1.1.4**?
::chatwidget::chatwidget *path* ?*options*?
$widget topic *command args*
$widget name *nick args*
$widget message *text args*
$widget hook *command args*
$widget names *args*
$widget entry *args*
$widget chat *args*

Description

This is a composite widget designed to simplify the construction of chat applications. The widget contains display areas for chat messages, user names and topic and an entry area. It automatically handles colourization of messages per nick and manages nick completion. A system of hooks permit the application author to adjust display features. The main chat display area may be split for use displaying history or for searching.

The widget is made up of a number of text widget and panedwindow widgets so that the size of each part of the display may be adjusted by the user. All the text widgets may be accessed via widget passthrough commands if fine adjustment is required. The topic and names sections can also be hidden if desired.

Commands

::chatwidget::chatwidget *path* ?*options*?

Create a new chatwidget using the Tk window id *path*. Any options provided are currently passed directly to the main chat text widget.

Widget commands

$widget topic *command args*

The chat widget can display a topic string, for instance the topic or name given to a multi-user chatroom or irc channel.

show

Enable display of the topic.

hide

Disable display of the topic

set *topic*

Set the topic text to *topic*.

$widget name *nick args*

Control the names and tags associated with names.

list ?*-full*?

Returns a list of all the user names from the names view. If ?-full? is given then the list returned is a list of lists where each sublist is made up of the nick followed by any options that have been set on this nick entry. This may be used to examine any application specific options that may be applied to a nick when using the **add** command.

- **add** *nick* ?*options*?
- **delete** *nick*

$widget message *text args*

Add messages to the display. options are -nick, -time, -type, -mark -tags

$widget hook *command args*

Manage hooks. add (message, post names_group, names_nick, chatstate), remove, run

$widget names *args*

Passthrough to the name display text widget. See the **text** widget manual for all available commands. The chatwidget provides two additional commands **show** and **hide** which are used to control the display of this element in the widget.

$widget entry *args*

Passthrough to the entry text widget. See the **text** widget manual for all available commands.

$widget chat *args*
Passthrough to the chat text widget. See the **text** widget manual for all available commands.

Example

```
chatwidget::chatwidget .chat
proc speak {w msg} {$w message $msg -nick user}
.chat hook add post [list speak .chat]
pack .chat -side top -fill both -expand 1
.chat topic show
.chat topic set "Chat widget demo"
.chat name add "admin" -group admin
.chat name add "user" -group users -color tomato
.chat message "Chatwidget ready" -type system
.chat message "Hello, user" -nick admin
.chat message "Hello, admin" -nick user
```

A more extensive example is available by examining the code for the picoirc program in the tclapps repository which ties the tcllib **picoirc** package to this **chatwidget** package to create a simple irc client.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *chatwidget* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

controlwidget

controlwidget

Keywords: controlwidget, controlling, displaying, numerical values, scale widget

Name

controlwidget - Collection of widgets for displaying and controlling numerical values

Synopsis

package require Tcl ?**8.5**?
package require Tk ?**8.5**?
package require snit ?**2.0**?
package require controlwidget ?**0.1**?
::controlwidget::meter *w args*
::controlwidget::slider *w args*
::controlwidget::equalizerBar *w args*
::controlwidget::thermometer *w args*
::controlwidget::voltmeter *w args*
::controlwidget::tachometer *w args*
::controlwidget::rdial *w args*
$widget get
$widget set *value*

::controlwidget::led *w args*
::controlwidget::radioMatrix *w args*
$led on
$led off
$led setstate *state*
$matrix get
$matrix set *index*

Description

The controlwidget package focuses on the display and interactive control of numerical values. It mimicks several of the meters and controls found in laboratory settings but also daily life: volt meters, equalizers and tachometers to name a few. They can be seen as alternatives for the *scale widget*.

Note: The package has not sofar been tested extensively, and that may result in an unexpected appearance if you use sizes that are different than the defaults. Also not all options for the coloring of the various parts and fonts and so on have been tested, so that may be another source of bugs.

A last note: some parts have not been included in any option, most notably the colors of parts that require lighter and darker shades to cooperate.

Types of widgets

The package distinguishes several typed of widgets:

- Vertical meters: the value of the variable is translated into a vertical position, like with the coloured bars you find on your stereo installation.
- Angle displays: the value of the variable is related to the angle of a needle, like with tachometers.
- Interactive widgets: most widgets allow you to change the value of the variable by pressing the mouse button on the needle and shifting it up and down or left and right.
- Non-interactive widgets: some widgets, like the thermometer widget, do not allow such interaction.
- Logical or choice widgets: some widgets display the values as either on/off (or true/false) or as one of a set of discrete choices.

All widgets have in common, however, that you can connect them to a variable and that changing the variable changes the display. Forthermore, all widgets have the set and get methods to interact with the value that the widget displays (whether that is stored in a global variable or not).

They also have in common that their appearance and behaviour is determined by one or more options that you can set at creation time and often later on as well. The widgets are all based on the *snit* package, so that the methods **configure** and **cget** are available to set and get these options.

Commands

Currently the package contains these widgets of the *vertical meter* type:

::controlwidget::meter *w args*

Create a vertical meter consisting of an axis and a moveable arrow.

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget:

variable name
Name of the variable to be associated with the meter

background color
Background color for the widget

height pixels
Height of the widget

width pixels
Width of the widget

arrowthickness pixels
Thickness of the displayed arrow in pixels

arrowcolor color
Color of the displayed arrow

readonly boolean
Whether the arrow can be moved interactively or not

::controlwidget::slider *w args*
Create a widget containing one or more vertical sliders and an axis. You can shift the slider handles interactively via the mouse.

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget:

variable name
Name of the (list) variable to be associated with the widget

number integer
Number of sliders in the widget

background color
Background color for the widget

height pixels
Height of the widget

width pixels
Width of the widget

sliderthickness pixels
Thickness of the slider handles

sliderwidth color
Width of the slider handles

troughwidth color
Width of the troughs holding the sliders

::controlwidget::equalizerBar *w args*

Create a widget containing one or more vertical bars resembling those found on hifi graphical equalizers. Note that it is a read-only widget.

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget:

variable name
Name of the (list) variable to be associated with the widget

number integer
Number of bars in the widget

background color
Background color for the widget

height pixels
Height of the widget

width pixels
Width of the widget

barwidth pixels
Width of the bars

segments integer
Number of segments the bars are divided in

safecolor color
Color for values lower than the warning level

warningcolor color
Color for values greater than the warning level

warninglevel double
Threshold value, determines at what level the color changes

::controlwidget::thermometer *w args*

Create a thermometer widget (read-only)

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget:

variable name
Name of the (list) variable to be associated with the widget

background color
Background color for the widget

height pixels
Height of the widget

width pixels
Width of the widget

linethickness pixels
Thickness of the line representing the "temperature"

linecolor color

Color of the line

These widgets all use a vertical axis and have the following options in common for configuring the axis:

axisformat string
Format string for formatting the numbers along the axis

axisfont string
Name of the font to be used for the numbers

axiscolor color
Color of the axis (including the numbers)

from color
Lower value for the axis

to color
Upper value for the axis

majorticks integer
Number of major tickmarks to draw (each is labelled)

minorticks integer
Number of minor tickmarks to draw between two major tickmarks

log boolean
Use a logarithmic axis (true) or a linear axis (false)

The package contains the following widget based on angle displays:

::controlwidget::voltmeter *w args*
Create a voltmeter-like widget.

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget:

variable name
Name of the variable to be associated with the meter

value value
Value for the meter (if not associated with a variable)

min value
The minimum value for data in the display

max value
The maximum value for data in the display

labels list
The labels to be shown along the scale. (These are simply considered texts, so no relation with the minimum and maximum perse)

title string
String to be shown below the dial

width pixels
The width of the widget

height pixels

The height of the widget

borderwidth pixels
The width of the border surrounding the entire widget

relief string
The relief of the border surrounding the entire widget

highlightthickness pixels
The width of the border to indicate the widget has keyboard focus

labelfont fontname
Font of the labels to the scale

titlefont fontname
Font of the title below the dial

dialcolor color
Color of the dial holding the needle

needlecolor color
Color of the needle

scalecolor color
Color of the scale (arc and tickmarks)

labelcolor color
Color of the labels to the scale

titlecolor color
Color of the title below the dial

::controlwidget::tachometer *w args*
Create a tachometer-like widget.

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget. In addition to the ones given for the voltmeter widget:

variable name
Name of the variable to be associated with the meter

value value
Value for the meter (if not associated with a variable)

dangerlevel value
Level above which values are indicated with a different color

dangercolor color
Color for the values above the "danger level"

dangerwidth pixels
Width for the colored band indicating the "danger level"

pincolor color
Color for the needle and the pin

::controlwidget::rdial *w args*
Create a rotating dial. You can drag the dial to change the value. With the shift button

depressed the value changes slowly, with the control button depressed it changes fast. Similarly, you can control the dial via the mouse wheel, with the shift and control button working in the same way as with the mouse pointer. *Note:* The shift and control buttons currently have no effect on Windows.

widget *w* (in)
Name of the widget to be created.

arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget:

variable name
Name of the variable to be associated with the dial

value value
Value for the dial (if not associated with a variable)

orient string
Whether a horizontal disk/dial is required ("horizontal") or a vertical one ("vertical")

step value
Initial step for incrementing or decrementing the value

bindwheel value
Step size for controlling the dial's value via the mouse wheel

slow value
Factor on the step size for incrementing or decrementing the value slowly

fast value
Factor on the step size for incrementing or decrementing the value fast

scale value
Scale value to be applied to the actual value

callback command
Command to be called when the value changes

width pixels
The width of the widget

height pixels
The height of the widget

background color
The background color of the widget (coloring of the disk/dial)

foreground color
The foreground color of the widget (the marker lines)

All these widgets have the following methods:

$widget get
Return the current value or values shown in the widget

$widget set *value*
Reset the value or values shown in the widget. If the widget is associated with a variable, that variable is set as well.

value *double/list*
New value or values for the widget

Two further widgets are available, meant to display logical values:

::controlwidget::led *w args*
Create a LED-like widget.
widget *w* (in)
Name of the widget to be created.
arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget. In addition to the ones given for the voltmeter widget:
variable name
Name of the variable to be associated with the LED
size pixels
Diameter of the LED widget
on color
Color to use for the "on" state
off color
Color to use for the "off" state

::controlwidget::radioMatrix *w args*
Create a matrix of radio buttons that behaves as a single widget.
widget *w* (in)
Name of the widget to be created.
arguments *args* (in)
List of key-value pairs, controlling the appearance and behaviour of the widget. In addition to the ones given for the voltmeter widget:
variable name
Name of the variable to be associated with the matrix
orient string
The way the radio buttons are to be arranged (horizontal or vertical first)
rows integer
Number of rows in the matrix
columns integer
Number of columns in the matrix
command list
Command associated with the radio buttons. Invoked when the active radio button changes.

The LED widget has the following public methods:

$led on
Set the state to "on"
$led off
Set the state to "off"
$led setstate *state*
Set the state according to the value of "state"
state *boolean* (in)

New state for the LED widget

The radioMatrix widget has two public methods:

$matrix get
Return the index of the currently selected radio button

$matrix set *index*
Select the radio button with index "index"
index *integer*
Index of the radio button to be set

Acknowledgments

The code for most of these widgets first appeared on the Wiki. In many cases, Arjen Markus merely refactored the code a bit and "snitified" some of them. The original code was developed by the following people:

- Vertical meter, LED display, radio matrix: Ron Fox
- Rotating dials: Gerhard Reithofer
- Voltmeter and tachometer: Marco Maggi
- Code for moving the needle: ?

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *controlwidget* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright

```
Copyright (c) 2010 Ron Fox <rfox@...>
Copyright (c) 2010 Gerhard Reithofer <...@...>
Copyright (c) 2010 Marco Maggi <...@...>
Copyright (c) 2010 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

crosshair

crosshair

Keywords: crosshair, canvas, cross-hairs, location, tracking, viewport

Name

crosshair - Crosshairs for Tk canvas

Synopsis

package require Tcl ?**8.4**?
package require Tk ?**8.4**?
package require crosshair ?**1.2.1**?
crosshair::crosshair *w* ?*arg...*?
crosshair::off *w*
crosshair::configure *w* ?*arg...*?

crosshair::track on *w cmdprefix*
crosshair::track off *w*
crosshair::bbox_add *w bbox*
crosshair::bbox_remove *token*

Description

The **crosshair** package provides commands to (de)activate and track crosshairs on canvas widgets.

API

The following commands are exported to the public:

crosshair::crosshair *w ?arg...?*
: This command activates the display of a pair of cross-hairs for the canvas widget *w*. The cross-hairs track the pointing device. The result of the command is the empty string.

  All arguments after the widget *w* are treated as options as for a canvas line item in *w*. Of particular interest are **-fill** and **-dash**.

crosshair::off *w*
: This command removes the cross-hairs from the canvas widget *w*. Nothing is done if the widget had no cross-hairs. The result of the command is the empty string.

crosshair::configure *w ?arg...?*
: This command changes the appearance of the cross-hairs in the canvas widget *w*. It is an error to call it for a canvas which has no cross-hairs.

  All arguments after the widget *w* are treated as options as for a canvas line item in *w*. Of particular interest are **-fill** and **-dash**.

  The result of the command are the current configuration settings.

crosshair::track on *w cmdprefix*
: This command activates reporting of the location of the cross-hairs in the canvas widget *w*. It is an error to use this command for a canvas which has no cross-hairs. The result of the command is the empty string.

  After the invokation of this command the specified command prefix *cmdprefix* will be called whenever the mouse moves within the canvas, with 7 arguments. These are, in order:
  1. The widget *w*
  2. The x-location of the cross-hairs, in pixels.
  3. The y-location of the cross-hairs, in pixels.
  4. The x-location of the top-left corner of the viewport, in pixels.
  5. The y-location of the top-left corner of the viewport, in pixels.
  6. The x-location of the bottom-right corner of the viewport, in pixels.
  7. The y-location of the bottom-right corner of the viewport, in pixels.

  A previously existing callback for *w* will be disabled. I.e. per canvas widget with cross-hairs only one callback reporting their location is possible.

crosshair::track off *w*
: This command disables the reporting of the location of the cross-hairs in the canvas widget *w*.

It is an error to use this command for a canvas which has no cross-hairs. The result of the command is the empty string.

crosshair::bbox_add *w bbox*

This command adds a bounding box to the crosshairs for canvas *w*. The crosshairs will only be active within that area.

The result of the command is a token with which the bounding box can be removed again, see **crosshair::bbox_remove** below.

The bounding box *bbox* is specified thorugh a list of 4 values, the lower left and upper right corners of the box. The order of values in the list is:

```
llx lly urx ury
```

Note that this command can be used multiple times, each call adding one more bounding box. In such a case the visible area is the *union* of all the specified bounding boxes.

crosshair::bbox_remove *token*

This command removes the bounding box specified by the *token* (a result of **crosshair::bbox_add**) from the crosshairs for its canvas widget.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tklib :: crosshair* of the Tcllib SF Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *crosshair* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) 2003 Kevin Kenny
Copyright (c) 2008 (docs) Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
Copyright (c) 2013 Frank Gover, Andreas Kupries
```


ctext

ctext

Keywords: ctext, syntax highlighting, text, widget

Name

ctext - Ctext a text widget with highlighting support

Synopsis

package require Tk
package require ctext ?**3.3**?

ctext *pathName* ?*options*?
::ctext::addHighlightClass *pathName class color keywordlist*
::ctext::addHighlightClassWithOnlyCharStart *pathName class color char*
::ctext::addHighlightClassForSpecialChars *pathName class color charstring*
::ctext::addHighlightClassForRegexp *pathName class color pattern*
::ctext::clearHighlightClasses *pathName*
::ctext::getHighlightClasses *pathName*
::ctext::deleteHighlightClass *pathName class*
::ctext::enableComments *enable*
::ctext::disableComments *enable*
pathName **highlight** *startIndex endIndex*
pathName **fastdelete** *index1* ?*index2*?
pathName **fastinsert**
pathName **copy**
pathName **cut**
pathName **paste**
pathName **append**
pathName **configure** *option value* ?*...*?

Description

The **ctext** package provides the ctext widget which is an enhanced text widget with support for configurable syntax highlighting and some extra commands.

Ctext overloads the text widget and provides new commands, named **highlight**, **copy**, **paste**, **cut**, **append**, and **edit**. It also provides several commands that allow you to define classes. Each class corresponds to a tag in the widget.

Commands

ctext *pathName* ?*options*?
Creates and configures a ctext widget.

Highlighting

Highlighting is controlled with text widget tags, that are called highlight classes. The *class* is a tag name and can be configured like any text widget tag. Four types of highlight classes are supported. All highlight classes are automatically used by the **highlight** method of the widget.

::ctext::addHighlightClass *pathName class color keywordlist*
Add a highlighting class *class* to the ctext widget *pathName*. The highlighting will be done with the color *color*. All words in the *keywordlist* will be highlighted.

```
    # highlight some tcl keywords
    ::ctext::addHighlightClass .t tclkeywords red [list set info interp
    uplevel upvar]
```

::ctext::addHighlightClassWithOnlyCharStart *pathName class color char*
Add a highlighting class *class* to the ctext widget *pathName*. The highlighting will be done with the color *color*. All words starting with *char* will be highlighted.

```
    ::ctext::addHighlightClassWithOnlyCharStart .t vars blue \$
```

::ctext::addHighlightClassForSpecialChars *pathName class color charstring*
Add a highlighting class *class* to the ctext widget *pathName*. The highlighting will be done with the color *color*. All chars in *charstring* will be highlighted.

::ctext::addHighlightClassForRegexp *pathName class color pattern*
Add a highlighting class *class* to the ctext widget *pathName*. The highlighting will be done with the color *color*. All text parts matching the regexp *pattern* will be highlighted.

::ctext::clearHighlightClasses *pathName*
Remove all highlight classes from the widget *pathName*.

::ctext::getHighlightClasses *pathName*
List all highlight classes for the widget *pathName*.

::ctext::deleteHighlightClass *pathName class*
Delete the highlight class *class* from the widget *pathName*

::ctext::enableComments *enable*
Enable C comment highlighting. The *class* for c-style comments is **_cComment**. The C comment highlighting is disabled by default.

::ctext::disableComments *enable*
Disable C comment highlighting.

Widget commands

Each ctext widget created with the above command supports the following commands and options in addition to the standard text widget commands and options.

pathName* highlight *startIndex endIndex
Highlight the text between *startIndex* and *endIndex*.

***pathName* fastdelete *index1* ?*index2*?**
Delete text range without updating the highlighting. Arguments are identical to the *pathName* **delete** command inherited from the standard text widget.

***pathName* fastinsert**
Insert text without updating the highlighting. Arguments are identical to the *pathName* **insert** command inherited from the standard text widget.

***pathName* copy**
Call **tk_textCopy** for the ctext instance.

***pathName* cut**
Call **tk_textCut** for the ctext instance.

***pathName* paste**
Call **tk_textPaste** for the ctext instance.

***pathName* append**
Append the current selection to the clipboard.

***pathName* configure *option value* ?...?**
Set the options for the ctext widget. Each option name must be followed the new value.

Widget options

-linemap
Creates (-linemap 1) or deletes (-linemap 0) a line number list on the left of the widget. The default is to have a linemap displayed.

-linemapfg
Changes the foreground of the linemap. The default is the same color as the main text widget.

-linemapbg
Changes the background of the linemap. The default is the same color as the main text widget.

-linemap_select_fg
Changes the selected line foreground. The default is black.

-linemap_select_bg
Changes the selected line background. The default is yellow.

-linemap_mark_command
Calls a procedure or command with the *pathName* of the ctext window, the *type* which is either **marked** or **unmarked**, and finally the line number selected. The proc prototype is:

```
proc linemark_cmd {win type line}.
```

See also ctext_test_interactive.tcl

-highlight
Takes a boolean value which defines whether or not to highlight text which is inserted or deleted. The default is 1.

-linemap_markable
Takes a boolean value which specifies whether or not lines in the linemap are markable with the mouse. The default is 1.

Example

```
    package require Tk
    package require ctext

    proc main {} {
    pack [frame .f] -fill both -expand 1
    pack [scrollbar .f.s -command {.f.t yview}] -side right -fill y

    pack [ctext .f.t -bg black -fg white -insertbackground yellow
    -yscrollcommand {.f.s set}] -fill both -expand 1

    ctext::addHighlightClass .f.t widgets purple  [list ctext button
    label text frame toplevel  scrollbar checkbutton canvas listbox
    menu menubar menubutton  radiobutton scale entry message
    tk_chooseDir tk_getSaveFile  tk_getOpenFile tk_chooseColor
    tk_optionMenu]

    ctext::addHighlightClass .f.t flags orange  [list -text -command
    -yscrollcommand  -xscrollcommand -background -foreground -fg -bg
    -highlightbackground -y -x -highlightcolor -relief -width  -height
    -wrap -font -fill -side -outline -style -insertwidth
```

```
    -textvariable -activebackground -activeforeground
    -insertbackground  -anchor -orient -troughcolor -nonewline -expand
    -type -message  -title -offset -in -after -yscroll -xscroll
    -forward -regexp -count  -exact -padx -ipadx -filetypes -all -from
    -to -label -value -variable  -regexp -backwards -forwards -bd
    -pady -ipady -state -row -column  -cursor -highlightcolors
    -linemap -menu -tearoff -displayof -cursor  -underline -tags -tag]

    ctext::addHighlightClass .f.t stackControl red  {proc uplevel
    namespace while for foreach if else}
    ctext::addHighlightClassWithOnlyCharStart .f.t vars mediumspringgreen
    "\$"
    ctext::addHighlightClass .f.t variable_funcs gold {set global
    variable unset}
    ctext::addHighlightClassForSpecialChars .f.t brackets green {[]{}}
    ctext::addHighlightClassForRegexp .f.t paths lightblue
    {\.[a-zA-Z0-9\_\-]+}
    ctext::addHighlightClassForRegexp .f.t comments khaki {#[^\n\r]*}
    .f.t fastinsert end [info body main]

    pack [frame .f1] -fill x

    .f.t highlight 1.0 end

    pack [button .f1.exit -text Exit -command exit] -side left

    pack [entry .e] -side bottom -fill x
    .e insert end "ctext::deleteHighlightClass .f.t "
    bind .e <Return> {eval [.e get]}
    }
    main
```

Further examples are in the source package for ctext.

Thanks

Kevin Kenny, Neil Madden, Jeffrey Hobbs, Richard Suchenwirth, Johan Bengtsson, Mac Cody, Günther, Andreas Sievers, and Michael Schlenker.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *ctext* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) George Peter Staplin <GeorgePS@XMission.com>
```

cursor

cursor

Keywords: cursor

Name

cursor - Procedures to handle CURSOR data

Synopsis

package require Tk
package require cursor ?**0.3.1**?
::cursor::propagate *widget cursor*
::cursor::restore *widget* ?*cursor*?
::cursor::display ?*parent*?

Description

The **cursor** package provides commands to handle Tk cursors.

Commands

The following commands are available:

::cursor::propagate *widget cursor*
Sets the cursor for the specified *widget* and all its descendants to *cursor*.

::cursor::restore *widget* ?*cursor*?
Restore the original or previously set cursor for the specified *widget* and all its descendants. If *cursor* is specified, that will be used if on any widget that did not have a preset cursor (set by a previous call to **::cursor::propagate**).

::cursor::display ?*parent*?
Pops up a dialog with a listbox containing all the cursor names. Selecting a cursor name will display it in that dialog. This is simply for viewing any available cursors on the platform.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *cursor* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) Jeffrey Hobbs <jeff@hobbs.org>
```


datefield

datefield

Keywords: datefield, clock, date, dateentry, entry, widget

Name

datefield - Tk datefield widget

Synopsis

package require Tk
package require datefield ?**0.3**?
::datefield::datefield *widgetpath* ?*options*?
-format

Description

The **datefield** package provides the datefield widget which is an enhanced text entry widget for the purpose of date entry. There are three valid formats for the dates which can be entered:

1. English form MM/DD/YYYY using *-format* "*%m/%d/%Y*" (default)
2. German form DD.MM.YYYY using *-format* "*%d.%m.%Y*"
3. ISO form YYYY-MM-DD using *-format* "*%Y-%m-%d*"

The datefield widget is, in fact, just an entry widget with specialized bindings. This means all the command and options for an entry widget apply equally here.

Commands

::datefield::datefield *widgetpath* ?*options*?
Creates and configures a date field widget.

Options

-format
One of "%m/%d/%Y" (English, default if option left), "%d.%m.%Y" (German), or "%Y-%m-%d" (ISO).

See the **entry** manual entry for details on all remaining/available options.

Example

```
package require datefield

wm title . "Datefield example"
proc DayOfWeek {args} {
    set now [clock scan $::myDate]
    set ::myDate2 [clock format $now -format %A]
}
trace variable myDate w DayOfWeek

::datefield::datefield .df -textvariable myDate -format "%m/%d/%Y"
label .l1 -text "Enter a date:"   -anchor e
label .l2 -text "That date is a:" -anchor e
label .l3 -textvariable myDate2 -relief sunken -width 12

grid .l1 .df -sticky ew
grid .l2 .l3 -sticky ew
focus .df
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *datefield* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Widget

Copyright

```
Copyright (c) Keith Vetter <keith@ebook.gemstar.com>
Copyright (c) Thomas Wunderlich <tcl.tk@blindenfreizeiten.de>
```


diagrams

diagram
Keywords: diagram, 2D geometry, arc, arrow, box, canvas, circle, diamond, drawing, drum, ellipse, image, interpolation, intersection, line, move, picture, plane geometry, plotting, point, raster image, spline, text, vector

Name
diagram - Diagram drawing

Synopsis
package require Tcl 8.5
package require Tk 8.5
package require diagram 1
::diagram *objectName canvas* ?*script*?
diagramObject **new direction** *name* ?*key value...*?
diagramObject **new element** *name attributes cmdprefix*
diagramObject **new alias** *name cmdprefix*
diagramObject **new command** *name arguments body*
diagramObject **new attribute** *name* ?*key value...*?
diagramObject **unknown attribute** *cmdprefix*
diagramObject **draw** *script*
arc *attr...*
arrow *attr...*
--> *attr...*
<--> *attr...*
<--> *attr...*
block *script attr...*
box *attr...*
circle *attr...*
O *attr...*
diamond *attr...*
<> *attr...*
drum *attr...*
ellipse *attr...*

line *attr...*
-- *attr...*
move *attr*
spline *attr...*
text *attr...*
west
w
left
l
south
s
down
bottom
bot
b
east
e
right
r
north
n
up
top
t
northwest
nw
up-left
upleft
leftup
northeast
ne
up-right
upright
rightup
southwest
sw
down-left
downleft
leftdown
southeast
se
down-right
downright
rightdown
number **cm**
number **mm**

number **inch**
number **pt**
number **number**
by *distance direction*
point1 **+** *point2*
point1 **-** *point2*
point **by** *distance direction*
point1 **|** *point2*
n **between** *poin1 point2*
intersect *elem1 elem2*
element **names** ?*pattern*?
element **corner**
element **corner1** *corner2...*
element ?*corner1... ?names ?pattern*? ?*]*?
nth ?*corner*?
nth **last** ?*corner*?
nth *shape* ?*corner*?
nth **last** *shape* ?*corner*?
last ?*corner*?
last *shape* ?*corner*?
1st
2nd
3rd

Description

Welcome to **diagram**, a package for the easy construction of diagrams (sic), i.e. 2D vector graphics, sometimes also called *pictures*. Note that this package is not a replacement for **Tk**'s canvas, but rather a layer sitting on top of it, to make it easier to use. In other words, using the canvas as the core graphics engine **diagram** abstracts away from the minutiae of handling coordinates to position and size the drawn elements, allowing the user to concentrate on the content of the diagram instead.

This is similar to Brian Kernighan's PIC language for troff, which is the spiritual ancestor of this package.

This document contains the reference to the API and drawing (language) commands. Its intended audience are users of the package wishing to refresh their memory. Newcomers should read the *Diagram Language Tutorial* first. Developers wishing to work on the internals of the package and its supporting packages should look at section **Diagram Classes** first, and then the comments in the sources of the packages itself.

In the remainder of the document we first describe the APIs of the diagram class and its instances, followed by the language reference for the drawing language itself.

API

Class API

The package exports the API described here.

::diagram *objectName canvas* ?*script*?

The command creates a new instance of a diagram controller and returns the fully qualified name of the object command as its result. The new instance is connected to the specified *canvas* object, which is used as the diagrams graphics engine. This is usually an instance of Tk's canvas, however any object which is API compatible to Tk's canvas can be used here.

The API of this object command is described in the following section, Object API. It may be used to invoke various operations on the object.

If the *script* argument is specified then method **draw** will be invoked on it.

Object API

Instances of the diagram class support the following methods:

***diagramObject* new direction *name* ?*key value*...?**

This method defines a new named direction and its attributes. The latter is given through the *key*/*value* pairs coming after the *name*.

Users are mostly free to specify arbitrary attributes with whatever meaning they desire. The exception are the names *angle* and *opposite*. They are special to the diagram package and have a fixed meaning.

angle

This attribute specifies the angle of the direction in degrees, where 0 points east (to the right) and 90 points north (up).

opposite

This attribute specifies the name of the direction which should be considered as complementary to the named one.

diagramObject* new element *name attributes cmdprefix

This method defines a new graphics element for the drawing language. I.e. *name* will become a new command in the language, and the specified command prefix (*cmdprefix*) will be called to perform the actual drawing.

attributes specifies the set of attributes for which data has to be available. I.e. the system will run the ...-callbacks for these attributes. See the method **new attribute** for more information on attribute definitions.

The command prefix is expected to conform to the following signature:

cmdprefix *canvas attributes*

Where *canvas* is the handle of the canvas widget to draw to, and *attributes* is a dictionary holding the attributes for the element, be they user-specified, or defaults.

The results of the command has to be a list containing at least two and at most four items. These are, in order:

1. The list of canvas items the drawn element consists of.
2. The dictionary of named locations in the element, its *corners*.
3. An optional mode, either "relative" or "absolute". When not returned "relative" is assumed. In the case of a relative element position the attributes "with" and "at" are used to determine the final position of the new element.
4. An optional name of a direction. If not the empty string this is handed to the automatic layouter as the new direction to follow.

diagramObject* new alias *name cmdprefix

This method defines a new command for the drawing language. I.e. *name* will become a new command in the language, and the specified command prefix (*cmdprefix*) will be called on use of this new command. Any arguments given to the command are simply passed to the prefix. There is no fixed siganture.

Note that the prefix is run in the context of the drawing language, allowing the direct use of any existing commands.

diagramObject* new command *name arguments body

This is like **new alias** except that the new command is defined as a procedure in the language's context, with regular argument list and body.

***diagramObject* new attribute *name* ?*key value*...?**

This method defines a new named attribute which can be used by graphical elements. The handling of the attribute by the processor is declared through the *key*/*value* pairs coming after the *name*.

The accepted keys and their meanings are:

key

The value of this key is the name of the key under which the attribute's value shall be stored in the attribute dictionary given to the drawing command after attribute processing is complete.

This key is optional. If it is not specified it defaults to the name of the attribute.

get

The value of this key is a command prefix which will be invoked to retrieve the attribute's argument(s) from the command line.

This key is optional. If it is not specified a default is used which takes the single word after the attribute name as the attribute's value.

The signature of the command prefix is

cmdprefix *wordqueue*

Where *wordqueue* is the handle of a queue object conforming to the API of the queues provided by package **struct::queue**. This queue contains the not-yet-processed part of the attribute definitions, with one entry per word, with the first entry the word *after* name of the attribute. In other words, the attribute's name has already been removed from the queue.

The result of the command is the value of the attribute, which may have been taken from the queue, or not.

transform

The value of this key is a command prefix which will be invoked to transform the retrieved value (See **get**) into their final form.

This key is optional. If it is not specified no transformation is done. The signature of the command prefix is

cmdprefix *value*

Where *value* is the value to transform.

The result of the command is the final value of the attribute.

type
The value of this key is a command prefix which will be invoked to validate the attribute's argument(s).

This key is optional. If it is not specified no validation is done.

The signature of the command prefix is that of snit validation types. See the documentation of the **snit** package.

merge
The value of this key is a command prefix which will be invoked to insert the transformed and validated attribute value into the dictionary of collected attributes.

This key is optional. If it is not specified a default merge is chosen, based on the data for key **aggregate**, see below. The signature of the command prefix is

cmdprefix *value dict*
Where *value* is the value to insert, and *dict* the dictionary of attributes and values collected so far.

The result of the command is the new dictionary of attributes.

aggregate
The value of this key is a boolean flag. It has an effect if and only if the key **merge** was not specified. This key is optional. If it is not specified it defaults to **False**.

If the key is effective, the value of **False** means that the attribute's value is *set* into the dictionary using the value of key *key* (see above) as index, *overwriting* any previously specified value.

If the key is effective, the value of **True** means that the attribute's value is *added* to the dictionary using the value of key *key* (see above) as index, *extending* any previously specified value. This means that the final value of the attribute as seen after processing will be a list of the collected values.

default
The value of this key is a command prefix which will be invoked after collection of attributes has been completed and this attribute is in the list of required attributes for the drawing element (See argument *attributes* of method **new element**).

Note that the connection is made through the value of key *key*, not through the attribute name per se.

Further note that this command prefix is invoked even if a user specified attribute value is present. This allows the command to go beyond simply setting defaults, it can calculate and store derived values as well.

This key is optional. If an element requires this attribute, but *default* is not specified then

nothing will be done.

The signature of the command prefix is

cmdprefix init

This method is run when the attribute is defined, its responsibility is to initialize anything in the language namespace for the attribute and default processing.

The result of this method is ignored.

cmdprefix fill *varname*

This method is run to put defaults, or derived values into the attribute dictionary named by *varname*. This variable will be found in the calling context.

The result of this method is ignored.

cmdprefix set *name value*

This method is run to push current a attribute value into the language namespace, to make it the new default.

The result of this method is ignored.

linked

This key is effective if and only if key **default** is not specified. In that case is supplies a default handling for **default**, linking the attribute to a variable in the language context.

The value for this key is a 2-element list containing the name of the variable to link to, and its initial value, in this order.

diagramObject* unknown attribute *cmdprefix

This method registers the command prefix with the subsystem processing the attributes for element commands, telling it to call it when it encounters an attribute it is unable to handle on its on.

It is allowed to register more than callback, these will be called in order of registration (i.e. first to last), until one of the callbacks accepts the current input. The command prefix is expected to conform to the following signature:

cmdprefix *wordqueue*

Where *wordqueue* is the handle of a queue object conforming to the API of the queues provided by package **struct::queue**. This queue contains the not-yet-processed part of the attribute definitions, with one entry per word, with the first entry the name of the attribute which could not be processed.

The results of the command has to be a boolean value where **True** signals that this callback has accepted the attribute, processed it, and the new state of the *wordqueue* is where the general processing shall continue.

Given the signature the command has basically two ways of handling (rewriting) the attributes it recognizes:

1. Replace the attribute (and arguments) with a different attribute and arguments.
2. Push additional words in front to get the general processing unstuck.

diagramObject* draw *script

This method runs the given *script* in the context of the drawing language definitions. See section Language reference for details on the available commands.

Note that *script* is *trusted*. It is executed in the current interpreter with access to its full abilities. For the execution of untrusted diagram scripts this interpreter should be a safe one.

Language reference

Elements

This section lists the commands for the predefined drawing elements, aka shapes. These commands are all defined in the language's context. All commands of this section return the handle of the newly created element as their result. This handle also exists as a command which can be used to query the element for its corners (names, values). See section Miscellaneous commands. IMAGE: figure-02-basic-shapes

arc *attr...*

IMAGE: figure-02-arc An open element with the corresponding corners, i.e. "start", "end", and "center". Note however that it also has the compass rose of closed elements as its corners, with the center of the arc's circle as the center of the compass and the other points on the circle the arc is part of. It handles the attributes

anchor *name*

see below ...

ljust

see below ...

rjust

see below ...

above

see below ...

below

IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust

"anchor west"

rjust

"anchor east"

above

"anchor south"

below

"anchor north"

clockwise
see below ...

cw
Specifies the direction of the **arc** element, here going clockwise. The complementary attribute is **counterclockwise**. If not specified the system falls back to the value taken from the language variable **clockwise**, which itself defaults to **false**, for counter-clockwise direction.

color *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

counterclockwise
see below ...

ccw
Specifies the direction of the **arc** element, here counter-clockwise. The complementary attribute is **clockwise**. If not specified the system falls back to the value taken from the language variable **clockwise**, which itself defaults to **false**, for counter-clockwise direction.

fillcolor *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

from *location*
Specifies the location where the **arc** element begins. Defaults to the current location as maintained by the layouting system.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

radius *length*
Specifies the radius of the **arc** element, or rather, the radius of the circle the shown arc is a part of. If not specified the system falls back to the value taken from the language variable **arcradius**, which itself defaults to the pixel equivalent of **1 cm**.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

to *location*
Specifies the location where the **arc** element ends. Defaults to a location such that a 90-degree arc is drawn in the chosen direction, starting at **from**.

arrow *attr...*
see below ...

--> *attr...*
see below ...

<--> *attr...*
see below ...

<--> *attr...*
IMAGE: figure-02-arrow An alias for the **line** element (see below), with the attribute **arrowhead** preset to **->**, **<->**, or **<-**. The **arrow** is equivalent to **-->**.

block *script attr...*
A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". The main effect is the aggregration of all elements created by the *script* into one element. This also means that while the elements created by the script are visible in the element history while the script is executing, afterward the history contains only the block itself, and not the elements it is composed of.

The script has access to the current state of all variables in the language context. Any changes to the variables will be reverted after execution of the block. However, also, any

variables set in the script will be exported as corners of the element, allowing users to define their own named locations in the block.

Regarding the layout mechanism any changes made by the script are reverted after the element is done. In other words, a block is an implicit **group**.

Blocks handle all attributes, propgating their settings into the script as the default values active during script execution.

box *attr...*

IMAGE: figure-02-box A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". It handles the attributes

anchor *name*

see below ...

ljust

see below ...

rjust

see below ...

above

see below ...

below

IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust

"anchor west"

rjust

"anchor east"

above

"anchor south"

below

"anchor north"

at *location*

Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

color *spec*

IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

fillcolor *spec*

IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

height *length*
see below ...

ht *length*
Specifies the height of the element. If not specified the system falls back to the value taken from the language variable **boxheight**, which itself defaults to the pixel equivalent of **2 cm**.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

slant *angle*
Specifies the angle by which the **box** element is slanted, in degrees. If not specified the system falls back to the value taken from the language variable **slant**, which itself defaults to **90**, i.e. vertical, no slant. 0 degrees is slanting straight east, pointing to the right. 90 degrees is slanting to the north, pointing straight up.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

width *length*
see below ...

wid *length*
Specifies the width of the element. If not specified the system falls back to the value taken from the language variable **boxwidth**, which itself defaults to the pixel equivalent of **2 cm**.

with *corner*
Specifies the corner of the element to place at the location given by the attribute **at**. Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

circle *attr...*
see below ...

O *attr...*
IMAGE: figure-02-circle A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". It handles the attributes

anchor *name*
see below ...

ljust
see below ...

rjust
see below ...

above
see below ...

below
IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust
"anchor west"

rjust
"anchor east"

above

"anchor south"

below
"anchor north"

at *location*
Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

color *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

diameter *length*
see below ...

diam *length*
Specifies the diameter of the **circle** element, as an alternative way to specify its **radius**. Effective if and only if the radius was not specified. I.e. if both diameter and radius are specified then the radius infomration has precendence. This attribute has no default, because the defaults are taken from the radius.

fillcolor *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

radius *length*
see below ...

rad *length*
Specifies the radius of the **circle** element. If not specified the system falls back to the value taken from the language variable **circleradius**, which itself defaults to the pixel equivalent of **1 cm**.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -

Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

with *corner*
Specifies the corner of the element to place at the location given by the attribute **at**. Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

diamond *attr*...
see below ...

***<>* *attr*...**
IMAGE: figure-02-diamond A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". It handles the attributes

anchor *name*
see below ...

ljust
see below ...

rjust
see below ...

above
see below ...

below
IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**,

which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust
"anchor west"

rjust
"anchor east"

above
"anchor south"

below
"anchor north"

aspect ***number***
Specifies the aspect ratio, i.e ratio of width to height, of the **diamond** element. The manner in which a default is calculated when not specified also depends on the specifications of the attributes **width** and **height**, if any.

If both **width**, and **height** are specified then any specification of **aspect** is ignored, as it is implicitly defined in the width and height as well, and this takes precedence. A missing specification is ignored in that case well, i.e. no defaults are required.

If the **aspect** is specified, and one of the attributes **width** or **height**, then the missing attribute is calculated from the two which are specified. No defaults are required for these cases either.

If only one of the attributes **width** or **height** is specified then the system uses a fallback for the **aspect**, the value taken from the language variable **diamondaspect**, which itselfs defaults to **2**.

If none of of the attributes **width** or **height** is specified then the system uses a fallback for the **width**, the value taken from the language variable **boxwidth**, which itselfs defaults to the pixel equivalent of **2 cm**. For the aspect it uses either the user-specified value or the default taken as described in the previous paragraph.

at ***location***
Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

color ***spec***
IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

fillcolor ***spec***
IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

height *length*
Specifies the height of the **diamond** element. The manner in which a default is calculated when not specified also depends on the specifications of the attributes **aspect** and **width**, if any.

If both **width**, and **height** are specified then any specification of **aspect** is ignored, as it is implicitly defined in the width and height as well, and this takes precedence. A missing specification is ignored in that case well, i.e. no defaults are required.

If the **aspect** is specified, and one of the attributes **width** or **height**, then the missing attribute is calculated from the two which are specified. No defaults are required for these cases either.

If only one of the attributes **width** or **height** is specified then the system uses a fallback for the **aspect**, the value taken from the language variable **diamondaspect**, which itselfs defaults to **2**.

If none of of the attributes **width** or **height** is specified then the system uses a fallback for the **width**, the value taken from the language variable **boxwidth**, which itselfs defaults to the pixel equivalent of **2 cm**. For the aspect it uses either the user-specified value or the default taken as described in the previous paragraph.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

width *length*
Specifies the width of the **diamond** element. The manner in which a default is calculated when not specified also depends on the specifications of the attributes **aspect** and **height** , if any.

If both **width**, and **height** are specified then any specification of **aspect** is ignored, as it is implicitly defined in the width and height as well, and this takes precedence. A missing specification is ignored in that case well, i.e. no defaults are required.

If the **aspect** is specified, and one of the attributes **width** or **height**, then the missing attribute is calculated from the two which are specified. No defaults are required for these cases either.

If only one of the attributes **width** or **height** is specified then the system uses a fallback for the **aspect**, the value taken from the language variable **diamondaspect**, which itselfs defaults to **2**.

If none of of the attributes **width** or **height** is specified then the system uses a fallback for the **width**, the value taken from the language variable **boxwidth**, which itselfs defaults to the pixel equivalent of **2 cm**. For the aspect it uses either the user-specified value or the default taken as described in the previous paragraph.

with *corner*
Specifies the corner of the element to place at the location given by the attribute **at**. Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

drum *attr...*
IMAGE: figure-02-drum A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". It handles the attributes

anchor *name*
see below ...

ljust

see below ...

rjust

see below ...

above

see below ...

below

IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust

"anchor west"

rjust

"anchor east"

above

"anchor south"

below

"anchor north"

aspect *number*

Specifies the aspect ratio, i.e ratio of width to height, of the ellipses which are used to draw the top and bottom of the **drum** element. If not specified the system falls back to the value taken from the language variable **drumaspect**, which itself defaults to **0.35**.

at *location*

Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

color *spec*

IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

fillcolor *spec*

IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

height *length*

see below ...

ht *length*

Specifies the height of the element. If not specified the system falls back to the value taken from the language variable **boxheight**, which itself defaults to the pixel equivalent of **2 cm**.

justify left|center|right

Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

stroke *width*

IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*

IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*

Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*

Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*

Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

width *length*

see below ...

wid *length*

Specifies the width of the element. If not specified the system falls back to the value taken from the language variable **boxwidth**, which itself defaults to the pixel equivalent of **2 cm**.

with *corner*

Specifies the corner of the element to place at the location given by the attribute **at**.

Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

ellipse *attr...*

IMAGE: figure-02-ellipse A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". It handles the attributes

anchor *name*

see below ...

ljust

see below ...

rjust

see below ...

above

see below ...

below

IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust

"anchor west"

rjust

"anchor east"

above

"anchor south"

below

"anchor north"

at *location*

Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

color *spec*

IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

fillcolor *spec*

IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

height *length*

see below ...

ht *length*
Specifies the height of the element. If not specified the system falls back to the value taken from the language variable **boxheight**, which itself defaults to the pixel equivalent of **2 cm**.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

width *length*
see below ...

wid *length*
Specifies the width of the element. If not specified the system falls back to the value taken from the language variable **boxwidth**, which itself defaults to the pixel equivalent of **2 cm**.

with *corner*
Specifies the corner of the element to place at the location given by the attribute **at**. Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

line *attr...*
see below ...

-- *attr...*
IMAGE: figure-02-line An open element with the corresponding corners, i.e. "start", "end", and "center". It handles the attributes

anchor *name*
see below ...

ljust
see below ...

rjust
see below ...

above
see below ...

below
IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust
"anchor west"

rjust
"anchor east"

above
"anchor south"

below
"anchor north"

arrowhead *spec*
IMAGE: figure-19-style-arrowheads Specifies where to draw arrowheads on the **line** element, at the beginning or end, at both ends, or none. If not specified the system falls back to the value taken from the language variable **arrowhead**, which itself defaults to **none**. The legal values are

none, -

Draw no arrowheads, at neither end of the line.

start, first, <-
Draw an arrowhead at the beginning of the line, but not at its end.

end, last, ->
Draw an arrowhead at the end of the line, but not at its beginning.

both, <->
Draw arrowheads at both ends of the line.

Note that the values "start", "end", "-", "->", "<-", and "<->" are all accepted as shorthands for the **arrowhead** command using them as argument.

at *location*
Line elements are normally positioned absolutely, using the locations specified through the attributes **from**, **then**, and **to**. If **at** is specified however then these positions are translated a last time, moving the line's corner named by the attribute **with** to the location given by this attribute.

chop ?*length*?
Specifies the length of the **line** element to remove from the beginning and/or end. Defaults to nothing. If specified once the chopping applies to both beginning and end of the line. If specified twice or more the last two specifications are used, and applied to beginning and end of the line, in this order. Whenever the attribute is specified without an explicit length, the system falls back to the value taken from the language variable **circleradius**, which itself defaults to the pixel equivalent of **1 cm**

color *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor** , which itself defaults to **black**.

fillcolor *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

from *location*
Specifies the location where the **line** element begins. Defaults to the current location as maintained by the layouting system.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

noturn
Specifies that the direction of **line** element at its end is not propagated to the layout management. If not specified the direction of the line becomes the new direction the layout.

smooth
Specifies the use of bezier splines for the **line** element. If not specified lines are drawn exactly through the specified waypoints, without any smooth curves.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

then *location*
see below ...

then (?*length*?)...
see below ...

(?*length*?)...
This attribute specifies an intermediate location the **line** element has to go through. It can be specified multiple times, with each use adding one additional location to the series which the line will go through. These location will be traversed in the order they were specified.

The location can be given explicitly, or as a series of directions with distances. In the latter

case the names of all known directions are accepted for the direction part. If no distance is specified for a direction the system falls back to the value taken from the language variable **movelength**, which itself defaults to the pixel equivalent of **2 cm**. The whole set of direction,distance pairs is treated as a series of translations which are added up to provide the final translation specifying the intermediate point (relative to the preceding point).

The last named direction is propagated to the layout system as the direction to follow. The use of **noturn** is not able to overide this behaviour.

At last, the names of the registered directions also serve as attribute commands, with an implicit attribute **then** in front of them.

If no intermediate or last location is specified for the line the system falls back to a point **movelength** pixels away from the starting location, in the current direction as maintained by the layouting system

to *location*
Specifies the location where the **line** element ends. This attribute has no default. The default is handled by the attribute **then**, which makes it appear as if **to** has a default when not specified.

with *corner*
Line elements are normally positioned absolutely, using the locations specified through the attributes **from**, **then**, and **to**. If **at** is specified however then these positions are translated a last time, moving the line's corner named by the attribute **with** to the location given by this attribute. This means that *with* is effective if and only if the attribute **at** was specified as well for the line.

move *attr*
An open element with the corresponding corners, i.e. "start", "end", and "center". A **move** element is in essence an invisible **line**. While the main effect we are interested in is the change it makes to the layout system, it is an actual element, i.e. it has the same corners as an ordinary line, and shows up in the history as well, allowing future references to all the places it touched. It handles the same attibutes as **line** elements.

spline *attr*...
IMAGE: figure-02-spline An alias for the **line** element (see above), with the attribute **smooth** preset.

text *attr*...
IMAGE: figure-02-text A closed element with the corresponding corners, i.e. the eight directions of the compass rose, and "center". It handles the attributes

anchor *name*
see below ...

ljust
see below ...

rjust
see below ...

above
see below ...

below
IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust
"anchor west"

rjust
"anchor east"

above
"anchor south"

below
"anchor north"

at *location*
Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

height *length*
Specifies the height of the **text** element. Defaults to the natural height of its text.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

width *length*
Specifies the width of the **text** element. Defaults to the natural width of its text.

with *corner*

Specifies the corner of the element to place at the location given by the attribute **at**. Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

Attributes

The set of all attributes supported by all the element commands is shown below. While we speak of them as commands, and provide a syntax, they are not truly available as actual commands, but only as part of the arguments for an element command.

Note that some of the attribute names are overloaded, i.e. have multiple, different, definitions. During processing of attributes for an element the actual definition used is chosen based on the type of the element the processing is for.

Further, as a catch-all clause, any attribute which could not be processed according to the definitions below will be treated as the argument of an implicit **text** attribute.

anchor *name*
see below ...

ljust
see below ...

rjust
see below ...

above
see below ...

below
IMAGE: figure-22-text-anchoring-3 Specifies the anchor of the text which is to be placed at the element's center, by name. I.e. this attribute defines the text's position relative to the element's center. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **anchor**, which itself defaults to **center**. The legal values are all those accepted by Tk_GetAnchor. The commands without arguments are all shorthands with the anchor implied. Note that they do not combine, only the last is used. For comined directions the main attribute command, **anchor** has to be used.

ljust
"anchor west"

rjust
"anchor east"

above
"anchor south"

below
"anchor north"

arrowhead *spec*
IMAGE: figure-19-style-arrowheads Specifies where to draw arrowheads on the **line** element, at the beginning or end, at both ends, or none. If not specified the system falls back to the value taken from the language variable **arrowhead**, which itself defaults to **none**. The legal values are

none, -
Draw no arrowheads, at neither end of the line.

start, first, <-
Draw an arrowhead at the beginning of the line, but not at its end.

end, last, ->
Draw an arrowhead at the end of the line, but not at its beginning.

both, <->
Draw arrowheads at both ends of the line.

Note that the values "start", "end", "-", "->", "<-", and "<->" are all accepted as shorthands for the **arrowhead** command using them as argument.

aspect *number*
Specifies the aspect ratio, i.e ratio of width to height, of the **diamond** element. The manner in which a default is calculated when not specified also depends on the specifications of the attributes **width** and **height**, if any.

If both **width**, and **height** are specified then any specification of **aspect** is ignored, as it is implicitly defined in the width and height as well, and this takes precedence. A missing specification is ignored in that case well, i.e. no defaults are required.

If the **aspect** is specified, and one of the attributes **width** or **height**, then the missing attribute is calculated from the two which are specified. No defaults are required for these cases either.

If only one of the attributes **width** or **height** is specified then the system uses a fallback for the **aspect**, the value taken from the language variable **diamondaspect**, which itselfs defaults to **2**.

If none of of the attributes **width** or **height** is specified then the system uses a fallback for the **width**, the value taken from the language variable **boxwidth**, which itselfs defaults to the pixel equivalent of **2 cm**. For the aspect it uses either the user-specified value or the default taken as described in the previous paragraph.

aspect *number*
Specifies the aspect ratio, i.e ratio of width to height, of the ellipses which are used to draw the top and bottom of the **drum** element. If not specified the system falls back to the value taken from the language variable **drumaspect**, which itself defaults to **0.35**.

at *location*
Specifies the location of the element's corner named by the attribute **with**. Defaults to the current location as maintained by the layouting system.

at *location*
Line elements are normally positioned absolutely, using the locations specified through the attributes **from**, **then**, and **to**. If **at** is specified however then these positions are translated a last time, moving the line's corner named by the attribute **with** to the location given by this attribute.

chop ?*length*?

Specifies the length of the **line** element to remove from the beginning and/or end. Defaults to nothing. If specified once the chopping applies to both beginning and end of the line. If specified twice or more the last two specifications are used, and applied to beginning and end of the line, in this order. Whenever the attribute is specified without an explicit length, the system falls back to the value taken from the language variable **circleradius**, which itself defaults to the pixel equivalent of **1 cm**

clockwise
see below ...

cw
Specifies the direction of the **arc** element, here going clockwise. The complementary attribute is **counterclockwise**. If not specified the system falls back to the value taken from the language variable **clockwise**, which itself defaults to **false**, for counter-clockwise direction.

color *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linecolor**, which itself defaults to **black**.

counterclockwise
see below ...

ccw
Specifies the direction of the **arc** element, here counter-clockwise. The complementary attribute is **clockwise**. If not specified the system falls back to the value taken from the language variable **clockwise**, which itself defaults to **false**, for counter-clockwise direction.

diameter *length*
see below ...

diam *length*
Specifies the diameter of the **circle** element, as an alternative way to specify its **radius**. Effective if and only if the radius was not specified. I.e. if both diameter and radius are specified then the radius infomration has precendence. This attribute has no default, because the defaults are taken from the radius.

fillcolor *spec*
IMAGE: figure-21-style-colors Specifies the color used to draw the inside of the element. If not specified the system falls back to the value taken from the language variable **fillcolor**, which itself defaults to the empty string, signaling "no filling".

from *location*
Specifies the location where the **line** element begins. Defaults to the current location as maintained by the layouting system.

from *location*
Specifies the location where the **arc** element begins. Defaults to the current location as maintained by the layouting system.

height *length*
see below ...

ht *length*
Specifies the height of the element. If not specified the system falls back to the value taken from the language variable **boxheight**, which itself defaults to the pixel equivalent of **2 cm**.

height *length*
Specifies the height of the **diamond** element. The manner in which a default is calculated when not specified also depends on the specifications of the attributes **aspect** and **width**, if any.

If both **width**, and **height** are specified then any specification of **aspect** is ignored, as it is implicitly defined in the width and height as well, and this takes precedence. A missing specification is ignored in that case well, i.e. no defaults are required.

If the **aspect** is specified, and one of the attributes **width** or **height**, then the missing attribute is calculated from the two which are specified. No defaults are required for these cases either.

If only one of the attributes **width** or **height** is specified then the system uses a fallback for the **aspect**, the value taken from the language variable **diamondaspect**, which itselfs defaults to **2**.

If none of of the attributes **width** or **height** is specified then the system uses a fallback for the **width**, the value taken from the language variable **boxwidth**, which itselfs defaults to the pixel equivalent of **2 cm**. For the aspect it uses either the user-specified value or the default taken as described in the previous paragraph.

height *length*
Specifies the height of the **text** element. Defaults to the natural height of its text.

justify left|center|right
Specifies how multi-line text associated with the element is positioned within its box. The value is ignored if no text was specified for the element. If not specified the system falls back to the value taken from the language variable **justify**, which itself defaults to **left**. The legal values are **left**, **right**, and **center**.

noturn
Specifies that the direction of **line** element at its end is not propagated to the layout management. If not specified the direction of the line becomes the new direction the layout.

radius *length*
see below ...

rad *length*
Specifies the radius of the **circle** element. If not specified the system falls back to the value taken from the language variable **circleradius**, which itself defaults to the pixel equivalent of **1 cm**.

radius *length*
Specifies the radius of the **arc** element, or rather, the radius of the circle the shown arc is a part of. If not specified the system falls back to the value taken from the language variable **arcradius**, which itself defaults to the pixel equivalent of **1 cm**.

slant *angle*
Specifies the angle by which the **box** element is slanted, in degrees. If not specified the system falls back to the value taken from the language variable **slant**, which itself defaults to **90**, i.e. vertical, no slant. 0 degrees is slanting straight east, pointing to the right. 90 degrees

is slanting to the north, pointing straight up.

smooth
Specifies the use of bezier splines for the **line** element. If not specified lines are drawn exactly through the specified waypoints, without any smooth curves.

stroke *width*
IMAGE: figure-20-style-stroke Specifies the width of the lines drawn for the the element, in pixels. If not specified the system falls back to the value taken from the language variable **linewidth**, which itself defaults to **1**.

style *spec*
IMAGE: figure-18-style-dash Specifies the style used to draw the lines of the element. If not specified the system falls back to the value taken from the language variable **linestyle**, which itself defaults to **solid** lines. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:

solid, empty string
Draw solid line.

dash, dashed, -
Draw a dashed line.

dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

Note that the values "solid", "dot(ted)", "dash(ed)", "dash-dot", and "dash-dot-dot" are all accepted as shorthands for the **style** command using them as argument.

text *string*
Specifies the text to associate with the element. Defaults to nothing. When specified multiple times the actually shown text is the concatenation of the individual strings, vertically stacked, with the first string specified being the topmost element.

textcolor *spec*
Specifies the color used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textcolor**, which itself defaults to **black**.

textfont *spec*
Specifies the font used to draw the text of an element with. Ignored if there is no text. If not specified the system falls back to the value taken from the language variable **textfont**, which itself defaults to **Helvetica 12pt**.

then *location*
see below ...

then (?*length*?)...
see below ...

(?*length*?)...
This attribute specifies an intermediate location the **line** element has to go through. It can be specified multiple times, with each use adding one additional location to the series which the

line will go through. These location will be traversed in the order they were specified.

The location can be given explicitly, or as a series of directions with distances. In the latter case the names of all known directions are accepted for the direction part. If no distance is specified for a direction the system falls back to the value taken from the language variable **movelength**, which itself defaults to the pixel equivalent of **2 cm**. The whole set of direction,distance pairs is treated as a series of translations which are added up to provide the final translation specifying the intermediate point (relative to the preceding point).

The last named direction is propagated to the layout system as the direction to follow. The use of **noturn** is not able to overide this behaviour.

At last, the names of the registered directions also serve as attribute commands, with an implicit attribute **then** in front of them.

If no intermediate or last location is specified for the line the system falls back to a point **movelength** pixels away from the starting location, in the current direction as maintained by the layouting system

to *location*
Specifies the location where the **line** element ends. This attribute has no default. The default is handled by the attribute **then**, which makes it appear as if **to** has a default when not specified.

to *location*
Specifies the location where the **arc** element ends. Defaults to a location such that a 90-degree arc is drawn in the chosen direction, starting at **from**.

width *length*
see below ...

wid *length*
Specifies the width of the element. If not specified the system falls back to the value taken from the language variable **boxwidth**, which itself defaults to the pixel equivalent of **2 cm**.

width *length*
Specifies the width of the **diamond** element. The manner in which a default is calculated when not specified also depends on the specifications of the attributes **aspect** and **height**, if any.

If both **width**, and **height** are specified then any specification of **aspect** is ignored, as it is implicitly defined in the width and height as well, and this takes precedence. A missing specification is ignored in that case well, i.e. no defaults are required.

If the **aspect** is specified, and one of the attributes **width** or **height**, then the missing attribute is calculated from the two which are specified. No defaults are required for these cases either.

If only one of the attributes **width** or **height** is specified then the system uses a fallback for the **aspect**, the value taken from the language variable **diamondaspect**, which itselfs defaults to **2**.

If none of of the attributes **width** or **height** is specified then the system uses a fallback for the **width**, the value taken from the language variable **boxwidth**, which itselfs defaults to the pixel equivalent of **2 cm**. For the aspect it uses either the user-specified value or the default taken as described in the previous paragraph.

width *length*
Specifies the width of the **text** element. Defaults to the natural width of its text.

with *corner*
Specifies the corner of the element to place at the location given by the attribute **at**. Defaults to the current corner as maintained by the layouting system, except if the value for **at** was specified by the user. In that case it defaults to **center**.

with *corner*
Line elements are normally positioned absolutely, using the locations specified through the attributes **from**, **then**, and **to**. If **at** is specified however then these positions are translated a last time, moving the line's corner named by the attribute **with** to the location given by this attribute. This means that *with* is effective if and only if the attribute **at** was specified as well for the line.

Corners

Corners are named values for in elements, usually locations.

- The *closed* elements define corners for the compass rose, including the "center", and their "width" and "height".

IMAGE: figure-27-corners-closed

- **block** elements additionally export all variables which were set during their definition as corners.
- The *open* elements on the other hand define "start", "end", and "center". The first two map to the locations originally provided through the attributes **from** and **to** of the element.

IMAGE: figure-28-corners-open

- The center of **line** and **move** elements is the location halfway between "start" and "end" corners, this is regardless of any intermediate locations the element may have.
- The **line** and **move** elements additionally name all their locations as corners using numbers as names, starting from **1** (equivalent to "start"), in order of traversal.

IMAGE: figure-15-spline-1

- The center of **arc** elements is the center of the circle the arc is part off.
- The **arc** elements additionally define the compass rose of closed elements as well.

Named directions

The named directions are commands which tell the layout system in which direction to go when placing the next element without an explicit position specification. They can also be used as arguments to the attribute **then**, and the command **by** for relative points, see there for the relevant syntax.

The diagram core defines the directions of the compass rose, plus a number of aliases. See below for the full list.

IMAGE: figure-27-corners-closed

This overlaps with the pre-defined corners for closed elements. This is used by the layout system, when are going in direction X the name of the opposite direction is the name of the corner at which the new element will be attached to the current position, and if this corner does not exist the nearest actual corner by angle is used.

- **west**
- **w**
- **left**
- **l**
- **south**
- **s**
- **down**
- **bottom**
- **bot**
- **b**
- **east**
- **e**
- **right**
- **r**
- **north**
- **n**
- **up**
- **top**
- **t**
- **northwest**
- **nw**
- **up-left**
- **upleft**
- **leftup**
- **northeast**
- **ne**
- **up-right**
- **upright**
- **rightup**
- **southwest**
- **sw**
- **down-left**
- **downleft**
- **leftdown**
- **southeast**
- **se**
- **down-right**
- **downright**
- **rightdown**

Miscellaneous commands

number **cm**

see below ...

***number* mm**

see below ...

***number* inch**

see below ...

***number* pt**

These commands allow the specification of distances and coordinates in metric and imperial units, returning the equivalent distance or coordinate in pixels, which is the unit used internally for all calculations.

The conversion factors are based on the result of **tk scaling** and are computed once, at the time the package is sourced, future changes of the **tk scaling** factor have no effect.

number number

IMAGE: figure-50-point-cons-absolute

This command takes the x and y coordinates of a location and returns the *absolute* point for it.

by *distance direction*

IMAGE: figure-51-point-cons-relative

This command takes a *distance* and *direction* (angle in degress, or registered direction name) and returns the *relative* point for it, i.e. the *delta* or *translation* it represents.

Note also the (dis)similarities to the directional specifications for the attribute **then** of **line** and **move** elements. Where we say here

```
by 50 east
```

for the attribute we say

```
... then east 50 ...
```

or just

```
... then east ...
```

point1 + point2

IMAGE: figure-48-point-vectoradd

This command interprets two points as vectors and adds them together. If at least one of the points is *absolute* the result is absolute as well. The result is a *relative* point if and only if both points are *relative*.

point1 - point2

IMAGE: figure-49-point-vectorsub

This command interprets two points as vectors and subtracts the second from the first. If at least one of the points is *absolute* the result is absolute as well. The result is a *relative* point if and only if both points are *relative*.

point* by *distance direction

This command is a more convenient, or at least shorter, form of

```
    [$point + [by $distance $direction]]
```

point1* | *point2

IMAGE: figure-31-point-projection

This command calculates the *projection* of two points, i.e. the result is the point having the x-coordinate of *point1* and the y-coordinate of *point2*.

n* between *poin1 point2

IMAGE: figure-29-point-interpolation-1

This command computes the point which is *n**100 percent of the way between *point1* and *point2*, and returns it as its result. This means that for

***n* == 0**

The result is *point1*.

***n* == 1**

The result is *point2*.

***n* == 0.5**

The result is half way between the two points.

etc. *Note* that it is allowed to use values < 0 and > 1 for *n*

intersect *elem1 elem2*

IMAGE: figure-32-point-intersection

This command takes two *open* elements, computes the lines going through their "start"- and "end"-corners, and returns the point where these two lines intersect. The command throws an error if the lines do not intersect, or are coincident.

***element* names ?*pattern*?**

This command returns a list containing the names of all corners for the *element*. If a pattern is specified then only the names matching it (via **string match** are returned. Otherwise all names are returned (equivalent to a default pattern of *****).

element corner

This command returns the value for the *corner* of the *element*. This can be anything, including points and elements.

element corner1 corner2...

This is a convenience shorthand for

```
[[[$elem $corner1] $corner2] ...]
```

assuming that the value for

```
 [$elem $corner1]
```

, etc. is again an element.

***element* ?*corner1*... ?names ?*pattern*??]?**

This is a convenience shorthand for

```
[[[$elem $corner1] ...] names ?pattern?]
```

assuming that the value for

```
 [$elem $corner1]
```

, etc. is again an element.

nth ?*corner*?
This command asks the diagram history for the **n**th element created, searching from the beginning of the history (counting from 1) and returns it as its result. If the *corner* is specified then the value for this corner is returned instead.

nth last ?*corner*?
This command asks the diagram history for the **n**th element created, searching from the end of the history and returns it as its result. If the *corner* is specified then the value for this corner is returned instead.

nth *shape* ?*corner*?
This command asks the diagram history for the **n**th element created, of the given *shape*, searching from the beginning of the history (counting from 1) and returns it as its result. If the *corner* is specified then the value for this corner is returned instead.

nth last *shape* ?*corner*?
This command asks the diagram history for the **n**th element created, of the given *shape*, searching from the end of the history and returns it as its result. If the *corner* is specified then the value for this corner is returned instead.

last ?*corner*?
see below ...

last *shape* ?*corner*?
Convenience commands mapping to "**1st last**" and "**1st last** *shape*".

1st
see below ...

2nd
see below ...

3rd
Aliases for **1th**, **2th**, and **3th**, for readability, usable whereever **n**th can ocur.

Variables

The language context contains a number of predefined variables which hold the default values for various attributes. These variables, their uses, and values are:

anchor
The default value for the attribute **anchor**. Initialized to **center**. The legal values are all those accepted by Tk_GetAnchor.

arcradius
The default value for the attribute **radius** of **arc** elements. Initialized to the pixel equivalent of **1 cm**.

arrowhead
The default value for the attribute **arrowhead**. Initialized to **none**. The legal values are
none, -
Draw no arrowheads, at neither end of the line.
start, first, <-
Draw an arrowhead at the beginning of the line, but not at its end.
end, last, ->
Draw an arrowhead at the end of the line, but not at its beginning.
both, <->
Draw arrowheads at both ends of the line.

boxheight
The default value for the attribute **height** of **box**, **diamond** and **ellipse** elements. Initialized to the pixel equivalent of **2 cm**.

boxwidth
The default value for the attribute **width** of **box**, **diamond** and **ellipse** elements. Initialized to the pixel equivalent of **2 cm**.

clockwise
The default value for the attributes **clockwise** and **counterclockwise** of **arc** elements. Initialized to **False**, for counter-clockwise direction.

circleradius
The default value for the attribute **radius** of **circle** elements, and also the default for the attribute **chop**, when specified without an explicit length. Initialized to the pixel equivalent of **1 cm**.

drumaspect
The default value for the attribute **aspect** of **drum** elements. Initialized to **0.35**.

fillcolor
The default value for the attribute **fillcolor** of all elements which can be filled. Initialized to the empty string, signaling that the element is not filled.

justify
The default value for the attribute **justify**. Initialized to **left**. The legal values are **left**, **right**, and **center**.

linecolor
The default value for the attribute **color** of all elements having to draw lines (all but **text**). Initialized to **black**.

linestyle
The default value for the attribute **style** of all elements having to draw some line. Initialized to **solid**. The legal values are all those accepted by Tk_GetDash, and additionally all which are listed below:
solid, empty string
Draw solid line.
dash, dashed, -
Draw a dashed line.
dot, dotted,.
Draw a dotted line.

dash-dot, -.
Draw a dash-dotted line

dash-dot-dot, -..
Draw a dash-dot-dotted line.

linewidth
The default value for the attribute **stroke** of all elements having to draw some line. Initialized to **1** (pixels).

movelength
The default value for the directional specification of intermediate locations by the attribute **then** of **line** and **move** elements. Initialized to the pixel equivalent of **2 cm**.

slant
The default value for the attribute **slant** of **box** elements. Initialized to 90 degrees, i.e. slant straight up.

textcolor
The default value for the attribute **textcolor** of all elements having to draw some text. Initialized to **black**.

textfont
The default value for the attribute **textfont** of all elements having to draw some text. Initialized to **Helvetica 12pt**.

Diagram classes" The intended audience of this section are developers wishing to work

on the internals of the diagram package. Regular users of **diagram** can skip this section without missing anything.

The main information seen here is the figure below, showing the hierarchy of the classes implementing diagram.

IMAGE: figure-00-dependencies

At the bottom, all at the same level are the supporting packages like **snit**, etc. These can all be found in Tcllib.

Above them is the set of diagram classes implementing the various aspects of the system, i.e.:

diagram
The main class, that which is seen by the user.

diagram::core
The core engine, itself distributed over four helper classes.

diagram::basic
The implementation of the standard shapes, like box, circle, etc., based on the extension features of the core.

diagram::element
Core support class, the database of created elements. It also keeps the history, i.e. the order in which elements were created.

diagram::attribute
Core support class, the generic handling of definition and processing of attributes.

diagram::direction

Core support class, the database of named directions.

diagram::navigation
Core support class, the state of layout engine, i.e. current position and directin, and operations on it.

diagram::point
General support class handling various vector operations.

References

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *diagram* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Documentation tools

getstring

tk_getString
Keywords: tk_getString, dialog, entry, string

Name
getstring - A string dialog

Synopsis
package require Tcl 8.4
package require getstring ?**0.1**?
::getstring::tk_getString *pathName variable text* ?*options*?

Description
This package provides a dialog which consists of an Entry, OK, and Cancel buttons.

::getstring::tk_getString *pathName variable text* ?options?
Creates a dialog which prompts the user with *text* to input a text string. The contents of the entry are put in the *variable* upon closure of the dialog. The command returns a boolean indicating if the user pressed OK or not. If -geometry is not specified, the dialog is centered in its parent toplevel unless its parent is. in which case the dialog is centered in the screen. Options: -title -allowempty a boolean argument indicating if the dialog should accept an empty entry -entryoptions simply passes its arguments through to the entry widget. This is valuble for performing extra validation using the Entry widget validation hooks. -geometry specifies the geometry of the window

Example

```
package require getstring
namespace import getstring::*

if {[tk_getString .gs text "Feed me a string please:"]} {
    puts "user entered: $text"
}
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *getstring* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

history

tklib_history

Keywords: tklib_history, entry, history

Name

history - Provides a history for Entry widgets

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require history ?**0.3**?
::history::init *pathName* ?*length*?
::history::remove *pathName*
::history::add *pathName text*
::history::get *pathName*
::history::clear *pathName*
::history::configure *pathName option* ?*value*?

bell

Description

This package provides a convenient history mechanism for Entry widgets. The history may be accessed with the up and down arrow keys.

::history::init *pathName* ?length?
Arranges to remember the history of the named Entry widget. An optional length determines the number of history entries to keep. This may be changed later with **::history::configure**. History entries must be added with the **::history::add** command before they can be seen.

::history::remove *pathName*
Forgets all history entries for the Entry *pathName* and removes the history bindings.

::history::add *pathName text*
This command is used to add history entries to an Entry that has previously had **::history::init** called on it. This command should be called from your Entry handler with the contents of the entry (or whatever you wish to add to the history).

::history::get *pathName*
This command returns a list containing the history entries for the Entry *pathName*

::history::clear *pathName*
This command clears the history list for the named Entry.

::history::configure *pathName option* ?value?
This command queries or sets configuration options. Currently the options recognized are *length* and *alert*. Setting the length determines the number of history entries to keep for the named Entry. Alert specifies the command to run when the user reaches the end of the history, it defaults to

bell
. Although configure requires a *pathName* argument, the setting for alert is global and the path is ignored.

```
entry .e
bind .e <Return> [list ProcessEntry %W]
::history::init .e
pack .e

proc ProcessEntry {w} {
    set text [$w get]
    if {$text == ""} { return }
    ::history::add $w $text
    puts $text
    $w delete 0 end
}
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *entry::history* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

ico

ico

Keywords: ico, dll, entry, exe, icon

Name

ico - Reading and writing windows icons

Synopsis

package require Tcl 8.4
package require ico ?**1.1.3**?
::ico::icons *file* ?*option value...*?
::ico::iconMembers *file name* ?*option value...*?
::ico::getIcon *file name* ?*option value...*?
::ico::getIconByName *file name* ?*option value...*?

::ico::getFileIcon *file ?option value...?*
::ico::writeIcon *file name depth data ?option value...?*
::ico::copyIcon *file index file2 index2 ?option value...?*
::ico::EXEtoICO *file ?dir?*
::ico::clearCache *?file?*
::ico::transparentColor *image color*
::ico::Show *file ?option value...?*

Description

This package provides functions for reading and writing Windows icons from ICO, EXE, DLL, ICL, and BMP files. As used in this module an icon is a visual representation of an object. An icon consists of one or more images usually with varying resolution and color depth. Each icon and image has a resource identifier which may be a text string or a positive integer value. Most commands use this identifier to specify which icon or image to operate on.

API

::ico::icons *file ?option value...?*

Returns a list of icons found in *file* where each element is the name or numeric ID. Recognizes the following options:
- **-type** fileFormat

::ico::iconMembers *file name ?option value...?*

Returns a list of images that make up the icon with ID *name*. Each element is itself a sublist in the format {name width height bpp}. Recognizes the following options:
- **-type** fileFormat

::ico::getIcon *file name ?option value...?*

Extracts the icon with ID *name* from *file*. The default **-format** is **image** which will return the name of a Tk image containing the icon. The resolution and color depth are selected with the ?-res?, ?-bpp?, and ?-exact? options. If -exact is specified and there is no exact match, an error is thrown. Optionally **-image** may be used to specify the name of the Tk image that is created. If **-format** is **colors** then a list of color names in the #RRGGBB format is returned. Each list element is a horizontal row. Each horizontal row contains a list of colors for all the pixels in that row from left to right. If **-format** is **name** then the resource name of the image chosen is returned. This is useful for calling writeIcon or getIconByName. Recognizes the following *option*s.
- **-type** fileFormat
- **-format** value
- **-image** value
- **-res** value
- **-bpp** value
- **-exact** value

::ico::getIconByName *file name ?option value...?*

Extracts the image with ID *name* from *file*. This name should be the name of a specific image as returned by **::ico::iconMembers**, not an icon name returned from **::ico::icons**. If there is no matching resource ID in *file* an error is thrown. Recognizes the following options:

- **-type** fileFormat
- **-format** value

::ico::getFileIcon *file* ?*option value*...?

This command is only functional when running under Windows. It reads the Windows registry to determine the display icon for *file* as it would appear in Explorer or similar. *file* does not need to exist and may also be specified as a file extension with a leading dot. If *file* is a directory or you specify the special name **Folder** then the icon representing a folder is returned. This command takes the same arguments and usage as **getIcon**:
- **-format** value
- **-image** value
- **-res** value
- **-bpp** value
- **-exact** value

::ico::writeIcon *file name depth data* ?*option value*...?

Writes an image to *file*. *name* is the resource identifier of the image in *file* to write. When writing to an EXE, DLL, or ICL file you may only overwrite existing icons with an icon of the same dimensions and color depth. No icons may be added to these file types.

When writing to BMP the name is ignored as this type can contain only one image. This means if the file already existed it is completely overwritten.

When writing to an ICO or ICODATA file if the name specified does not exist then an image is appended and will be named the next in sequence (the specified name is ignored). Images in ICO and ICODATA files may be overwritten with differing dimensions or color depths. Note that you will get strange results when displaying icons if you fail to change every image which makes up a given icon.

integer *depth* (in)

This argument must have a value of **1**, **4**, **8**, **24**, or **32**. If *data* has more colors than the color depth allows an error will be generated.

options *data* (in)

This argument is either a list of colors in the format returned by **::ico::getIcon -format colors** or the name of a Tk image.

Recognizes the following *option*s.
- **-type** fileFormat

::ico::copyIcon *file index file2 index2* ?*option value*...?

Copies the icon at *index* in *file* to *index2* in *file2*.
- **-fromtype** fileFormat
- **-totype** fileFormat

::ico::EXEtoICO *file* ?dir?

Extracts all icons from the executable *file* to ICO files placed in *dir*. ?dir? defaults to the directory *file* is located in. Icon files will be named in the form *file*-ID.ico where ID is the icon resource identifier.
- **-type** fileFormat

::ico::clearCache ?file?
The **::ico::getIconList** command caches icon offsets inside EXE, DLL, ICL, and ICO files in order to speed up extraction. This command clears that cache for the specific ?file? or all files.

::ico::transparentColor *image color*
If *image* is a single word it is assumed to be the name of a Tk image. All pixels matching *color* in the *image* will be set transparent. Alternatively, *image* may be a color list in which case a modified list is returned.

::ico::Show *file* ?*option value*...?
Application level command which displays a window showing all the icons in *file* and their name.
- **-type** fileFormat
- **-parent** pathName

Example

```
button .explore -image [::ico::getIcon explorer.exe 0 -name
explore -res 16 -bpp 8]

set i [lsearch -inline [::ico::iconMembers tclkit.exe 0] {* 32 32
8}]
set colorlist [::ico::getIconByName tclkit.exe [lindex $i 0]
-format colors]
```

Limitations
Icons may not be added or removed from file types other than ICO. Icons in these files may only be replaced with icons of the same dimensions and color depth.

Icons of 8bpp or lower must include black in the pallete, this means if your icon does not have black in it, you will need to leave a color free so that it may be included by writeIcon.

There is currently no way to read alpha channel information from 32bpp icons.

Tk images do not have an alpha channel so the only way to write a true 32bpp icon is from a color list. writing a 32bpp icon from a Tkimage is identical to writing a 24bpp icon.

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *ico* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

ipentry

ipentry
Keywords: ipentry, entry, ip address, network

Name
ipentry - An IP address entry widget

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require ipentry ?**0.3.2**?
::ipentry::ipentry *pathName* ?*option value...*?
::ipentry::ipentry6 *pathName* ?*option value...*?
pathName **complete**
pathName **get**
pathName **insert** *iplist*
pathName **icursor** *index*
pathName **configure** *option value...*
pathName **cget** *option*

Description

This package provides a widget for the entering of a IP address.

::ipentry::ipentry *pathName* ?*option value*...?
Creates a new IPv4 ipentry widget and configures it with the given options and their values.

::ipentry::ipentry6 *pathName* ?*option value*...?
Creates a new ipentry widget for the entry of an IPv6 address. All options are the same as the IPv4 widget.

Each widget created with the command above supports the following methods:

***pathName* complete**
Returns a boolean value. True indicates that the entry contains a complete IP address, meaning all fields have a value. In some cases IPv6 address are valid when fields are missing. You will need to do your own validation to detect this.

***pathName* get**
Returns the contents of the entry as a list consisting of 4 or 8 elements.

pathName* insert *iplist
IPv4 Takes a list of 4 elements and inserts one into each quad of the entry, in order. All values in the list must be empty or integers. Values outside the range 0 to 255 are modified to be within the range. IPv6 Takes a list of 8 elements and inserts one into each quad of the entry, in order. All values in the list must be empty or 1 to 4 hex digits.

pathName* icursor *index
Sets the position of the widgets insertion cursor. Only integer values between 0 and 15 are valid for ipentry and 0 to 31 for ipentry6. Setting the icursor will only have an effect if the widget already has the input focus.

***pathName* configure *option value*...**
Modifies the configuration of the widget. For options and their meaning see the widget options section.

pathName* cget *option
Returns information about the current configuration of the widget, for the specified option. For options and their meaning see the widget options section.

Widget options

-textvariable (textvariable/Variable)
The name of a variable which holds the value of the IP address. The value must be a string of the form NNN.NNN.NNN.NNN for IPv4 or HHHH:HHHH:HHHH:HHHH:HHHH:HHHH:HHHH:HHHH for IPv6 where H is a hex digit. The variable will be modified to represent a valid IP address if it is not already.

-state (state/State)
Specifies one of three states for the entry: **normal**, **disabled**, or **readonly**.

-font (font/Font)
see below ...

-bd (borderWidth/BorderWidth)
see below ...

-fg (foreground/Foreground)
see below ...

-bg (background/Background)
see below ...

-relief (relief/Relief)
see below ...

-highlightthickness (highlightThickness/HighlightThickness)
see below ...

-highlightcolor (highlightColor/HighlightColor)
see below ...

-highlightbackground (highlightBackground/HighlightBackground)
see below ...

-selectbackground (selectBackground/Background)
see below ...

-selectforeground (selectForeground/Foreground)
see below ...

-selectborderwidth (selectBorderWidth/BorderWidth)
see below ...

-disabledbackground (disabledBackground/DisabledBackground)
see below ...

-disabledforeground (disabledForeground/DisabledForeground)
see below ...

-readonlybackground (readonlyBackground/ReadonlyBackground)
see below ...

-insertbackground (insertBackground/Background)
Standard widget options. See **options** for a description of their meanings and values.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *ipentry* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Widget

khim

khim

Keywords: khim, character, i18n, input, international, method

Name

khim - Provides key bindings for entering international characters on a keyboard that does not support them

Synopsis

package require Tcl
package require khim ?**1.0.3**?
::khim::getOptions *path*
::khim::getConfig
::khim::setConfig *version enabled compose map*
::khim::showHelp *path*

Description

This package provides a set of key bindings to allow a user to enter arbitrary characters on a keyboard that does not support them. It works by designating some seldom-used key of the keyboard as a "Compose" key (this key is distinct from any key so labeled, and is often "Pause," "F12" or "L2"), and having the "Compose" key, followed by a two-key sequence, have the effect of inserting some character in a widget. In addition, the "Compose" key, when struck twice in succession, brings up a dialog containing a Unicode character map, allowing arbitrary characters to be inserted.

The vast bulk of the package's functionality is implemented in a single bindtag, **KHIM**. An application can request that any text or entry widget use the package to allow for entry of arbitrary characters by placing this binding tag ahead of the **Text** or **Entry** binding tag for the widget:

```
text .t -width 80 -height 24
bindtags .t {.t KHIM Text . all}
```

Note that the **KHIM** tag must precede the **Text** or **Entry** class binding, or the package will not function correctly.

Procedures

In addition to commands supporting the KHIM binding tag, the following commands are exported from the package:

::khim::getOptions *path*
: Posts a top-level modal dialog with the path name *path* that prompts the user for KHIM options. The user is allowed to reconfigure the key sequences for the "Compose" key, change the choice of key to use for the "Compose" function, and enable/disable the KHIM key bindings entirely.

::khim::getConfig

Returns a Tcl script that restores the current configuration of KHIM: the enabled/disabled state, the choice of "Compose" key, and the key sequences that may be composed. This script is designed to be saved to a configuration file for use in a subsequent invocation of the same application:

```
# Save KHIM configuration
set f [open ~/.khimrc w]
puts $f [::khim::getConfig]
close $f

# Restore KHIM configuration
source ~/.khimrc
```

::khim::setConfig *version enabled compose map*
Restores an earlier saved configuration. Few, if any, applications will call this command in any other way than to evaluate it as returned from **::khim::getConfig**.

::khim::showHelp *path*
Displays a top-level dialog giving user-level help for KHIM; the dialog will have the path name *path*.

Localisation

Programmers who wish to make KHIM available in a non-English-speaking locale may do so by providing a **.msg** file with the appropriate localised text. The catalog requires the following messages:

Apply
Text that will appear on the "Apply" button in the dialog that sets KHIM options.

Cancel
Text that will appear on the "Cancel" button in several dialogs.

Change
Text that will appear on the "Change" button, which alters the binding of a pair of composed characters (creating or replacing as appropriate).

Character
Text that will appear on the label of the entry widget that accepts a character resulting from a composed sequence.

{Compose Key}
Window title for a dialog that prompts the user to strike the key that will be used for the "Compose" key.

{Compose key:}
Label that identifies a component showing the "Compose" key choice in the KHIM options dialog.

{Composed sequence must be two characters long}
Error message that is displayed if the user attempts to define a "Compose" sequence that is shorter or longer than two characters.

Delete
Text for a button that deletes a "Compose" sequence.

Help...
Text for a button that displays the KHIM user help dialog.

HELPTEXT
Complete text for the user-level help for KHIM. Refer to "*en.msg*" for the English-language version of the help.

{Input key sequence}
Text for a label of the entry widget that prompts the user for a two-character sequence to use with the "Compose" key.

{Insert Character}
Window title of the dialog box that displays a Unicode character map and prompts the user to select a character to insert.

{Key sequences}
Text for a label at the head of a listbox showing the composed sequences that are currently bound.

{KHIM Controls}
Window title for the dialog box that prompts for KHIM settings.

{KHIM Help}
Window title for the window that display help text for KHIM.

OK
Label for the OK button on several dialogs.

Select code page:
Label for a spinbox that prompts the user for a Unicode code page number.

SELECT COMPOSE KEY
A message, which should be composed in short lines, prompting the user to press the key that will become the "Compose" key in KHIM.

Unicode...
Text for a button that brings up the character map to select the character to which a composed sequence binds.

{Use KHIM}
Text for a checkbutton that asks whether the user wishes to use KHIM to manage composed key sequences.

Acknowledgments

KHIM was originally inspired by the key bindings that Brent Welch developed for the 'sedit' editor used in the 'exmh' mail user agent. The code for KHIM is entirely separate from that for 'sedit'.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *khim* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

map

area-display

Keywords: area-display, area display, tabular, map

Name
map::area::display - Widget to display a single geo/area definition

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::area::display ?**0.1**?

Description
This package provides a widget to show the information of a single *geo/area* definition.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

area-file
Keywords: area-file, file, geo/area, map

Name
map::area::file - Reading/writing tklib geo/area files

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::file ?**0.1**?

Description
This package provides read/write functions for Tklib *geo/area* files.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

area-map-display
Keywords: area-map-display, addon, area display, map display, map

Name
map::area::map-display - Map Action Engine - Layer to display area definitions

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::polyline
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::map-display ?**0.1**?

Description
This package provides a map::display attachment extending the map's behaviour. This attachment shows a set of *geo/area* definitions. It ensures that only the visible areas use canvas resources (items).

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

area-store-fs
Keywords: area-store-fs, filesystem store, geo/area, map, store, filesystem

Name
map::area::store::fs - Filesystem-based store of geo/area definitions

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::file
package require map::area::store::fs ?**0.1**?

Description
This package provides a store loading *geo/area* definitions from a directory in the filesystem.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

area-store-mem
Keywords: area-store-mem, center, geo/area, diameter, geo/area pixels, zoom, memory store, perimeter length, length, perimeter, map, pixels, store, memory

Name
map::area::store::memory - In-memory store for geo/area definitions

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::area::store::memory ?**0.1**?

Description
This package provides an in-memory store for *geo/area* definitions, with memoized calculation of extended attributes.

The base data is pulled from a backing store. A map::area::store::fs or anything API-compatible to it.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

area-table-display
Keywords: area-table-display, area display, tabular, map

Name
map::area::table-display - Widget to display a table of area definitions

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::area::table-display ?**0.1**?

Description
This package provides a widget showing the information of many *geo/area* definitions in a table.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

box-display
Keywords: box-display, box display, tabular, map

Name
map::box::display - Widget to display a single box definition

Synopsis
package require Tcl 8.6-

package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::box::display ?**0.1**?

Description
This package provides a widget to show the information of a single *geo/box* definition.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

box-entry
Keywords: box-entry, addon, box entry, map display, map

Name
map::box::entry - Map Action Engine - Box Entry

Synopsis
package require Tcl 8.6-
package require canvas::edit::rectangle
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::entry ?**0.1**?

Description
This package provides a map::display attachment extending the map's behaviour. This attachment enables the user to enter a (bounding) *geo/box*.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems.

Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

box-file
Keywords: box-file, file, geo/box, map

Name
map::box::file - Reading/writing tklib geo/box files

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::file ?**0.1**?

Description
This package provides read/write functions for Tklib *geo/box* files.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

box-map-display
Keywords: box-map-display, addon, box display, map display, map

Name
map::box::map-display - Map Action Engine - Layer to display box definitions

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::rectangle
package require debug
package require debug::caller
package require map::slippy 0.8

package require snit
package require map::box::map-display ?**0.1**?

Description
This package provides a map::display attachment extending the map's behaviour. This attachment shows a set of *geo/box* definitions. It ensures that only the visible boxes use canvas resources (items).

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

box-store-fs
Keywords: box-store-fs, filesystem store, geobox, map, store, filesystem

Name
map::box::store::fs - Filesystem-based store of geo/box definitions

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::store::fs ?**0.1**?

Description
This package provides a store loading *geo/box* definitions from a directory in the filesystem.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

box-store-mem

Keywords: box-store-mem, center, geo/box, diameter, geo/box pixels, zoom, memory store, perimeter length, length, perimeter, map, pixels, store, memory

Name

map::box::store::memory - In-memory store for geo/box definitions

Synopsis

package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::box::store::memory ?**0.1**?

Description

This package provides an in-memory store for *geo/box* definitions, with memoized calculation of extended attributes.

The base data is pulled from a backing store. A map::box::store::fs or anything API-compatible to it.

API

Example

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Map Display and Supporting Utilities

box-table-display

Keywords: box-table-display, box display, tabular, map

Name

map::box::table-display - Widget to display a table of box definitions

Synopsis

package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit

package require tablelist
package require map::box::table-display ?**0.1**?

Description
This package provides a widget showing the information of many *geo/box* definitions in a table.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

display
Keywords: display, map, map display

Name
map::display - Map Display Widget

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::sqmap
package require canvas::zoom
package require crosshair
package require debug
package require debug::caller
package require snit
package require widget::scrolledwindow
package require map::display ?**0.1**?

Description
This package provides a map display widget with basic behaviour (Drag, Center, Zoom +/-). The map tiles are fetched from a configurable provider like map::provider::osm or anything API compatible. The widget is extensible with attachable engines providing additional behaviours, like the display of geo/* resources, and/or editing of geo/* resources.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for

enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

map_overview
Keywords: map_overview, map

Name
map_overview - Overview of the packages in the Map module

Description
Storage
see below ...

File
Definitions of simple file formats to hold various geo/* resources, and the ability to read/write these files.
- map::area::file
- map::box::file
- map::point::file
- map::track::file

Disk
On-disk stores for various geo/* resources. Exist as basic examples to start with, and as API demonstrators. In the original project these packages were factored out of **sqlite** was used as a store. This store is however entwined too much with that project to be factored.
- map::area::store::fs
- map::box::store::fs
- map::point::store::fs
- map::track::store::fs

Memory
In-memory stores caching geo/* data and adding derived attributes. The bridge between the actual on-disk stores and the various widgets and behaviours.
- map::area::store::memory
- map::box::store::memory
- map::point::store::memory
- map::track::store::memory

Widgets
To show maps, singular geo resources, and tables of many geo resources of the same kind.

Map
map::display

Geo Resource Tables
- map::area::table-display
- map::box::table-display
- map::point::table-display

- map::track::table-display

Geo Resource Display
- map::area::display
- map::box::display
- map::track::display

Behaviours

Engines attachable to map::display to add custom behaviours to the shown map. The two classes of engines are for displaying overlays for specific geo resources, and the editing/entry of specific single geo resources.

Display
- map::area::map-display
- map::box::map-display
- map::point::map-display
- map::track::map-display

Editing
- map::box::entry
- map::mark
- map::track::entry

Support

map::provider::osm

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Map Display and Supporting Utilities

mark

Keywords: mark, location marking, map, mark location

Name

map::mark - Map Action Engine - Mark A Point

Synopsis

package require Tcl 8.6-
package require debug
package require debug::caller
package require snit
package require map::mark ?**0.1**?

Description

This package provides a map::display attachment extending the map's behaviour. This attachment enables the user to mark locations, i.e. *geo/point*s. The marked locations are reported via a callback.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

point-file
Keywords: point-file, file, geo/point, map

Name
map::point::file - Reading/writing tklib geo/point files

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::file ?**0.1**?

Description
This package provides read/write functions for Tklib *geo/point* files.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

point-map-display
Keywords: point-map-display, addon, point display, map display, map

Name
map::point::map-display - Map Action Engine - Layer to display point definitions

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::points
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::map-display ?**0.1**?

Description
This package provides a map::display attachment extending the map's behaviour. This attachment shows a set of *geo/point* definitions. It ensures that only the visible points use canvas resources (items).

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

point-store-fs
Keywords: point-store-fs, filesystem store, geo/point, map, store, filesystem

Name
map::point::store::fs - Filesystem-based store of geo/point definitions

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::store::fs ?**0.1**?

Description
This package provides a store loading *geo/point* definitions from a directory in the filesystem.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

point-store-mem
Keywords: point-store-mem, center, geo/point, diameter, geo/point pixels, zoom, memory store, perimeter length, length, perimeter, map, pixels, store, memory

Name
map::point::store::memory - In-memory store for geo/point definitions

Synopsis
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::point::store::memory ?**0.1**?

Description
This package provides an in-memory store for *geo/point* definitions, with memoized calculation of extended attributes.

The base data is pulled from a backing store. A map::point::store::fs or anything API-compatible to it.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

point-table-display
Keywords: point-table-display, map, point display, tabular

Name
map::point::table-display - Widget to display a table of point definitions

Synopsis

package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require .map::point::table-display ?**0.1**?

Description

This package provides a widget showing the information of many *geo/point* definitions in a table.

API

Example

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Map Display and Supporting Utilities

provider-osm

Keywords: provider-osm, map, mapnik, openstreetmap, provider, tiles, tile provider

Name

map::provider::osm - Tile provider using OpenStreetMap Mapnik as origin

Synopsis

package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require map::slippy::cache
package require map::slippy::fetcher
package require snit
package require map::provider::osm ?**0.1**?

Description

This package provides a tile provider suitable for use with map::display. It uses OpenStreetMap Mapnik as origin and caches fetched tiles in the local filesystem.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-display
Keywords: track-display, map, tabular, track display

Name
map::track::display - Widget to display a single track definition

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::track::display ?**0.1**?

Description
This package provides a widget to show the information of a single *geo/track* definition.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-entry
Keywords: track-entry, addon, track entry, map display, map

Name
map::track::entry - Map Action Engine - Track Entry

Synopsis
package require Tcl 8.6-
package require canvas::edit::polyline
package require debug
package require debug::caller
package require snit
package require map::slippy 0.8
package require map::track::entry ?**0.1**?

Description
This package provides a map::display attachment extending the map's behaviour. This attachment enables the user to enter an open series of points, i.e. a *geo/track*.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-file
Keywords: track-file, file, geo/track, map

Name
map::track::file - Reading/writing tklib geo/track files

Synopsis
package require Tcl 8.6-
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::file ?**0.1**?

Description
This package provides read/write functions for Tklib *geo/track* files.

API

Example

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-map-display
Keywords: track-map-display, addon, track display, map display, map

Name
map::track::map-display - Map Action Engine - Layer to display track definitions

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require canvas::edit::polyline
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::map-display ?**0.1**?

Description
This package provides a map::display attachment extending the map's behaviour. This attachment shows a set of *geo/track* definitions. It ensures that only the visible tracks use canvas resources (items).

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-store-fs
Keywords: track-store-fs, filesystem store, geo/track, map, store, filesystem

Name
map::track::store::fs - Filesystem-based store of geo/track definitions

Synopsis
package require Tcl 8.6-

package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::store::fs ?**0.1**?

Description
This package provides a store loading *geo/track* definitions from a directory in the filesystem.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-store-mem
Keywords: track-store-mem, center, geo/track, diameter, geo/track pixels, zoom, memory store, perimeter length, length, perimeter, map, pixels, store, memory

Name
map::track::store::memory - In-memory store for geo/track definitions

Synopsis
package require Tcl 8.6-
package require Tcl 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require snit
package require map::track::store::memory ?**0.1**?

Description
This package provides an in-memory store for *geo/track* definitions, with memoized calculation of extended attributes.

The base data is pulled from a backing store. A map::track::store::fs or anything API-compatible to it.

API

Example

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

track-table-display
Keywords: track-table-display, map, tabular, track display

Name
map::track::table-display - Widget to display a table of track definitions

Synopsis
package require Tcl 8.6-
package require Tk 8.6-
package require debug
package require debug::caller
package require map::slippy 0.8
package require scrollutil
package require snit
package require tablelist
package require map::track::table-display ?**0.1**?

Description
This package provides a widget showing the information of many *geo/track* definitions in a table.

API

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *map* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Map Display and Supporting Utilities

menubar

menubar
Keywords: menubar

Name
menubar - Creates an instance of the *menubar* Class.

Synopsis

package require Tcl 8.6
package require Tk 8.6
package require menubar ?**0.5.1**?
menubar new ?*options*?
mBarInst **define** *body*
mBarInst **install** *pathName body*
mBarInst **menu.configure** *option tag-settings* ?option tag-settings ...?
mBarInst **menu.namespace** *tag namespace*
mBarInst **menu.hide** *tag*
mBarInst **menu.show** *tag*
mBarInst **tag.add** *tag value*
mBarInst **tag.configure** *pathName tag* ?option value ...option value?
mBarInst **tag.cget** *pathName tag* ?option?
mBarInst **group.add** *tag label* ?*cmd*? ?*accel*? ?*sequence*? ?state?
mBarInst **group.delete** *tag label*
mBarInst **group.move** *direction tag label*
mBarInst **group.configure** *tag label* ?option value ...option value?
mBarInst **group.serialize** *tag*
mBarInst **group.deserialize** *tag stream*
mBarInst **notebook.addTabStore** *pathname*
mBarInst **notebook.deleteTabStore** *pathname*
mBarInst **notebook.setTabValue** *pathname tag*
mBarInst **notebook.restoreTabValues** *pathname*

Description

menubar new ?options?

see below ...

Create and return a new instance of the menubar class. The menubar class encapsulates the definition, installation and dynamic behavior of a menubar. The class doesn't depend on a widget framework and therefore can be used with or without a framework (e.g. Bwidget, IWidget, Snit, etc.). Unlike other Tk widget commands, the menubar command doesn't have a *pathName* argument because menubars are handled by the window manager (i.e. wm) and not the application.

Options

The following options can be passed to the *menubar new* command.

These options are inherited from the Tk menu command, their effect is platform specific.

- -activebackground
- -activeborderwidth
- -activeforeground
- -background
- -borderwidth
- -cursor
- -disabledforeground

- -font
- -foreground
- -relief

Introduction

An instance of the menubar class provides methods for compiling a description of the menubar, configuring menu items and installing the menubar in toplevel windows.

A menubar can be thought of as a tree of cascading menus. Users define a menubar using a language that results in a human readable description of a menubar. The description of the menubar is then compiled by an instance of the menubar class after which it can be installed in one or more toplevel windows.

The menubar class provides many unique capabilities that are not found in other tcl/tk menubar implementation. Some of these are:

- A tagging system that simplifies access to menu entries in the menu tree.
- Support for user defined tags that depend on the toplevel window context.
- A simplified and uniform interface for all callback commands.
- Namespace support for all callback commands so callback commands can be easily grouped into namespaces.
- Support for hiding and exposing menus on the menubar.
- A simplified method for creating radiobutton groups.
- Automatic management of state variables for checkbuttons and radiobuttons.
- Scope control for the state variables of checkbuttons and radiobuttons.
- Tear-off menu management that ensures only one tearoff menu is created.
- Support for dynamic menu extension to simplify the creation of recent document menus.
- Support for saving and restoring dynamic menu extensions.

Terminology

MENUBAR

The visible rendering of a menubar in a toplevel window is a horizontally group of cascading Tk menus.

MENU

A menu is an ordered list of items that is rendered vertically. Menus are not visible until a user preforms some action (normally a event). A menu may contain any number of child menus that are rendered as cascading menus. Cascading menus are rendered next to the parent menu when they are activated.

MENU ENTRY

A menu contains an ordered list of items called entries. Menu entries have a type and the menubar class supports the following 6 entry types: *Command*, *Checkbutton*, *Radiobutton*, *Separator*, *Group* and *Menu*.

ENTRY LABEL

Each menu entry has a visible string that is called the entry label.

TAG

A tag is name that is normally used to referr to an item in a menu tree. A tag name is an alphanumeric character string that may include the underscore character. Menu tree tags are defined for all nodes and leafs in a menu tree. This provides a flat abstraction of the tree and

simplifies item referencing in menubar methods. Without this abstraction it would be necessary to reference menu elements using a tree path which could change at run-time. The menubar class also has a method that can create a user defined tag. User defined tags store values that change based on the currently active toplevel window. User defined tags can be used to store widget pathnames use by callback code so that output can be routed to the appropriate toplevel window.

Methods

mBarInst define *body*

Compiles *body* into a tree of menu entries which define the visual layout of the menubar. The *body* argument describes the layout using the following syntax, where the elements of the syntax are described below.

body == definitions

```
definitions    ::= { <ignore> | <definition> | <definition>
    <definitions> }
ignore         ::= { <nl> | <white-space> <nl> | # <comment> <nl> }
definition     ::= { <command> | <checkbutton> | <radiobutton> |
    <separator> | <group> | <menu> }
command        ::= <label> C <tag> <nl>
checkbutton    ::= <label> X<scope> { <tag> | <tag>+ } <nl>
radiobutton    ::= <label> R<scope> { <tag> | <tag>+ } <nl>
separator      ::= <stext> S <tag> <nl>
group          ::= <dummy> G <tag> <nl>
menu           ::= <label> { M:<tag> | M:<tag>+ } <nl> <definitions>
stext          ::= '--' | <label>
scope          ::= '' | '@' | '='
```

C - Command

The C type entry is the most common type of entry. This entry executes a command when it is invoked.

X - Checkbutton

A X type entry behaves much like a Tk checkbutton widget. When it is invoked it toggles back and forth between a selected and deselected states. The value of a checkbutton is a boolean (i.e. 1 or 0). By default all checkbuttons are deselected. If you want the checkbutton to be initially selected then include a trailing plus (+) with the tag name. See SCOPE CONTROL below for a description of the scope indicator.

R - Radiobutton

A R type menu entry behaves much like a Tk radiobutton widget. Each radiobutton entry is a member of a radiobutton group that controls the behavior of the radiobuttons in the group. All radiobuttons in a group are given the same tag name. In the example below Red, Green and Blue all have the same tag and are therefore all in the same radiobutton group. A trailing plus (+) on the tag name of a radiobutton entry will cause the entry to be the initially selected entry. See SCOPE CONTROL below for a description of the scope indicator.

S - Separator

A S type menu entry is an entry that is displayed either as a horizontal dividing line or a

label. Separators are not active elements of a menu and have no associated behavior if they are invoked. If is two dashes (i.e. '--') then the separator will be displayed as a horizontal line otherwise will be displayed as a bold label surrounded by double dashes (e.g. "-- --") with a lightgray background.

G - Command Group
The G type menu entry marks a location in the menu tree where entries can be dynamically added and removed. Menu extension can only occur at the end of a menu so G type entries must be the last item on a menu. A G type entry is rendered as a separator line. The *group.* sub-commands are used to manipulate command group entries.

M - Menu
An M type entry is used to define both menubar menus and cascading menus. Menu entries are the most complicated of the 6 menu types. A menu entry is composed of three list elements. The first element of the list is its label. The second element of the list is a composite string consisting of a type identifier (M) followed by an optional tag (beginning with a ':' separator) and finally an optional plus (+) which indicates that the menu is a tear-off menu. The final element of the list is a LIST VALUE.

mBarInst* install *pathName body
The *install* method installs the menubar created with the *define* method into toplevel window *pathName*. The *body* argument of the command contains a tcl script which is used to initialize the installed menubar. Normally the tcl script will contain calls to various menubar methods to perform the initialization. The initialization code is only run once when the menubar is installed. The namespace in which the *install* method is executed becomes the default namespace for callback commands (see *menu.namespace* below for more details).

Methods - menu.xxx

mBarInst* menu.configure *option tag-settings ?option tag-settings ...?
Configures the tags of a menubar and returns an empty string. This method provides a convenient way to configure a larger number of tags without the verbosity of using the *tag.configure* method.

option
Option may have any of the values accepted by the *tag.configure* method.

tag-settings
The *tag-settings* argument is a string that is converted to a list of tag-value pairs using the following syntax.

Syntax for *tag-settings*.

```
tag-settings ::= { <ignore> | <value> | <value> <tag-settings> }
ignore       ::= { <nl> | <white-space> <nl> | # <comment> <nl> }
value        ::= <tag> <option-value> <nl>
```

mBarInst* menu.namespace *tag namespace
Change the namespace for a sub-tree of the menubar starting at entry *tag*. The new value will be *namespace*. Each entry in the menubar tree has an associated namespace which will be used for its callback procedure. The default namespace is the namespace where the *install* method was executed. The *namespace* method can be used to change the namespace that

will be used for callbacks in a sub-tree of the menubar. This method can only be used in the context of an *install* script.

mBarInst* menu.hide *tag
Remove (hide) a menubar entry. When a menubar tree is defined all entries are visible by default. This method can be used to hide a menubar entry. The *hide* methods can be used in the context of an *install* script so that a menu will be initially hidden at application start up. The *tag* argument is the tag name of the menu to be hidden.

mBarInst* menu.show *tag
Exposes (shows) a hidden menubar entry. When a menubar tree is defined all entries are visible by default. If a entry is hidden from the user (using the menu.hide method) then it can be exposed again using the show method. The *tag* argument is the tag name of the menu to be shown.

Methods - tag.xxx

mBarInst* tag.add *tag value
Add a user defined *tag* value. The *tag.add* method adds a new tag-value pair to the the tags defined for a menubar. User defined tags are different from the tags created by the *define* method. The *tag.add* method can only be used in an *install* script and its value is associated with the toplevel where the menubar is installed. This makes the tag context sensitive so callback code that queries the tag value will receive a value that is associated with the window that performed the callback.

mBarInst* tag.configure *pathName tag ?option value ...option value?
Given the *pathName* of a toplevel window and a *tag* this method configures the menu entry associated with the tag and return an empty string.

Standard Options
These option are the same as those described for menu entries in the Tk *menu* documentation.
- **-activebackground**
- **-activeforeground**
- **-background**
- **-bitmap**
- **-columnbreak**
- **-compound**
- **-font**
- **-foreground**
- **-hidemargin**
- **-image**
- **-indicatoron**
- **-label**
- **-selectcolor**
- **-selectimage**
- **-state**

Class Specific Options
see below ...
-bind {uline accel sequence}

The value of the *-bind* option is three element list where the values are as follows.

uline
An integer index of a character to underline in the entry. This value performs the same function as the Tk *menu* -underline option. If this value is an empty string then no underlining is performed.

accel
A string to display at the right side of the menu entry. The string normally describes an accelerator keystroke sequence that may be typed to invoke the same function as the menu entry. This value performs the same function as the Tk *menu* -accelerator option. If this value is an empty string then no accelerator is displayed.

sequence
A bind sequence that will cause the entries associated command to fire.

-command cmdprefix
The value of the *-command* option a command prefix that is evaluated when the menu entry is invoked. By default the callback is evaluate in the namespace where the *install* method was executed. Additional values are appended to the *cmdprefix* and are thus passed to the callback command as argument. These additional arguments are described in the list below.

command entry
1) The pathname of the toplevel window that invoked the callback.

checkbutton entry
1) The pathname of the toplevel window that invoked the callback.
1. The checkbutton's tag name
2. The new value for the checkbutton

radiobutton entry
1) The pathname of the toplevel window that invoked the callback.
1. The radiobutton's tag name
2. The label of the button that was selected

group entry
1) The pathname of the toplevel window that invoked the callback.

mBarInst* tag.cget *pathName tag ?option?
Returns the value of the configuration option given by *option* or the value of a user defined tag. The option argument may be any of the options accepted by the *tag.configure* method for the *tag* type. User defined tags are queried without an *option* value.

Methods - group.xxx

mBarInst* group.add *tag label ?cmd? ?accel? ?sequence? ?state?
Add a command to the group with tag name *tag*. This method appends a new command entry to the end of a command group. The order of the arguments is fixed but arguments to the right can be ignored. Arguments to this method have the following meaning.

tag *(string)*
The tag name of the command group.

label *(string)*
The displayed label for the menu entry.

cmd *(string)*
A command prefix that will be used for callback command.

accel *(string)*
An accelerator string that will be displayed next to the entry label.

sequence *(string)*
A bind sequence that will be bound to the callback command.

state *(enum)*
Sets the active state of the command. One of: normal, disabled, active

mBarInst* group.delete *tag label
Delete a command from a group with tag name *tag*. This method deletes command *label* from a command group.

mBarInst* group.move *direction tag label
Change the position of an entry in a group with tag name *tag*. The *direction* argument is the direction ('up' or 'down') the entry will be moved. The entry that is moved has the name *label*.

mBarInst* group.configure *tag label ?option value ...option value?
Configure the options of an entry in the command group with tag name *tag*. This method is similar to the *tag.configure* method except that it works on entries in a command group. Set documentation for the *tag.configure* method (above) for more details on command entry options.

mBarInst* group.serialize *tag
Return a string serialization of the entries in a command group. The argument *tag* is the tag name for the group that is to be serialized. The resulting serialization is a list containing three element (1) the tag name of the group (2) a dictionary containing group level options (3) a list of zero or more similar three element lists that describe the entries in the group.

mBarInst* group.deserialize *tag stream
Replace the contents of group tag *tag* with the commands defined in the serialization *stream*. The original contents of the group are lost.

Methods - notebook.xxx

mBarInst* notebook.addTabStore *pathname
This method should be used in code that creates a new notebook tab. Execution of this method will cause state storage to be allocated for the new notebook tab. The pathname for the notebook tab is passed as an argument to the method.

mBarInst* notebook.deleteTabStore *pathname
This command deallocates the state store for a notebook tab. The pathname for the notebook tab is passed as an argument to the method.

mBarInst* notebook.setTabValue *pathname tag
This method should be used in the callback for menubar checkbuttons or radiobuttons that have notebook tab scope control. When this method is executed it will move the value associated with tag into the tab store for the tab identified by pathname.

mBarInst* notebook.restoreTabValues *pathname

This method should be place in a bind script that is triggered by a notebooks <> event.

Scope Control

By default a menubar instance looks the same in all installed toplevel windows. As changes are made to one instance of a menubar all the other instances are immediately updated. This means the internal state of all the menu entries for the instances are synchronized. This behavior is called global scope control of the menubar state.

The menubar class allows finer scope control on check and radio buttons. The scope of these entry types can be modified by adding a modifier character to their type character. Two modifier characters are supported as show in the table below.

```
''  ::= global scope (no character)
'@' ::= local scope modifier
'=' ::= notebook tab scope modifier
```

When the local scope character (@) is added to the definition of a button, the button is given a new variable for each installed toplevel window. This has the effect of making the button's state local to the window (i.e. local scope). An example use case for this behavior might be a status bar that can be toggled on an off by a checkbutton. The developer may want to allow the user to control the visibility of the status bar on a per window basis. In this case a local modifier would be added to the status bar selector so the callback code would receive an appropriate value based on the current toplevel window.

The notebook tab scope character (=) is similar in effect to the local scope character but it allows a notebook tab selection to also manage the state of of a button. Adding the notebook tab scope modifier enables notebook tab scope control but the developer must then make use of the notebook.xxxx sub-commands to actively manage state values as tabs are added, deleted and selected.

Example

```
package require Tcl
package require Tk
package require menubar

set tout [text .t -width 25 -height 12]
pack ${tout} -expand 1 -fill both
set mbar [menubar new \
    -borderwidth 4 \
    -relief groove  \
    -foreground black \
    -background tan \
    ]
${mbar} define {
    File M:file {
        Exit                C      exit
    }
```

```
    Edit M:items+ {
    #   Label               Type    Tag Name(s)
    #   -----------------   ----    ---------
        "Cut"               C       cut
        "Copy"              C       copy
        "Paste"             C       paste
        --                  S       s2
        "Options" M:opts {
            "CheckList" M:chx+ {
                Coffee      X       coffee+
                Donut       X       donut
                Eggs        X       eggs
                }
            "RadioButtons" M:btn+ {
                "Red"       R       color
                "Green"     R       color+
                "Blue"      R       color
                }
        }
    }
    Help M:help {
        About               C       about
    }
}
${mbar} install . {
    ${mbar} tag.add tout ${tout}
    ${mbar} menu.configure -command {
        # file menu
        exit            {Exit}
        # Item menu
        cut             {CB Edit cut}
        copy            {CB Edit copy}
        paste           {CB Edit paste}
        # boolean menu
        coffee          {CB CheckButton}
        donut           {CB CheckButton}
        eggs            {CB CheckButton}
        # radio menu
        color           {CB RadioButton}
        # Help menu
        about           {CB About}
    } -bind {
        exit        {1 Cntl+Q  Control-Key-q}
        cut         {2 Cntl+X  Control-Key-x}
        copy        {0 Cntl+C  Control-Key-c}
        paste       {0 Cntl+V  Control-Key-v}
```

```
        coffee      {0 Cntl+A  Control-Key-a}
        donut       {0 Cntl+B  Control-Key-b}
        eggs        {0 Cntl+C  Control-Key-c}
        about       0
    } -background {
        exit red
    } -foreground {
        exit white
    }
}
proc pout { txt } {
    global mbar
    set tout [${mbar} tag.cget . tout]
    ${tout} insert end "${txt}\n"
}
proc Exit { args } {
    puts "Goodbye"
    exit
}
proc CB { args } {
    set alist [lassign ${args} cmd]
    pout "${cmd}: [join ${alist} {, }]"
}
wm minsize . 300 300
wm geometry . +4+4
wm protocol . WM_DELETE_WINDOW exit
wm title . "Example"
wm focusmodel . active
pout "Example started ..."
```

Caveats

This implementation uses TclOO so it requires 8.6. The code has been tested on Windows (Vista), Linux and OSX (10.4).

References

See also

1. menu
2. A command that creates menubar objects

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *menubar* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright

```
Copyright (c) 2009 Tom Krehbiel <krehbiel.tom@gmail.com> All rights
    reserved.
```

notifywindow

notifywindow

Keywords: notifywindow, alert, notification, tk

Name

notifywindow - Provides unobtrusive window for alerts/notifications from Tk applications

Synopsis

package require tk
package require notifywindow ?**1.0.2**?
::notifywindow::notifywindow *msg img*
::notifywindow::demo

Description

This package provides a low-key, unobtrusive window for alerts and notifications from Tk applications that briefly appears in the upper-right hand corner of the screen. Modeled after the "Growl" framework on Mac OS X, the window fades in and out on platforms that support such effects, and appears and disappears on other platforms.

::notifywindow::notifywindow *msg img*
: Posts the toplevel notification window with the *msg* string, which should not exceed 40 characters, and an optional image, which should be 16x16 pixels.

::notifywindow::demo
: Simple demo that illustrates how the package works.

Acknowledgments

While **notifywindow** was inspired by the Growl notification package for Mac OS X, it shares no code with that project whatsoever.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *notifywindow* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright

```
Copyright (c) 2015 Kevin Walzer/WordTech Communications LLC
    <kw@codebykevin.com>
```

ntext

ntext

Keywords: ntext, bindtags, re_syntax, regexp, text

Name

ntext - Alternative Bindings for the Text Widget

Synopsis

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?
::ntext::new_textCopy *pathName*
::ntext::new_textCut *pathName*
::ntext::new_textPaste *pathName*
::ntext::syncIndentColor *pathName*
::ntext::syncTabColor *pathName*

Description

The purpose of the **ntext** package is to make the text widget behave more like other text-editing applications. It makes the text widget more useful for implementing a text editor, and makes it behave in a way that will be more familiar to most users.

The package provides a binding tag named *Ntext* for use by text widgets in place of the default *Text* binding tag.

Package **ntext** 's functions and variables are contained entirely in the **::ntext** namespace; its other code is contained in the binding tag *Ntext*. **ntext** has no exports to the global or other namespaces, and no new widget commands. It uses modified copies of the **Tk** code, leaving the original code, and the *Text* binding tag, unchanged.

The differences between the *Ntext* binding tag and the default *Text* binding tag are in three categories:

- Some *Text* bindings behave differently from most text-editing applications. *Ntext* gives these bindings more familiar behaviour. For details see *ntextBindings*.
- When a logical line with leading whitespace is word-wrapped onto more than one display line, the wrapped display lines begin further to the left than the first display line, which can make the text layout untidy and difficult to read. *Ntext* can indent the wrapped lines to match the leading whitespace of the first display line (this facility is switched off by default). For details see *ntextIndent*.
- When the user navigates or selects text, **Tcl/Tk** sometimes needs to detect word boundaries. *Ntext* provides improved rules for word boundary detection. For details see *ntextWordBreak*.

The remainder of this page describes the basic use and configuration of all three aspects of *Ntext*. For more detailed information on the different facilities of *Ntext*, see the pages *ntextBindings*, *ntextIndent*, and *ntextWordBreak*.

See Section Example for how to apply the *Ntext* binding tag in place of the *Text* binding tag.

Commands

::ntext::new_textCopy *pathName*
Replacement for ::tk_textCopy.

::ntext::new_textCut *pathName*
Replacement for ::tk_textCut that also maintains *Ntext* indentation (see *ntextIndent*).

::ntext::new_textPaste *pathName*

Replacement for ::tk_textPaste that also maintains *Ntext* indentation (see *ntextIndent*).

::ntext::syncIndentColor *pathName*

Command to apply the current value of the variable **::ntext::indentColor** to existing lines in a text widget. This command is useful if a text widget has been created, text has been inserted in the widget, *and then* the value of **::ntext::indentColor** is changed (see *ntextIndent*).

::ntext::syncTabColor *pathName*

Command to apply the current values of the variables **::ntext::tabColor** and **::ntext::tabSelColor** to tab spaces in a text widget. This command is useful if a text widget has been created, text has been inserted in the widget, *and then* the value of **::ntext::tabColor** or **::ntext::tabSelColor** is changed.

Configuration options

Ntext provides alternatives to a number of behaviours of the classic *Text* binding tag. Where there is an option, the *Ntext* behaviour (except for display-line indentation) is switched on by default.

The behaviour of *Ntext* may be configured application-wide by setting the values of a number of namespace variables:

::ntext::classicAnchor

- **0** - (default value) selects *Ntext* behaviour, i.e. the anchor point is fixed
- **1** - selects classic *Text* behaviour, i.e. the anchor point is variable
- For more information see *ntextBindings*

::ntext::classicExtras

- **0** - (default value) selects *Ntext* behaviour, i.e. several traditional *Text* bindings are de-activated
- **1** - selects classic *Text* behaviour, i.e. all *Text* bindings are activated
- For more information see *ntextBindings*

::ntext::classicMouseSelect

- **0** - (default value) selects *Ntext* behaviour, i.e. the anchor point for mouse selection operations is moved by keyboard navigation
- **1** - selects classic *Text* behaviour
- For more information see *ntextBindings*

::ntext::classicParagraphs

- **0** - (default value) on macOS Aqua, certain keyboard bindings are made to behave in the same way as the Mac application TextEdit. The bindings involve vertical scrolling of the screen and are <?Shift-?Option-(Up|Down)>.
- **1** - on macOS Aqua, certain keyboard bindings are made to behave in the same way as classic *Text*, ignoring the conventions of Aqua. The bindings involve vertical scrolling of the screen and are <?Shift-?Option-(Up|Down)>.
- For more information see *ntextBindings*

::ntext::classicSelection

- **0** - (default value on macOS Aqua) selects Mac-like behaviour, i.e. when a navigation keystroke cancels a selection, the insert mark first moves to the end of the selection determined by the

navigation direction of the keystroke, and then the keystroke is applied.
- **1** - (default value except on macOS Aqua) selects PC-like behaviour (the same as classic *Text*), i.e. when a navigation keystroke cancels a selection, the insert mark is not moved before the keystroke is applied.
- For more information see *ntextBindings*

::ntext::classicWordBreak

- **0** - (default value) selects *Ntext* behaviour, i.e. platform-independent, two classes of word characters and one class of non-word characters.
- **1** - selects classic *Text* behaviour, i.e. platform-dependent, one class of word characters and one class of non-word characters
- After changing this value, the matching patterns should be recalculated. See *ntextWordBreak* for details and advanced configuration options.

::ntext::classicWrap

- **0** - selects *Ntext* behaviour, i.e. display lines of text widgets in *-wrap word* mode are indented to match the initial whitespace of the first display line of a logical line. If the widget already holds text when this value is set, a function call may be necessary. See *ntextIndent* for detailed instructions on the use of *Ntext* 's indentation.
- **1** - (default value) selects classic *Text* behaviour, i.e. no indentation
- For more information see *ntextIndent*

::ntext::indentColor

- **{}** - if the value is the empty string, then the indent of wrapped display lines has the same color as the widget background.
- *color* - a valid **Tk** color to use for the indent of wrapped display lines (default value *#d9d9d9*).

  Coloring is implemented with the text widget tag option *-lmargincolor*, which is available from **Tk** 8.6.6 onwards. *Ntext* indent coloring has no effect in earlier versions of **Tk**.

  The value of **::ntext::indentColor** will often be set at startup. If the value is changed when text widgets already exist and contain text, those widgets can be updated by calling command **::ntext::syncIndentColor**.
- For more information see *ntextIndent*

::ntext::overwrite

- **0** - (initial value) text typed at the keyboard is inserted into the widget
- **1** - text typed at the keyboard overwrites text already in the widget
- The value is toggled by the *Insert* key (except on macOS Aqua where there is no such key).
- For more information see *ntextBindings*

::ntext::tabColor

- **{}** - if the value is the empty string, then the tab spaces have the same color as the widget background.
- *color* - a valid **Tk** color to use for tab spaces (default value *#ffffaa*).

  The value of **::ntext::tabColor** will often be set at startup. If the value is changed when text widgets already exist and contain text, those widgets can be updated by calling command **::ntext::syncTabColor**.

::ntext::tabSelColor

- **{}** - if the value is the empty string, then the color of tab spaces in the selection defaults to the same color as unselected tabs (see **::ntext::tabColor**), and *not* to the widget *-selectbackground* color.
- *color* - a valid **Tk** color to use for tab spaces in the selection (default value *#418bd4*).

The value of **::ntext::tabSelColor** will often be set at startup. If the value is changed when text widgets already exist and contain text, those widgets can be updated by calling command **::ntext::syncTabColor**.

::ntext::useBlockCursor

- **0** - the block cursor will not be used. When the widget is in *overwrite* mode (see **::ntext::overwrite**), the ordinary cursor will change color instead.
- **1** - the block cursor will be used when the widget is in *overwrite* mode (see **::ntext::overwrite**).
- The default value depends on the version of **Tk**. In versions of **Tk** earlier than **8.5.12**, the sizing of the block cursor had a bug, and **::ntext::useBlockCursor** defaults to **0**. From **8.5.12** onwards the bug is fixed, and **::ntext::useBlockCursor** defaults to **1**.
- For more information see *ntextBindings*

Bugs

This version of **ntext** is intended to be compatible with all releases of **Tk** 8.5, 8.6, and 8.7, and with the branches *core-8-5-branch*, *core-8-6-branch*, and *trunk* in the source code repository for **Tk**. Any incompatibility with any of these versions, for any **Tk** windowing system, should be reported as a bug. Please report such in the category *ntext* of the Tklib Trackers.

Example

To create a text widget .t and use the *Ntext* bindings:

```
package require ntext
text .t
bindtags .t {.t Ntext . all}
```

See **bindtags** for more information.

ntextBindings

Keywords: ntextBindings, bindtags, text

Name

ntextBindings - Alternative Bindings for the Text Widget

Synopsis

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?

Description

The **ntext** package provides a binding tag named *Ntext* for use by text widgets in place of the default *Text* binding tag.

The *Text* binding tag provides around one hundred bindings to the text widget (the exact number

is platform-dependent). A few of these behave in a way that is different from most contemporary text-editing applications. *Ntext* aims to provide more familiar behaviour.

Features of the *Ntext* bindings that differ from the default *Text* bindings:

- Clicking near the end of a (logical) line moves the cursor to the end of that line *(not the start of the next line)*. If the widget is in *-wrap word* mode, the same rule applies to display lines.
- Double-clicking or dragging near the end of a (logical) line will highlight/select characters from the end of that line *(not the next line, or the region at the end of the line where there are no characters)*. If the widget is in *-wrap word* mode, the same rule applies to display lines.
- On windowing systems other than macOS Aqua, the *Home* key implements "Smart Home" (successive keypresses move the cursor to the start of the display line, then to the first non-whitespace character at the start of the logical line, then toggles between the beginning and end of leading whitespace); the *End* key implements "Smart End" (which is similar to "Smart Home"). On macOS Aqua, this functionality is provided by the operations and.
- When a selection exists, a "Paste" operation (e.g.) overwrites the selection (as most editors do), and does so on all platforms.
- On windowing systems other than macOS Aqua, the *Insert* key toggles between "Insert" and "Overwrite" modes for keyboard input. *(In contrast, the Text binding tag uses* Insert *as a method to paste the "primary selection", a task that can be accomplished instead by mouse middle-click.)*
- The *Escape* key clears the selection.
- On the macOS Aqua windowing system, *Ntext* follows Aqua conventions for keyboard navigation. Certain bindings related to vertical scrolling can be forced to behave in the same way as for the Windows and X11 windowing systems, by setting **::ntext::classicParagraphs** to **1** (see Section Configuration options).
- On the macOS Aqua windowing system, *Ntext* follows Aqua conventions for placement of the insert mark when keyboard navigation cancels a selection. This behavior can be switched off by setting **::ntext::classicSelection** to **1** (see Section Configuration options).
- Selecting with selects from the previous position of the insertion cursor. *(In the Text binding tag, the selection anchor may be the position of the previous mouse click.)*
- operations do not alter the selection anchor. *(In the Text binding tag, they do.)*
- By default, the *Ntext* binding tag does not provide several of the Control-key bindings supplied by the *Text* binding tag. Modern keyboards offer alternatives, such as cursor keys for navigation; modern applications often use the Control-key bindings for other purposes (e.g. for "print").

In the last three cases, the behavior of *Text* is often useful, so *Ntext* gives you the option of retaining it, by setting variables defined in the ::ntext namespace to 1 (instead of their default 0). Explaining these features in more detail:

- If the mouse is clicked at position A, then the keyboard is used to move the cursor to B, then shift is held down, and the mouse is clicked at C: the *Text* binding tag gives a selection from A to C; the *Ntext* gives a selection from B to C. If you want *Ntext* to behave like *Text* in this respect, set **::ntext::classicMouseSelect** to 1.
- The *Text* binding tag allows successive events to change both ends of the selection, by moving the selection anchor to the end of the selection furthest from the mouse click. Instead, the *Ntext* binding tag fixes the anchor, and multiple Shift-Button-1 events can only move the non-anchored end of the selection. If you want *Ntext* to behave like *Text* in this respect, set

::ntext::classicAnchor to 1. In both *Text* and *Ntext*, keyboard navigation with the Shift key held down alters the selection and keeps the selection anchor fixed.
- The following "extra" *Text* bindings are switched off by default, but can be activated in *Ntext* by setting **::ntext::classicExtras** to 1:,,,,,,,,,,,,.

Configuration options

Ntext provides alternatives to a number of behaviours of the classic *Text* binding tag. Where there is an option, the *Ntext* behaviour is switched on by default, except for display-line indentation which is discussed on a separate page at *ntextIndent*.

The behaviour of *Ntext* may be configured application-wide by setting the values of a number of namespace variables:

::ntext::classicAnchor

- **0** - (default value) selects *Ntext* behaviour, i.e. the anchor point is fixed
- **1** - selects classic *Text* behaviour, i.e. the anchor point is variable

::ntext::classicExtras

- **0** - (default value) selects *Ntext* behaviour, i.e. several "extra" *Text* bindings are de-activated
- **1** - selects classic *Text* behaviour, i.e. the "extra" *Text* bindings are activated

::ntext::classicMouseSelect

- **0** - (default value) selects *Ntext* behaviour, i.e. the anchor point for mouse selection operations is moved by keyboard navigation
- **1** - selects classic *Text* behaviour

::ntext::classicParagraphs

- **0** - (default value) on macOS Aqua, certain keyboard bindings are made to behave in the same way as the Mac application TextEdit. The bindings involve vertical scrolling of the screen and are <?Shift-?Option-(Up|Down)>.
- **1** - on macOS Aqua, certain keyboard bindings are made to behave in the same way as for the Windows and X11 windowing systems and the classic *Text*, ignoring the conventions of Aqua. The bindings involve vertical scrolling of the screen and are <?Shift-?Option-(Up|Down)>.

::ntext::classicSelection

- **0** - (default value on macOS Aqua) selects Mac-like behaviour, i.e. when a navigation keystroke cancels a selection, the insert mark first moves to the end of the selection determined by the navigation direction of the keystroke, and then the keystroke is applied.
- **1** - (default value except on macOS Aqua) selects PC-like behaviour (the same as classic *Text*), i.e. when a navigation keystroke cancels a selection, the insert mark is not moved before the keystroke is applied.

::ntext::overwrite

- **0** - (initial value) text typed at the keyboard is inserted into the widget
- **1** - text typed at the keyboard overwrites text already in the widget
- The value is toggled by the *Insert* key.

::ntext::useBlockCursor

- **0** - the block cursor will not be used. When the widget is in *overwrite* mode (see **::ntext::overwrite**), the ordinary cursor will change color instead.

- **1** - the block cursor will be used when the widget is in *overwrite* mode (see **::ntext::overwrite**).
- The default value depends on the version of **Tk**. In versions of **Tk** earlier than **8.5.12**, the sizing of the block cursor had a bug, and **::ntext::useBlockCursor** defaults to **0**. From **8.5.12** onwards the bug is fixed, and **::ntext::useBlockCursor** defaults to **1**.

Internals

In order to remain independent of the version of **Tk** (8.5 or 8.6), **ntext** defines its own virtual events. These new virtual events are used only in the *Ntext* binding tag, and therefore do not interfere with the real or virtual events used in other code.

These events include <>, <>, <> which are used in place of <>, <>, <> respectively.

The definition of the virtual event <> (etc) is similar to that of <> (etc) in **Tk** 8.6.

Bugs

This version of **ntext** is intended to be compatible with all releases of **Tk** 8.5, 8.6, and 8.7, and with the branches *core-8-5-branch*, *core-8-6-branch*, and *trunk* in the source code repository for **Tk**. Any incompatibility with any of these versions, for any **Tk** windowing system, should be reported as a bug. Please report such in the category *ntext* of the Tklib Trackers.

Example

To use *Ntext* but keep classic *Text* 's variable-anchor feature:

```
package require ntext
text .t
set ::ntext::classicAnchor 1
bindtags .t {.t Ntext . all}
```

ntextIndent

Keywords: ntextIndent, bindtags, re_syntax, regexp, text

Name

ntextIndent - ntext Indentation for the Text Widget

Synopsis

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?
::ntext::new_textCopy *pathName*
::ntext::new_textCut *pathName*
::ntext::new_textPaste *pathName*
::ntext::syncIndentColor *pathName*

Description

The **ntext** package provides a binding tag named *Ntext* for use by text widgets in place of the default *Text* binding tag.

Tk's text widget may be configured to wrap lines of text that are longer than the width of the

text area, a feature that is familiar from text editors and word processors. A complete line of text (delimited by newlines, or by the beginning or end of the document) is called a "logical line". When a logical line is wrapped onto more than one line of the display area, these fragments of the logical line are called "display lines".

If a logical line begins with whitespace, then wrapped display lines begin further to the left than the first display line, which can make the text layout untidy and difficult to read. The *Ntext* binding tag provides facilities so that a text widget in *-wrap word* mode will automatically indent display lines (other than the first) to match the initial whitespace of the first display line.

This indentation is available to text widgets only in *-wrap word* mode.

Commands

::ntext::new_textCopy *pathName*
Replacement for ::tk_textCopy.

::ntext::new_textCut *pathName*
Replacement for ::tk_textCut that also maintains *Ntext* indentation.

::ntext::new_textPaste *pathName*
Replacement for ::tk_textPaste that also maintains *Ntext* indentation.

::ntext::syncIndentColor *pathName*
Command to apply the current value of the variable **::ntext::indentColor** to existing lines in a text widget. This command is useful if a text widget has been created, text has been inserted in the widget, *and then* the value of **::ntext::indentColor** is changed.

Configuration options

The behavior of *Ntext* may be configured application-wide by setting the values of a number of namespace variables:

::ntext::classicWrap

- **0** - selects *Ntext* behaviour, i.e. display lines are indented to match the initial whitespace of the first display line of a logical line.

  No other action is required if this option, and the text widget's *-wrap* option, are set before any text is entered in the widget, and if text is entered and edited only by the mouse and keyboard. If, instead, text is manipulated by the script, or if the text widget's *-wrap* option or the value of **::ntext::classicWrap** are changed while the widget holds text, then calls to *ntext* functions are needed to alter the indentation. See the section Indenting display lines for detailed instructions.
- **1** - (default value) selects classic *Text* behaviour, i.e. no indentation.

::ntext::indentColor

- **{}** - if the value is the empty string, then the indent of wrapped display lines has the same color as the widget background.
- *color* - a valid Tk color to use for the indent of wrapped display lines (default value *#d9d9d9*).

  Coloring is implemented with the text widget tag option *-lmargincolor*, which is available from **Tk** 8.6.6 onwards. *Ntext* indent coloring has no effect in earlier versions of **Tk**.

  The value of **::ntext::indentColor** will often be set at startup. If the value is changed when text widgets already exist and contain text, those widgets can be updated by calling command

::ntext::syncIndentColor.

Advanced Use

::ntext::newWrapRegexp

- the value is a regexp pattern that determines the character of a logical line to which display lines other than the first will be aligned. The default value, **[^[:space:]]**, ensures alignment with the first non-whitespace character.

Indenting display lines

To use *Ntext* 's display line indentation:

1. Set the variable **::ntext::classicWrap** to **0** (default value is **1**). This enables bindings that will preserve indentation whenever the user modifies the widget contents using the keyboard and mouse. If the widget already holds text, call **::ntext::wrapIndent** to initialise indentation.

   Further instructions apply if the program changes the widget's contents, wrap configuration, or indent configuration.
2. The program can change the text contents, e.g. by the .text insert command. Such a change does not trigger a window binding, so the program should explicitly call function **::ntext::wrapIndent** after inserting text.
3. Auto-indentation occurs only if the widget is in *-wrap word* mode. If the program changes to or from *-wrap word* when the widget is not empty, it should call **::ntext::wrapIndent** to format the widget's text.
4. If indentation is used, and then switched off by setting **::ntext::classicWrap** to **1**, call **::ntext::wrapIndent** to remove indentation.

Functions

::ntext::wrapIndent *textWidget* ?index1? ?index2?

- Adjust the indentation of a text widget. Different cases are discussed below.

::ntext::wrapIndent *textWidget*

- Adjust the indentation of all the text in text widget *textWidget*.

::ntext::wrapIndent *textWidget index1*

- Adjust the indentation of a single logical line of a text widget - the line of *textWidget* that contains the index *index1*.

::ntext::wrapIndent *textWidget index1 index2*

- Adjust the indentation of a range of logical lines of a text widget - the lines of *textWidget* that contain indices *index1* to *index2*.

Usage

- **::ntext::wrapIndent** should be called only if the script changes the widget's contents or display properties. If the contents of the widget have been modified by the keyboard or mouse, it is not necessary for the script to call **::ntext::wrapIndent** because the appropriate calls are made automatically by the *Ntext* bindings.
- The script should normally call **::ntext::wrapIndent** if, for example, the script changes one of the following when the widget is not empty: the value of **::ntext::classicWrap**, or the widget's *-wrap* status, or the widget's tab spacing, or the font size, or the widget's contents.

- A call of the form **::ntext::wrapIndent** *textWidget* will always suffice, but if changes are needed only to certain lines, it is more efficient to specify those lines with the optional arguments ?index1?, ?index2?.
- If the widget is in *-word wrap* mode, and if **::ntext::classicWrap** is set to **0**, **::ntext::wrapIndent** will apply indentation to the logical lines within the range specified by the function's arguments.
- In other cases, i.e. if the widget is in *-word char* or *-word none* mode, or if **::ntext::classicWrap** is set to **1**, **::ntext::wrapIndent** will remove the indentation of the logical lines within the range specified by the function's arguments.

Bugs

This version of **ntext** is intended to be compatible with all releases of **Tk** 8.5, 8.6, and 8.7, and with the branches *core-8-5-branch*, *core-8-6-branch*, and *trunk* in the source code repository for **Tk**. Any incompatibility with any of these versions, for any **Tk** windowing system, should be reported as a bug. Please report such in the category *ntext* of the Tklib Trackers.

Examples

To switch on *Ntext* 's indentation and use it in widget .t, using the default indent color *#d9d9d9*:

```
package require ntext
set ::ntext::classicWrap 0
text .t -wrap word
bindtags .t {.t Ntext . all}
```

To switch on *Ntext* 's indentation and use it in widget .t, without colored indents:

```
package require ntext
set ::ntext::classicWrap 0
set ::ntext::indentColor {}
text .t -wrap word
bindtags .t {.t Ntext . all}
```

To switch on *Ntext* 's indentation and use it in widget .t, coloring the indents black:

```
package require ntext
set ::ntext::classicWrap 0
set ::ntext::indentColor black
text .t -wrap word
bindtags .t {.t Ntext . all}
```

To decide later to switch off *Ntext* 's indentation:

```
set ::ntext::classicWrap 1
::ntext::wrapIndent .t
```

To decide later to switch *Ntext* 's indentation back on:

```
set ::ntext::classicWrap 0
::ntext::wrapIndent .t 1.0 end
```

To inject some text into the widget:

```
set foo [.t index end]
.t insert end {This line was added by the script, not the keyboard!}
::ntext::wrapIndent .t $foo end
```

To change the indentation color when a widget .t already holds text:

```
set ::ntext::indentColor black
::ntext::syncIndentColor .t
```

To switch to *-wrap char* mode:

```
.t configure -wrap char
::ntext::wrapIndent .t
```

ntextWordBreak

Keywords: ntextWordBreak, bindtags, re_syntax, regexp, text

Name

ntextWordBreak - ntext Word Boundary Detection for the Text Widget

Synopsis

package require Tcl 8.5-
package require Tk 8.5-
package require ntext ?**1.0**?

Description

The **ntext** package provides a binding tag named *Ntext* for use by text widgets in place of the default *Text* binding tag.

Navigation and selection in a text widget require the detection of words and their boundaries. The word boundary detection facilities provided by Tcl/Tk through the *Text* binding tag are limited because they define only one class of "word" characters and one class of "non-word" characters. The *Ntext* binding tag uses more general rules for word boundary detection, that define *two* classes of "word" characters and one class of "non-word" characters.

Configuration options

The behaviour of *Ntext* may be configured application-wide by setting the values of a number of namespace variables. One of these is relevant to word boundary detection:

::ntext::classicWordBreak

- **0** - (default value) selects *Ntext* behaviour, i.e. platform-independent, two classes of word characters and one class of non-word characters.
- **1** - selects classic *Text* behaviour, i.e. platform-dependent, one class of word characters and one class of non-word characters
- After changing this value, *Ntext* 's regexp matching patterns should be recalculated. See Functions for details and advanced configuration options.

Advanced use

Variables (advanced use)

::ntext::tcl_match_wordBreakAfter

::ntext::tcl_match_wordBreakBefore

::ntext::tcl_match_endOfWord

::ntext::tcl_match_startOfNextWord

::ntext::tcl_match_startOfPreviousWord

These variables hold the regexp patterns that are used by *Ntext* to search for word boundaries. If they are changed, subsequent searches are immediately altered. In many situations, it it unnecessary to alter the values of these variables directly: instead call one of the functions **::ntext::initializeMatchPatterns**, **::ntext::createMatchPatterns**.

In the *Text* binding tag one can change the search rules by changing the values of the global variables **tcl_wordchars** and **tcl_nonwordchars**. The equivalent operation in the *Ntext* binding tag is to call **::ntext::createMatchPatterns** with appropriate arguments.

Functions (advanced use)

If a simple regexp search should prove insufficient, the following functions (analogous to the Tcl/Tk core's **tcl_wordBreakAfter** etc) may be replaced by the developer:

ntext::new_wordBreakAfter

ntext::new_wordBreakBefore

ntext::new_endOfWord

ntext::new_startOfNextWord

ntext::new_startOfPreviousWord

Functions

Each function calculates the five regexp search patterns that define the word boundary searches. These values are stored in the namespace variables listed above.

::ntext::initializeMatchPatterns

- This function is called when *Ntext* is first used, and needs to be called again only if the script changes the value of either **::ntext::classicWordBreak** or **::tcl_platform(platform)**. The function is called with no arguments. It is useful when the desired search patterns are the default patterns for either the *Ntext* or *Text* binding tag, and so are implicitly specified by the values of **::ntext::classicWordBreak** and **::tcl_platform(platform)** alone.

::ntext::createMatchPatterns *new_nonwordchars new_word1chars* ?new_word2chars?

- This function is useful in a wider range of situations than **::ntext::initializeMatchPatterns**. It calculates the regexp search patterns for any case with one class of "non-word" characters and one or two classes of "word" characters.

  Each argument should be a regexp expression defining a class of characters. An argument will usually be a bracket expression, but might alternatively be a class-shorthand escape, or a single character. The third argument may be omitted, or supplied as the empty string, in which case it is unused.

  The first argument is interpreted as the class of non-word characters; the second argument

(and the third, if present) are classes of word characters. The classes should include all possible characters and will normally be mutually exclusive: it is often convenient to define one class as the negation of the other two.

Word boundary matching

The problem of word boundary selection is a vexed one, because text is used to represent a universe of different types of information, and there are no simple rules that are useful for all data types or for all purposes.

Ntext attempts to improve on the facilities available in classic *Text* by providing facilities for more complex definitions of words (with three classes of characters instead of two).

What is a word? Why two classes of word?

When using the modified cursor keys and to navigate through a *Ntext* widget, the cursor is placed at the start of a word. A word is defined as a sequence of one or more characters from only one of the two defined "word" classes; it may be preceded by a character from the other "word" class or from the "non-word" class.

The double-click of mouse button 1 selects a word of text, where in this case a "word" may be as defined above, or alternatively may be a sequence of one or more characters from the "non-word" class of characters.

Traditionally Tcl has defined only one word class and one non-word class: on Windows, the non-word class is whitespace, and so alphanumerics and punctuation belong to the same class. On other platforms, punctuation is bundled with whitespace as "non-word" characters. In either case, the navigation and selection of text are unnecessarily coarse-grained, and sometimes give unhelpful results.

The use of three classes of characters might make selection too fine-grained; but in this case, holding down the *Shift* key and double-clicking another word is an excellent way to select a longer range of text (a useful binding that Tcl/Tk has long provided but which is missing in other systems).

As well as its defaults, *Ntext* permits the developer to define their own classes of characters, or to revert to the classic *Text* definitions, or to specify their own regexp matching patterns.

Bugs

This version of **ntext** is intended to be compatible with all releases of **Tk** 8.5, 8.6, and 8.7, and with the branches *core-8-5-branch*, *core-8-6-branch*, and *trunk* in the source code repository for **Tk**. Any incompatibility with any of these versions, for any **Tk** windowing system, should be reported as a bug. Please report such in the category *ntext* of the Tklib Trackers.

Example

To use *Ntext* with Tcl/Tk's usual word-boundary detection rules:

```
package require ntext
text .t
bindtags .t {.t Ntext . all}
set ::ntext::classicWordBreak 1
::ntext::initializeMatchPatterns
```

See bindtags for more information.

To define a different set of word-boundary detection rules:

```
package require ntext
text .t
bindtags .t {.t Ntext . all}
::ntext::createMatchPatterns  {[[:space:][:cntrl:]]} {[[:punct:]]}
    {[^[:punct:][:space:][:cntrl:]]}
```

See regexp, re_syntax for more information.

persistentSelection

persistentSelection

Keywords: persistentSelection

Name

persistentSelection - Enhanced PRIMARY selection

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require persistentSelection ?**1.0**?
persistentSelection::fixText
persistentSelection::getClassicPrimarySel ?*displayOf*? ?*withOthers*?
persistentSelection::getPrimarySel ?*displayOf*?
persistentSelection::report *type pathName*

Description

Consider a **Tk** widget that sets its **-exportselection** option to boolean **true**, thereby exporting its selection. When the selection in the widget is canceled, by default **Tk** also cancels the **PRIMARY** selection. In this situation, an attempt to read the **PRIMARY** selection, e.g. by a **<>** event, will fail.

The **persistentSelection** package offers more convenient behavior. It will sustain as the **PRIMARY** selection the last non-empty value set by any **Tk** widget that it monitors, as long as the **PRIMARY** selection is not claimed by another process, or by a widget that it does not monitor.

The **persistentSelection** package works by binding to the **<>** event on the widgets of interest, and using this binding to monitor any change of the selection in the widget. Its commands are confined to the namespace **::persistentSelection**.

Commands

persistentSelection::fixText

Call this command once to resolve problems with **text** widgets that use the default **Text** bindtag (rather than **Ntext**). See Text widgets for an explanation.

The command adjusts some **Text** bindings to generate **<>** events; and it re-orders calls to the **text** widget **tag** command in the private command **::tk::TextSelectTo**, so that this

command cooperates better with **persistentSelection**.

These adjustments are needed only to older versions of **Tk**. The command **persistentSelection::fixText** has no effect on sufficiently recent releases. It can safely be called for any version of **Tk**.

persistentSelection::getClassicPrimarySel ?*displayOf*? ?*withOthers*?

- *displayOf* - a **Tk** window path on the display to be interrogated (optional, default {**.**})
- *withOthers* - whether to return the **PRIMARY** selection if it is owned by another process or interpreter (optional, boolean, default **false**)

A command to return the classic (non-persistent) form of the **PRIMARY** selection. If the selection does not exist, or is sustained only by **persistentSelection**, the command returns the empty string.

persistentSelection::getPrimarySel ?*displayOf*?

- *displayOf* - a **Tk** window on the display to be interrogated (optional, default {**.**})

A command to return the value of the **PRIMARY** selection. If the selection does not exist, the command returns the empty string. The usual **Tk** commands are perfectly valid, but they raise an error if the selection does not exist.

persistentSelection::report *type pathName*

- *type* - (value **text** or **entry**) classification of the widget whose selection has changed
- *pathName* - the **Tk** window path of the widget whose selection has changed

To make package **persistentSelection** monitor a widget, this command must be bound to the **<>** event on that widget. The command records changes in the widget's selection, and uses the information to sustain a non-empty **PRIMARY** selection on each display.

Usage

Script requirements

For each widget that will be monitored by **persistentSelection**, the command **persistentSelection::report** must be bound to event **<>** in one of the widget's binding tags.

For example, the developer may wish to use **persistentSelection** with every **text** and **entryPlus** widget in an application: this can be achieved by adding the **<>** binding to the **Text** and **EntryPlus** bindtags. See Examples.

If **persistentSelection** is not required for all widgets of a particular class, the binding can be added to per-widget bindtags. See Examples.

Widget requirements

To be capable of being monitored by **persistentSelection**, a widget must satisfy three conditions:

- It must have an **-exportselection** option that allows it to export its selection to the **PRIMARY** selection.
- It must generate a **<>** event whenever its selection changes.
- its bindings to other events should be suitable - specifically, in versions of **Tk** older than **8.6.6**, a GUI event that extends or reduces the selection from one non-empty value to another should

not use an intermediate state that has an empty selection, because this confuses **persistentStore**.

The **Tk** and **Ttk** widgets that can export their selection are **text**, **entry**, **listbox**, **spinbox**, **ttk::entry**, **ttk::spinbox**, and **ttk::combobox**.

Text widgets

In versions of **Tk** older than **8.6.9**, the **text** widget does not generate **<>** events in a few "corner cases" in which its selection changes. These omissions can be corrected by changes to the **Text** bindings, in order to satisfy the second condition of Widget requirements.

In addition, versions of **Tk** older than **8.6.6** process selection events slightly differently from newer versions, and in combination with the **Text** bindings this confuses **persistentStore**. If an upgrade to the current version of **Tcl/Tk** is not feasible, this problem can be resolved by making a slight modification to the widget bindings, in order to satisfy the third condition of Widget requirements.

Either the script should call the command **persistentSelection::fixText** to adjust the **Text** bindings and the commands that they call; or the widget can use the **Ntext** binding tag (for **ntext** version 1.0 or above) instead of the default **Text** bindtag.

In either case, the argument *type* supplied to command **persistentSelection::report** should have the value **text**.

Entry, spinbox, ttk::entry, ttk::spinbox, and ttk::combobox widgets

The **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox**, and **ttk::combobox** widgets do not generate a **<>** event when their selection changes, and therefore require modification.

The package **widgetPlus** provides **snit** wrappers **widgetPlus::entryPlus**, **widgetPlus::spinboxPlus**, **widgetPlus::ttkEntryPlus**, **widgetPlus::ttkSpinboxPlus**, and **widgetPlus::ttkComboboxPlus** respectively. Each **widgetPlus** widget generates the required **<>** events.

In all these cases, the argument *type* supplied to command **persistentSelection::report** should have the value **entry**. This argument determines how **persistentSelection** will inspect the widget's selection, and the commands that do so are the same for all these widgets.

Listbox widgets

A similar wrapper has not been created for the **listbox** widget, which has the complication of permitting multiple selections.

To do

- Test on a system with multiple displays.

Bugs

This version of **persistentSelection** is intended to be compatible with all releases of **Tk** 8.5- and 8.6, and with the branches *core-8-5-branch*, *core-8-6-branch*, *revised_text*, and *trunk* in the source code repository for **Tk**. Any incompatibility with any of these versions, for any **Tk** windowing system, should be reported as a bug. Please report such in the category *persistentSelection* of the Tklib Trackers.

Examples

Each example uses **persistentSelection** to retain the last non-empty value of the selection in certain widgets. Each example also includes the creation of sample widgets.

Monitor all **entryPlus** widgets.

```
package require widgetPlus
widgetPlus::entryPlus .ep
pack .ep

package require persistentSelection
bind EntryPlus <<Selection>> {::persistentSelection::report entry %W}
```

Monitor all **text** widgets that use the default **Text** bindings.

```
text .t
pack .t

package require persistentSelection
::persistentSelection::fixText
bind Text <<Selection>> {::persistentSelection::report text %W}
```

Monitor all **text** widgets that use the default **Text** bindings, and all **entryPlus** widgets.

```
text .t
pack .t

package require widgetPlus
widgetPlus::entryPlus .ep
pack .ep

package require persistentSelection
::persistentSelection::fixText
bind Text <<Selection>> {::persistentSelection::report text %W}
bind EntryPlus <<Selection>> {::persistentSelection::report entry %W}
```

Monitor all **text** widgets that use **Ntext** bindings.

```
text .t
pack .t

package require ntext
bindtags .t {.t Ntext . all}

package require persistentSelection
bind Ntext <<Selection>> {::persistentSelection::report text %W}
```

Monitor a single **entryPlus** widget **.ep**

```
package require widgetPlus
widgetPlus::entryPlus .ep
pack .ep

package require persistentSelection
bind .ep <<Selection>> {::persistentSelection::report entry %W}
```

Monitor a single **text** widget **.t**

```
text .t
pack .t

package require persistentSelection
bind .t <<Selection>> {::persistentSelection::report text %W}
```

plotchart

plotchart

Keywords: plotchart

Name

Plotchart - Simple plotting and charting package

Synopsis

package require Tcl ?**8.5**?
package require Tk ?**8.5**?
package require Plotchart ?**2.6.2**?
::Plotchart::createXYPlot *w xaxis yaxis args*
::Plotchart::createStripchart *w xaxis yaxis args*
::Plotchart::createTXPlot *w timeaxis xaxis args*
::Plotchart::createXLogYPlot *w xaxis yaxis args*
::Plotchart::createLogXYPlot *w xaxis yaxis args*
::Plotchart::createLogXLogYPlot *w xaxis yaxis args*
::Plotchart::createPolarPlot *w radius_data args*
::Plotchart::createWindrose *w radius_data sectors*
::Plotchart::createIsometricPlot *w xaxis yaxis stepsize*
::Plotchart::createHistogram *w xaxis yaxis args*
::Plotchart::create3DPlot *w xaxis yaxis zaxis args*
::Plotchart::create3DRibbonPlot *w yaxis zaxis*
::Plotchart::createPiechart *w*
::Plotchart::createSpiralPie *w*
::Plotchart::createRadialchart *w names scale style*
::Plotchart::createBarchart *w xlabels yaxis noseries args*
::Plotchart::createHorizontalBarchart *w xaxis ylabel noseries*
::Plotchart::create3DBarchart *w yaxis nobars*
::Plotchart::create3DRibbonChart *w names yaxis zaxis*
::Plotchart::createBoxplot *w xdata ydata orientation*

::Plotchart::createTimechart *w time_begin time_end args*
::Plotchart::createGanttchart *w time_begin time_end args*
::Plotchart::createRightAxis *w_or_plot yaxis*
::Plotchart::createTableChart *w columns* ?*widths*?
::Plotchart::createTernaryDiagram *w args*
::Plotchart::createNormalPlot *w xscale args*
::Plotchart::createStatusTimeline *w xaxis ylabel args*
::Plotchart::createDendrogram *w args*
$anyplot title *text position*
$anyplot subtitle *text*
$anyplot canvas
$anyplot saveplot *filename args*
$anyplot xtext *text*
$anyplot ytext *text*
$anyplot vtext *text*
$anyplot xsubtext *text*
$anyplot ysubtext *text*
$anyplot vsubtext *text*
$anyplot xconfig -option *value ...*
$anyplot yconfig -option *value ...*
$anyplot background *part colour_or_image dir* ?*brightness*?
$anyplot xticklines *colour* ?*dash*?
$anyplot yticklines *colour* ?*dash*?
$anyplot legend *series text* ?*spacing*?
$anyplot removefromlegend *series*
$anyplot legendconfig -option *value ...*
$anyplot balloon *x y text dir*
$anyplot balloonconfig *args*
$anyplot plaintext *x y text dir*
$anyplot plaintextconfig *args*
$anyplot object *itemtype series args*
$anyplot deletedata
$xyplot plot *series xcrd ycrd*
$normalplot plot *series mean stdev data*
$normalplot diagonal
$xyplot plotlist *series xlist ylist every*
$xyplot region *series xlist ylist*
$xyplot minmax *series xcrd ymin ymax*
$histogram plotcumulative *series xcrd ycrd*
$xyplot trend *series xcrd ycrd*
$xyplot rchart *series xcrd ycrd*
$xyplot interval *series xcrd ymin ymax* ?*ycentr*?
$xyplot box-and-whiskers *series xcrd ycrd*
$xyplot violin *series xcrd ycrd*
$xyplot vector *series xcrd ycrd ucmp vcmp*
$xyplot vectorconfig *series* **-option** *value ...*

$xyplot dot *series xcrd ycrd value*
$xyplot dotconfig *series* **-option** *value ...*
$xyplot contourlines *xcrd ycrd values* ?*classes*?
$xyplot contourlinesfunctionvalues *xvec yvec valuesmat* ?*classes*?
$xyplot contourfill *xcrd ycrd values* ?*classes*?
$xyplot contourbox *xcrd ycrd values* ?*classes*?
$xyplot colorMap *colours*
$xyplot legendisolines *values classes*
$xyplot legendshades *values classes*
$xyplot grid *xcrd ycrd*
$xyplot xband *ymin ymax*
$xyplot yband *xmin xmax*
$xyplot labeldot *x y text orient*
$polarplot plot *series radius angle*
$windrose plot *data colour*
$plot3d plotfunc *function*
$plot3d plotfuncont *function contours*
$plot3d gridsize *nxcells nycells*
$plot3d plotdata *data*
$plot3d interpolatedata *data contours*
$plot3d colour *fill border*
$plot3d ribbon *yzpairs*
$plot3d plot *yzpairs*
$xyplot dataconfig *series* **-option** *value ...*
$pie plot *data*
$pie colours *colour1 colour2 ...*
$pie explode *segment*
$radial plot *data colour thickness*
$pie colours *colour1 colour2 ...*
$barchart plot *series ydata colour* ?*dir*? ?*brightness*?
$barchart config -option *value ...*
$barchart plot *series xdata colour* ?*dir*? ?*brightness*?
$barchart config -option *value ...*
$barchart plot *label yvalue colour*
$barchart config -option *value ...*
$ribbon line *xypairs colour*
$ribbon area *xypairs colour*
$boxplot plot *series label values*
$boxplot violin *series label values*
$timechart period *text time_begin time_end colour*
$timechart addperiod *time_begin time_end colour*
$timechart milestone *text time colour*
$timechart addmilestone *time colour*
$timechart vertline *text time colour*
$timechart hscroll *scrollbar*
$timechart vscroll *scrollbar*

$ganttchart task *text time_begin time_end completed*
$ganttchart milestone *text time colour*
$ganttchart vertline *text time*
$ganttchart connect *from to*
$ganttchart summary *text args*
$ganttchart color *keyword newcolor*
$ganttchart font *keyword newfont*
$ganttchart hscroll *scrollbar*
$ganttchart vscroll *scrollbar*
$isoplot plot rectangle *x1 y1 x2 y2 colour*
$isoplot plot filled-rectangle *x1 y1 x2 y2 colour*
$isoplot plot circle *xc yc radius colour*
$isoplot plot filled-circle *xc yc radius colour*
$table row *items*
$table separator
$table formatcommand *procname*
$table cellconfigure *args*
$ternary plot *series xcrd ycrd zcrd text dir*
$ternary line *series coords*
$ternary fill *series coords*
$ternary text *xtext ytext ztext*
$ternary ticklines *colour*
$timeline plot *series item start stop color*
$timeline vertline *text time args*
$dendrogram plot *series data*
::Plotchart::viewPort *w pxmin pymin pxmax pymax*
::Plotchart::worldCoordinates *w xmin ymin xmax ymax*
::Plotchart::world3DCoordinates *w xmin ymin zmin xmax ymax zmax*
::Plotchart::coordsToPixel *w x y*
::Plotchart::coords3DToPixel *w x y z*
::Plotchart::polarCoordinates *w radmax*
::Plotchart::polarToPixel *w rad phi*
::Plotchart::pixelToCoords *w x y*
::Plotchart::pixelToIndex *w x y*
::Plotchart::determineScale *xmin xmax inverted*
::Plotchart::determineScaleFromList *values inverted*
::Plotchart::plotconfig *charttype component property value*
::Plotchart::plotstyle *subcmd style args*
::Plotchart::eraseplot *anyplot*
::Plotchart::createTargetDiagram *w limits scale*
$target plot *series xvalues yvalues*
::Plotchart::createPerformanceProfile *w max*
$performance plot *series_and_data_pairs*
::Plotchart::createTaylorDiagram *w radius_data args*
$taylor plot *series stdev corr*
::Plotchart::createHeatmap *w rowlabels columnlabels args*

$heatmap plot row *label data*
$heatmap plot column *label data*
$heatmap plot cell *rowlabel columnlabel value*
$heatmap scale values *min max*
$heatmap scale colours *mincolour maxcolour*
::Plotchart::createCircleplot *w labels args*
$circleplot connect *label1 label2 colour width*
$circleplot modify *label args*
::Plotchart::plotmethod *charttype methodname plotproc*
::Plotchart::plotpack *w dir args*
$anyplot bindplot *event command args*
$anyplot bindlast *series event command*

Description

Plotchart is a Tcl-only package that focuses on the easy creation of xy-plots, barcharts and other common types of graphical presentations. The emphasis is on ease of use, rather than flexibility. The procedures that create a plot use the entire canvas window, making the layout of the plot completely automatic.

This results in the creation of an xy-plot in, say, ten lines of code:

```
    package require Plotchart

    canvas .c -background white -width 400 -height 200
    pack   .c -fill both

    #
    # Create the plot with its x- and y-axes
    #
    set s [::Plotchart::createXYPlot .c {0.0 100.0 10.0} {0.0 100.0
    20.0}]

    foreach {x y} {0.0 32.0 10.0 50.0 25.0 60.0 78.0 11.0 } {
        $s plot series1 $x $y
    }

    $s title "Data series"
```

A drawback of the package might be that it does not do any data management. So if the canvas that holds the plot is to be resized, the whole plot must be redrawn. The advantage, though, is that it offers a number of plot and chart types:

- XY-plots like the one shown above with any number of data series.
- Stripcharts, a kind of XY-plots where the horizontal axis is adjusted automatically. The result is a kind of sliding window on the data series.
- Polar plots, where the coordinates are polar instead of cartesian.
- Histograms, for plotting statistical information.
- Barcharts, piecharts, Gantt charts, time charts.
- Isometric plots, where the scale of the coordinates in the two directions is always the same, i.e.

a circle in world coordinates appears as a circle on the screen.

You can zoom in and out, as well as pan with these plots (*Note:* this works best if no axes are drawn, the zooming and panning routines do not distinguish the axes), using the mouse buttons with the control key and the arrow keys with the control key.

- Piecharts, with automatic scaling to indicate the proportions.
- Barcharts, with either vertical or horizontal bars, stacked bars or bars side by side.
- Timecharts, where bars indicate a time period and milestones or other important moments in time are represented by triangles.
- 3D plots (both for displaying surfaces and 3D bars)

With version 1.5 a new command has been introduced: plotconfig, which can be used to configure the plot options for particular types of plots and charts (cf. Configuration options and other commands) With version 1.8.3 several new features were introduced, which allow more interactivity (cf. Interactive use) With version 2.0 it has become possible to put several plots or charts in one canvas.

Plot creation commands

You create the plot or chart with one single command and then fill the plot with data:

::Plotchart::createXYPlot *w xaxis yaxis args*

Create a new xy-plot (configuration type: xyplot).

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order. For an inverted axis, where the maximum appears on the left-hand side, use: maximum, minimum and a *negative* stepsize.

list *yaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order. For an inverted axis, where the maximum appears at the bottom, use: maximum, minimum and a *negative* stepsize.

list *args* (in)

Zero or more options that influence the appearance of the plot:

- *-xlabels {labels}:* Custom labels for the x-axis. If the labels are numeric, they are positioned according to the given scale, otherwise they are positioned with equal distance, based on the number of labels. Note: this only works if the stepsize of the xaxis argument is the empty string.
- *-ylabels {labels}:* Similarly, custom labels for the y-axis.
- *-box {measures}:* See Arranging multiple plots in a canvas
- *-axesbox {measures}:* See Arranging multiple plots in a canvas
- *-timeformat format:* See Using date/time labels
- *-gmt 0/1:* See Using date/time labels
- *-axestozero:* Draw the axes at the origin (1), or at the sides of the plot area (0, default). This option is implemented for XY plots only.
- *-isometric:* Rescale the axes, so that a square in the coordinates appears as a square on the screen (1), or use the given extremes (0, default). This option is implemented for XY plots only.

::Plotchart::createStripchart *w xaxis yaxis args*

Create a new strip chart (configuration type: stripchart). The only difference to a regular XY plot is that the x-axis will be automatically adjusted when the x-coordinate of a new point exceeds the maximum.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order. Note that an inverted x-axis is *not* supported for this type of plot.

list *yaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order. For an inverted axis, where the maximum appears at the bottom, use: maximum, minimum and a *negative* stepsize.

list *args* (in)

Zero or more options - see the XY-plot for more information.

::Plotchart::createTXPlot *w timeaxis xaxis args*

Create a new time-x-plot (configuration type: txplot). The horizontal axis represents the date/time of the data and the vertical axis the values themselves.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *timeaxis* (in)

A 3-element list containing the minimum and maximum date/time to be shown and the stepsize (*in days*) for the time-axis, in this order. Note that an inverted time-axis is *not* supported.

list *xaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the vertical axis, in this order. For an inverted axis, where the maximum appears at the bottom, use: maximum, minimum and a *negative* stepsize.

list *args* (in)

Zero or more options - see the XY-plot for more information.

::Plotchart::createXLogYPlot *w xaxis yaxis args*

Create a new xy-plot where the y-axis has a logarithmic scale (configuration type: xlogyplot).

The data should be given as for a linear scale, as the logarithmic transformation is taken of internally.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order. For an inverted axis, where the maximum appears on the left-hand side, use: maximum, minimum and a *negative* stepsize.

list *yaxis* (in)

A 2-element list containing minimum and maximum for the y-axis, in this order. Note that an inverted logarithmic axis is *not* supported.

list *args* (in)
Zero or more options - see the XY-plot for more information.

::Plotchart::createLogXYPlot *w xaxis yaxis args*
Create a new xy-plot where the x-axis has a logarithmic scale (configuration type: logxyplot).

The data should be given as for a linear scale, as the logarithmic transformation is taken of internally.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)
A 2-element list containing minimum and maximum for the x-axis, in this order. Note that an inverted logarithmic axis is *not* supported.

list *yaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order. For an inverted axis, where the maximum appears on the left-hand side, use: maximum, minimum and a *negative* stepsize.

list *args* (in)
Zero or more options - see the XY-plot for more information.

::Plotchart::createLogXLogYPlot *w xaxis yaxis args*
Create a new xy-plot where both the x-axis and the y-axis have a logarithmic scale (configuration type: logxlogyplot).

The data should be given as for a linear scale, as the logarithmic transformation is taken of internally.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)
A 2-element list containing minimum and maximum for the x-axis, in this order. Note that an inverted logarithmic axis is *not* supported.

list *yaxis* (in)
A 2-element list containing minimum and maximum for the y-axis, in this order. Note that an inverted logarithmic axis is *not* supported.

list *args* (in)
Zero or more options - see the XY-plot for more information.

::Plotchart::createPolarPlot *w radius_data args*
Create a new polar plot (configuration type: polarplot).

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *radius_data* (in)
A 2-element list containing maximum radius and stepsize for the radial axis, in this order.

list *args* (in)
Zero or more options - see the XY-plot for more information.

::Plotchart::createWindrose *w radius_data sectors*

Create a new windrose diagram. The diagram will consist of concentric circles as defined by the *radius_data* argument and a number of sectors (given by the *sectors* argument). The sectors are drawn in the "nautical" convention, that is: the first is located at the positive y-axis, the second is to the right and so on in a clockwise direction.

widget *w* (in)
Name of the *existing* canvas widget to hold the diagram

list *radius_data* (in)
A 2-element list, the first element is the maximum radius, the second is the step to be used for the circles.

int *sectors*
Number of sectors to use (defaults to 16).

::Plotchart::createIsometricPlot *w xaxis yaxis stepsize*
Create a new isometric plot, where the vertical and the horizontal coordinates are scaled so that a circle will truly appear as a circle (configuration type: isometric).

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)
A 2-element list containing minimum, and maximum for the x-axis, in this order.

list *yaxis* (in)
A 2-element list containing minimum, and maximum for the y-axis, in this order.

float|noaxes *stepsize* (in)
Either the stepsize used by both axes or the keyword **noaxes** to signal the plot that it should use the full area of the widget, to not draw any of the axes.

::Plotchart::createHistogram *w xaxis yaxis args*
Create a new histogram (configuration type: histogram).

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order.

list *yaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order.

list *args* (in)
Zero or more options - see the XY-plot for more information.

::Plotchart::create3DPlot *w xaxis yaxis zaxis args*
Create a new 3D plot.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order.

list *yaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order.

list *zaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the z-axis, in this order.

list *args* (in)

Optional list of options (keyword-value pairs). Currently supported: -xlabels to set the labels for the x-axis.

::Plotchart::create3DRibbonPlot *w yaxis zaxis*

Create a new 3D ribbon plot. It is a simplification of the full 3D plot and allows for the drawing of a ribbon only (the x-axis is dropped).

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *yaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order.

list *zaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the z-axis, in this order.

::Plotchart::createPiechart *w*

Create a new piechart (configuration type: piechart).

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

::Plotchart::createSpiralPie *w*

Create a new "spiral pie" (configuration type: spiralpie), a variation on the ordinary piechart. The value is used to scale the radius, rather than the angle. By default the data are sorted.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

::Plotchart::createRadialchart *w names scale style*

Create a new radial chart (the data are drawn as a line connecting the spokes of the diagram) (configuration type: radialchart).

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *names* (in)

Names for the spokes.

float *scale* (in)

Scale value to determine the position of the data along the spokes.

string *style* (in)

Style of the chart (optional). One of:
- *lines* - the default: draw the data as independent polylines.
- *cumulative* - draw the data as polylines where the data are accumulated.
- *filled* - draw the data as filled polygons where the data are accumulated

::Plotchart::createBarchart *w xlabels yaxis noseries args*

Create a new barchart with vertical bars (configuration type: vertbars). The horizontal axis will display the labels contained in the argument *xlabels*. The number of series given by *noseries* determines both the width of the bars, and the way the series will be drawn.

If the keyword **stacked** was specified the series will be drawn stacked on top of each other.

Otherwise each series that is drawn will be drawn shifted to the right.

The number of series determines the width of the bars, so that there is space of that number of bars. If you use a floating-point number, like 2.2, instead of an integer, like 2, a small gap between the sets of bars will be drawn - the width depends on the fractional part.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xlabels* (in)
List of labels for the x-axis. Its length also determines the number of bars that will be plotted per series.

list *yaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order.

int|stacked *noseries* (in)
The number of data series that will be plotted. This has to be an integer number greater than zero (if **stacked** is not used).

string *args* (in)
One or more key-value pairs defining the options. Specific to barcharts: **-xlabelangle** value, the angle under which the labels should be drawn. This defaults to 0 - horizontal text. This option is especially useful if you have fairly long labels.

::Plotchart::createHorizontalBarchart *w xaxis ylabel noseries*

Create a new barchart with horizontal bars (configuration type: horizbars). The vertical axis will display the labels contained in the argument *ylabels*. The number of series given by *noseries* determines both the width of the bars, and the way the series will be drawn.

If the keyword **stacked** was specified the series will be drawn stacked from left to right. Otherwise each series that is drawn will be drawn shifted upward.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *xaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order.

list *ylabels* (in)
List of labels for the y-axis. Its length also determines the number of bars that will be plotted per series.

int|stacked *noseries* (in)
The number of data series that will be plotted. This has to be an integer number greater than zero (if **stacked** is not used).

::Plotchart::create3DBarchart *w yaxis nobars*

Create a new barchart with 3D vertical bars (configuration type: 3dbars). The horizontal axis will display the labels per bar. The number of bars given by *nobars* determines the position and the width of the bars. The colours can be varied per bar. (This type of chart was inspired by the Wiki page on 3D bars by Richard Suchenwirth.)

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

list *yaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order.

int *nobars* (in)
The number of bars that will be plotted.

::Plotchart::create3DRibbonChart *w names yaxis zaxis*
Create a new "ribbon chart" (configuration type: 3dribbon). This is a chart where the data series are represented as ribbons in a three-dimensional axis system. Along the x-axis (which is "into" the screen) the names are plotted, each representing a single series. The first plot command draws the furthest series, the second draws the series in front of that and so on.
widget *w* (in)
Name of the *existing* canvas widget to hold the plot.
widget *w* (in)
Names of the series, plotted as labels along the x-axis
list *yaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the y-axis (drawn horizontally!), in this order.
list *zaxis* (in)
A 3-element list containing minimum, maximum and stepsize for the z-axis (drawn vertically), in this order.
int *nobars* (in)
The number of bars that will be plotted.

::Plotchart::createBoxplot *w xdata ydata orientation*
Create a new boxplot with horizontal or vertical boxes (box-and-whiskers) (configuration type: boxplot). Depending on the orientation the x- or y-axis is drawn with labels. The boxes are drawn based on the raw data (see the plot subcommand for this type of plot).
widget *w* (in)
Name of the *existing* canvas widget to hold the plot.
list *xdata* (in)
This is either a 3-element list containing minimum, maximum and stepsize for the x-axis, in this order (when orientation is horizontal), or a list of labels for the x-axis (when orientation is vertical). The length of the label list also determines the number of boxes that can be plotted. The labels are also used in the plot subcommand.
list *ydata* (in)
This is either a 3-element list containing minimum, maximum and stepsize for the y-axis, in this order (when orientation is vertical), or a list of labels for the y-axis (when orientation is horizontal). The length of the label list also determines the number of boxes that can be plotted. The labels are also used in the plot subcommand.
string *orientation* (in)
If given, "horizontal" or "vertical" determines the orientation of the boxes. This optional value (default: horizontal) also determines the interpretation of the xdata and ydata arguments.

::Plotchart::createTimechart *w time_begin time_end args*
Create a new timechart (configuration type: timechart). The time axis (= x-axis) goes from *time_begin* to *time_end*, and the vertical spacing is determined by the number of items to plot.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

string *time_begin* (in)
The start time given in a form that is recognised by the **clock scan** command (e.g. "1 january 2004").

string *time_end* (in)
The end time given in a form that is recognised by the **clock scan** command (e.g. "1 january 2004").

arguments *args* (in)
The remaining arguments can be:
- The expected/maximum number of items. This determines the vertical spacing. (If given, it must be the first argument after "time_end"
- The keyword -barheight and the number of pixels per bar. This is an alternative method to determine the vertical spacing.
- The keyword -ylabelwidth and the number of pixels to reserve for the labels at the y-axis.

::Plotchart::createGanttchart *w time_begin time_end args*
Create a new Gantt chart (configuration type: ganttchart). The time axis (= x-axis) goes from *time_begin* to *time_end*, and the vertical spacing is determined by the number of items to plot. Via the specific commands you can then add tasks and connections between the tasks.

widget *w* (in)
Name of the *existing* canvas widget to hold the plot.

string *time_begin* (in)
The start time given in a form that is recognised by the **clock scan** command (e.g. "1 january 2004").

string *time_end* (in)
The end time given in a form that is recognised by the **clock scan** command (e.g. "1 january 2004").

arguments *args* (in)
The remaining arguments can be:
- The expected/maximum number of items. This determines the vertical spacing. (If given this way, it must be the first argument after "time_end")
- The expected/maximum width of the descriptive text (roughly in characters, for the actual space reserved for the text, it is assumed that a character is about ten pixels wide). Defaults to 20. (If given this way, it must be the second argument after "time_end").
- The keyword -barheight and the number of pixels per bar. This is an alternative method to determine the vertical spacing.
- The keyword -ylabelwidth and the number of pixels to reserve for the labels at the y-axis.

::Plotchart::createRightAxis *w_or_plot yaxis*
Create a plot command that will use a right axis instead of the left axis (configuration type: inherited from the existing plot). The canvas widget must already contain an ordinary plot, as the horizontal axis and other properties are reused. Preferably use the plot command, as with multiple plots in a canvas (also when redefining an existing plot!), the wrong geometry might be used.

To plot data using the right axis, use this new command, to plot data using the *left* axis, use the original plot command.

widget *w_or_plot* (in)

Name of the *existing* canvas widget to hold the plot or preferably the plot command for the plot with the left axis.

list *yaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the y-axis, in this order.

::Plotchart::createTableChart *w columns* ?widths?

Create a command to draw a table. You can use a variety of commands to draw the actual rows of the table, but the number of columns is fixed. (See Table charts for an example)

widget *w* (in)

Name of the canvas widget to hold the table.

list *columns* (in)

The headers of the columns in the table. The number of elements determines the number of columns.

list *widths* (in)

If given, either a single value, the width in pixels for all columns or for each column the width of that column. If not given, the table is spread out over the width of the canvas (minus the margins).

::Plotchart::createTernaryDiagram *w args*

Create a command to draw a ternary diagram (configuration type: ternary). You can draw individual (labelled) data points in the diagram, lines and filled polygons.

widget *w* (in)

Name of the canvas widget to hold the ternary diagram.

list *args* (in)

Zero or more option-value pairs to influence the position and the appearance of the diagram. In addition to the positioning options (-box and -axesbox) the diagram supports: *-fractions yes/no*, to display numbers 0 to 1 instead of 0 to 100% and *-steps number* to influence the number of labels along the three sides.

::Plotchart::createNormalPlot *w xscale args*

Create a command to draw a normal plot - useful to investigate whether a data set is normally distributed or not. In that case the data will fall on or near the diagonal. As such, it is a specialised plotting procedure.

The details of the plotting procedure have been adopted from the *qqnorm* in the "R" *stats* package and described on Wikipedia.

As the implementation of this plot type relies on the *math::statistics* package, it is only available if that package can be loaded.

widget *w* (in)

Name of the canvas widget to hold the normal plot.

list *xscale* (in)

A 3-element list containing minimum, maximum and stepsize for the x-axis in this order. The scaling of the y-axis is determined from that. *Important:* the scale is to be given in

terms of the normalised data, that is: 0 represents the mean of the data, 1 one standard deviation away from the mean etc.

list *args* (in)

Zero or more option-value pairs to influence the position and the appearance of the plot - see the XY-plot for more details.

::Plotchart::createStatusTimeline *w xaxis ylabel args*

Create a command to draw a so-called status timeline. Its layout is similar to a horizontal barchart, but the bars are drawn in different colours, each representing the status of the item as it varies over time (the horizontal axis).

widget *w* (in)

Name of the canvas widget to hold the ternary diagram.

list *xaxis* (in)

A 3-element list containing minimum, maximum and stepsize for the x-axis, in this order.

list *ylabels* (in)

List of labels for the y-axis. Its length also determines the number of bars that will be plotted per series.

list *args* (in)

Zero or more option-value pairs to influence the position and the appearance of the diagram. In addition to the positioning options (-box and -axesbox) the diagram supports: *-xaxis yes/no*, to turn displaying the numeric labels on or off.

::Plotchart::createDendrogram *w args*

Create a dendrogram to plot hierarchically organised data. The labels that identify the endpoints of the hierarchy can all be drawn on the same level or where their branch ends. This is controlled by the option "-extend".

The data are organised as nested lists, like:

```
    \$dendrogram plot data { {"Label1"  "Label2"} {F {A B} {C {D
   E}}} }
```

Sublists with more than one element are considered a branching and are drawn in that way. Sublists with only one element are considered a leaf and are drawn as text. Because a string with two or more words can not be distinguished from a list with two or more elements, a provision is made for this situation: if the first element is "LEAF", then the list is considered to consist of leafs only and all elements (even if they are actually nested lists themselves are drawn as text:

```
    \$dendrogram plot data { {LEAF "Label 1"  "Label 2"} {F {A B}
   {C {D E}}} }
```

widget *w* (in)

Name of the canvas widget to hold the dendrogram.

list *args* (in)

Zero or more option-value pairs:

- The option "-direction" takes one of the following values: left-right, right-left, top-bottom, bottom-top, with "top-bottom" as the default. The tree will be drawn according to this

option, the labels appear either horizontally or vertically at the end of the tree.
- The option "-extend" with value 1 (default) or 0 (no extension) to indicate that the labels should all appear at the same level.

Plot methods

Each of the creation commands explained in the last section returns the name of a new object command that can be used to manipulate the plot or chart. The subcommands available to a chart command depend on the type of the chart.

General subcommands for all types of charts. \$anyplot is the command returned by the creation command:

$anyplot title *text position*

Specify the title of the whole chart.

string *text* (in)

The text of the title to be drawn.

string *position* (in)

The position of the title. The default position is "center", but you can alternatively use "left" or "right". You can use multiple titles with different positions.

$anyplot subtitle *text*

Specify the subtitle of the whole chart.

string *text* (in)

The text of the subtitle to be drawn.

$anyplot canvas

Return the name of the canvas (or the alias if you use more than one plot within a canvas). Use this value for the coordinate transformations.

$anyplot saveplot *filename args*

Draws the plot into a file, using PostScript.

string *filename* (in)

Contain the path name of the file to write the plot to.

list *args* (in)

If the standard PostScript output is used, the option -plotregion can be specifed to save the whole plot (value: bbox) regardless of what is visible in the window. The default (value: window) is to only plot the visible part of the plot.

Optionally you can specify the option -format "some picture format" to store the plot in a different file than a PostScript file. This, however, relies on the Img package to do the actual job.

Note: Because the window holding the plot must be fully visible before Img can successfully grab it, it is raised first. On some systems, for instance Linux with KDE, raising a window is not done automatically, but instead you need to click on the window in the task bar. Similar things happen on Windows XP.

There seems to be something wrong under some circumstances, so instead of waiting for

the visibility of the window, the procedure simply waits two seconds. It is not ideal, but it seems to work better.

$anyplot xtext *text*
Specify the title of the (horizontal) x-axis, for those plots that have a straight x-axis.
string *text* (in)
The text of the x-axis label to be drawn.

$anyplot ytext *text*
Specify the title of the (horizontal) y-axis, for those plots that have a straight y-axis.
string *text* (in)
The text of the y-axis label to be drawn.

$anyplot vtext *text*
Draw a *vertical* label to the y-axis. Note: this requires Tk 8.6 or later, for older versions it does nothing.
string *text* (in)
Text to drawn to the y-axis

$anyplot xsubtext *text*
Specify the subtext of the (horizontal) x-axis, for those plots that have a straight x-axis. This text is drawn below the primary text.

Since this involves positioning the primary text and setting margins, you need to set the option "usesubtext" for the bottom axis via the plotstyle command. The relevant options are: usesubtext, subtextcolor and subtextfont.
string *text* (in)
The secondary text of the x-axis label to be drawn.

$anyplot ysubtext *text*
Specify the subtext of the (vertical) y-axis, for those plots that have a straight y-axis. This text is drawn below the primary text, for both axes on the left and the right.

Since this involves positioning the primary text and setting margins, you need to set the option "usesubtext" for the left or right axis via the plotstyle command. The relevant options are: usesubtext, subtextcolor and subtextfont.
string *text* (in)
The secondary text of the y-axis label to be drawn.

$anyplot vsubtext *text*
Specify the subtext of the (vertical) y-axis, for those plots that have a straight y-axis. This text is drawn to the *right* of the primary text, for both axes on the left and the right.

Since this involves positioning the primary text and setting margins, you need to set the option "usesubtext" for the left or right axis via the plotstyle command. The relevant options are: usevsubtext, vsubtextcolor and vsubtextfont. (Note the "v" to distinguish this option from the text at the top of a vertical axis that is drawn via *$anyplot ytext* or *$anyplot ysubtext*.)
string *text* (in)
The secondary (vertical) text of the y-axis label to be drawn.

$anyplot xconfig -option *value* ...

Set one or more configuration parameters for the x-axis. The following options are supported:

format fmt

The format for the numbers along the axis.

ticklength length

The length of the tickmarks (in pixels).

ticklines boolean

Whether to draw ticklines (**true**) or not (**false**).

scale scale_data

New scale data for the axis, i.e. a 3-element list containing minimum, maximum and stepsize for the axis, in this order.

Beware: Setting this option will clear all data from the plot.

$anyplot yconfig -option *value* ...

Set one or more configuration parameters for the y-axis. This method accepts the same options and values as the method **xconfig**.

$anyplot background *part colour_or_image dir* ?brightness?

Set the background of a part of the plot

string *part*

Which part of the plot: "axes" for the axes area and "plot" for the inner part. The interpretation depends on the type of plot. Two further possibilities are:
- *image*, in which case a predefined image is loaded into the background of the plot.
- *gradient*, in which case the background is coloured in different shades of the given colour. The "dir" argument specifies the direction in which the colour gets whiter.

string *colour_or_image*

Colour for that part or the name of the image if "part" is "image"

string *dir*

The direction of the gradient. One of: top-down, bottom-up, left-right or right-left.

string *brightness*

Indicates whether the colour should become brighter (bright) or darker (dark). Defaults to bright

$anyplot xticklines *colour* ?dash?

Draw vertical ticklines at each tick location

string *colour*

Colour of the lines. Specifying an empty colour ("") removes them again. Defaults to "black"

string *dash*

Optional argument to specify the dash pattern for the lines. Defaults to "lines" Possible values: lines, dots1, dots2, dots3, dots4, dots5. The actual effect depends on the platform.

$anyplot yticklines *colour* ?dash?

Draw horizontal ticklines at each tick location

string *colour*

Colour of the lines. Specifying an empty colour ("") removes them again Defaults to "black"

string *dash*

Optional argument to specify the dash pattern for the lines. Defaults to "lines" Possible values: lines, dots1, dots2, dots3, dots4, dots5. The actual effect depends on the platform.

$anyplot legend *series text* ?spacing?

Add an entry to the legend. The series determines which graphical symbol is to be used. (As a side effect the legend is actually drawn.)

string *series*

Name of the data series. This determines the colour of the line and the symbol (if any) that will be drawn.

string *text*

Text to be drawn next to the line/symbol.

integer *spacing*

Optional argument to specify the vertical spacing between the entries (in pixels). (Note that this spacing will be reused later.)

$anyplot removefromlegend *series*

Remove an entry for a series from the legend and redraw it.

string *series*

Name of the data series to be removed.

$anyplot legendconfig -option *value* ...

Set one or more options for the legend. The legend is drawn as a rectangle with text and graphics inside.

background colour

Set the colour of the background (the default colour is white). Set to the empty string for a transparant legend.

border colour

Set the colour of the border (the default colour is black). Set to the empty string if you do not want a border.

canvas c

Draw the legend in a different canvas widget. This gives you the freedom to position the legend outside the actual plot.

font font

Set the font used to draw the text next to the symbol.

legendtype type

Override the type of the legend, that is pre-defined for the current type of plot. May be one of: rectangle or line.

position corner

Set the position of the legend. May be one of: top-left, top-right, bottom-left or bottom-right. (Default value is top-right.)

spacing pixels

Set the vertical spacing between entries in the legend (in pixels). bottom-left or bottom-right. (Default value is 10 pixels.)

order string
Determine the ordering of the legend entries - "normal" means the order in which the entries are defined, "reverse" exactly the opposite. (Default: normal)

$anyplot balloon *x y text dir*
Add balloon text to the plot (except for 3D plots). The arrow will point to the given x- and y-coordinates. For xy-graphs and such, the coordinates are directly related to the axes; for vertical barcharts the x-coordinate is measured as the number of bars minus 1 and similar for horizontal barcharts.

float *x*
X-coordinate of the point that the arrow of the balloon will point to.

float *y*
Y-coordinate of the point that the arrow of the balloon will point to.

string *text*
Text to be drawn in the balloon.

string *dir*
Direction of the arrow, one of: north, north-east, east, south-east, south, south-west, west or north-west.

$anyplot balloonconfig *args*
Configure the balloon text for the plot. The new settings will be used for the next balloon text.

font fontname
Font to be used for the text

justify left|center|right
Way to justify multiline text

textcolour colour
Colour for the text (synonym: textcolor)

background colour
Background colour for the balloon

outline colour
Colour of the outline of the balloon

margin value
Margin around the text (in pixels)

rimwidth value
Width of the outline of the balloon (in pixels)

arrowsize value
Length factor for the arrow (in pixels)

$anyplot plaintext *x y text dir*
Add plain text to the plot (except for 3D plots). The text is positioned at the given x- and y-coordinates. For xy-graphs and such, the coordinates are directly related to the axes; for vertical barcharts the x-coordinate is measured as the number of bars minus 1 and similar for horizontal barcharts.

float *x*
X-coordinate of the text position

float *y*
Y-coordinate of the text position

string *text*
Text to be drawn.

string *dir*
Anchor for the text, one of: north, north-east, east, south-east, south, south-west, west or north-west.

$anyplot plaintextconfig *args*
Configure the plain text annotation for the plot. The new settings will be used for the next plain text.

font fontname
Font to be used for the text

justify left|center|right
Way to justify multiline text

textcolour colour
Colour for the text (synonym: textcolor)

$anyplot object *itemtype series args*
Draw a canvas item in the plot where the coordinates are scaled using the coordinate system of the plot. In addition to the standard canvas types, it also supports circles, dots and crosses.

Note: Currently implemented for xy-plots, (vertical and horizontal) barcharts, and piecharts.

Note: To add an entry in the legend for the object, you can use the *dataconfig* subcommand with a type "rectangle". This will cause a rectangle to be shown.

string *itemtype* (in)
Name of a standard canvas item or "circle", "dot" or "cross"

string *series* (in)
The data series it belongs to, used for setting the default drawing options

list *args* (in)
List of coordinates and drawing options

$anyplot deletedata
Remove the lines, symbols and other graphical object associated with the actual data from the plot.

Note: Currently implemented for xy-plots only

Note: The existing options for data series and the legend entry are kept as they were.

Note: Currently there are side effects if the canvas contains more than one plot.

Note: The commands **xconfig** and **yconfig** are currently implemented only for XY-plots and only the option **-format** has any effect.

For *xy plots*, *stripcharts*, *histograms* and *time-x-plots*:

$xyplot plot *series xcrd ycrd*

Add a data point to the plot.

string *series* (in)
Name of the data series the new point belongs to.

float *xcrd* (in)
X-coordinate of the new point. (For time-x plots this must be valid date/time that can be read with the *clock scan* command).

float *ycrd* (in)
Y-coordinate of the new point.

For *normal plots*:

$normalplot plot *series mean stdev data*
Plot the data set using the given mean and stanard deviation.

As you give the mean and standard deviation separately, the plot can be used for several data series or for adding to an existing data series.

string *series* (in)
Name of the data series - used to determine the appearance

float *mean* (in)
Assumed mean of the data set.

float *stdev* (in)
Assumed standard deviation of the data set.

list *data* (in)
List of the data comprising the data set

$normalplot diagonal
Draw a diagonal line, indicating the ideal normally distributed data set.

For *xy, x-log y, log-x-y, log-x-log-y plots* there is the additional command *plotlist*, which is useful for plotting a large amount of data:

$xyplot plotlist *series xlist ylist every*
Draw a series of data as a whole. If symbols are asked for, draw them only for every Nth data point.

string *series* (in)
Name of the data series the new line belongs to.

float *xlist* (in)
List of X-coordinates for the data series.

float *ycrd* (in)
List of Y-coordinates for the data series.

int *every* (in)
Optional argument stating how often a symbol (if any) should be drawn. If left out, use a simple heuristic: N = sqrt(number of data points).

Other commands for *xy, x-log y, log-x-y, log-x-log-y plots* are *region* and *minmax* to draw filled polygons:

$xyplot region *series xlist ylist*
Draw a filled polygon (region). The configuration of the series influences the polygon as

follows: *-fillcolour* is used to fill the polygon, *-colour* is used for the boundary (set it to {} if no boundary is required and *-width* determines the width of the boundary.

string *series* (in)
Name of the data series the new region belongs to.

float *xlist* (in)
List of X-coordinates for the region.

float *ycrd* (in)
List of Y-coordinates for the region.

$xyplot minmax *series xcrd ymin ymax*
Draw a filled strip representing a minimum and a maximum. The configuration of the series influences the polygon as follows: *-fillcolour* is used to fill the polygon, *-colour* is used for the boundary (set it to {} if no boundary is required and *-width* determines the width of the boundary.

The arguments *ymin* and *ymax* may be empty to get an extra vertex in the strip. If both are empty, a new strip is started. For best results, the x-coordinate should be specified in ascending order.

string *series* (in)
Name of the data series the new min/max strip belongs to.

float *xcrd* (in)
X-coordinate for the strip.

float *ymin* (in)
The minimum y-coordinate for the strip.

float *ymax* (in)
The maximum y-coordinate for the strip.

Note on histograms:

For histograms the x-coordinate that is given is interpreted to be the x-coordinate of the *right* side of the bar (or line segment). The first bar starts at the y-axis on the left. To completely fill the range of the x-axis, you should draw a bar at the maximum x-coordinate.

For histograms you can also use the **plotcumulative** command:

$histogram plotcumulative *series xcrd ycrd*
see below ...

The arguments mean exactly the same as for the **plot** command, but the data are accumulated to the previous values.

For *xy plots*:

$xyplot trend *series xcrd ycrd*
Draw or update a trend line using the data given sofar.

string *series* (in)
Name of the data series the trend line belongs to.

float *xcrd* (in)
X-coordinate of the new data point

float *ycrd* (in)
Y-coordinate of the new data point

$xyplot rchart *series xcrd ycrd*

Draw data in the same way as the plot method, but with two lines added that indicate the expected range (+/- 3*standard deviation) of the data.

string *series* (in)

Name of the data series the data point belongs to.

float *xcrd* (in)

X-coordinate of the new data point

float *ycrd* (in)

Y-coordinate of the new data point

$xyplot interval *series xcrd ymin ymax* ?ycentr?

Add a vertical error interval to the plot. The interval is drawn from ymin to ymax. If the ycentr argument is given, a symbol is drawn at that position.

string *series* (in)

Name of the data series the interval belongs to.

float *xcrd* (in)

X-coordinate of the interval

float *ymin* (in)

Minimum y-coordinate of the interval.

float *ymax* (in)

Maximum y-coordinate of the interval.

float *ycentr* (in)

Y-coordinate to draw the symbol at (optional)

$xyplot box-and-whiskers *series xcrd ycrd*

Draw a box and whiskers in the plot. If the argument *xcrd* is a list of several values and the argument *ycrd* is a single value, a horizontal box is drawn with the quartiles determined from the list of values contained in *xcrd*.

If, instead, the argument *ycrd* contains a list of several values and the argument *xcrd* a single value, then a vertical box is drawn and the quartiles are determined from *ycrd*. (There must be exactly one list of several values. Otherwise an error is reported.)

The option -boxwidth to the dataconfig command determines the width (or height) of the box (default: 10 pixels).

The option -whiskers to the dataconfig command determines whether the whiskers are drawn to the extreme values (value: extremes), to 1.5 times the interquartile range (value: IQR or iqr), or not at all (value: none). If the value is 'IQR' (uppercase), then also extreme values will be shown (from 1.5 to 3 times the IQR as dots, above 3 times IQR as stars). If the value is 'iqr' (lowercase) no extreme values will be shown (default value: IQR).

The option -whiskerwidth to the dataconfig command determines the thickness of the line that draws the whiskers (default: 1 pixel).

The option -mediancolour to the dataconfig command determines the colour of the line used to draw the median within the box (default: same as -colour).

The option -medianwidth to the dataconfig command determines the thickness of the line

that draws the median within the box (default: 1 pixel).

string *series* (in)
Name of the data series the box-and-whiskers belongs to.

float *xcrd* (in)
X-coordinate of the box or a list of values.

float *ymin* (in)
Y-coordinate of the box or a list of values.

The box ends at the 1st and 3rd quartile, while the whiskers by default are plotted to span 1.5 IQR (interquartile range) from the 1st and 3rd quartile.

$xyplot violin *series xcrd ycrd*
Draw a violin plot based on the given data set. It is an alternative to the box and whiskers presentation. Just as with the box-and-whiskers subcommand, if the argument *xcrd* is a list of several values and the argument *ycrd* is a single value, a horizontal "violin" is drawn.

If, instead, the argument *ycrd* contains a list of several values and the argument *xcrd* a single value, then a vertical "violin" is drawn.

The option -violinwidth to the dataconfig command determines the width (or height) of the violin (default: 50 pixels).

The option -colour (-color) determines the outline colour and the option -fillcolour (-fillcolor) determines the colour by which the violin is filled (defaults: black and not filled).

string *series* (in)
Name of the data series to use.

float *xcrd* (in)
X-coordinate of the violin or a list of values.

float *ymin* (in)
Y-coordinate of the violin or a list of values.

$xyplot vector *series xcrd ycrd ucmp vcmp*
Draw a vector in the plot. The vector can be given as either cartesian coordinates or as length/angle, where the angle is in degrees and is interpreted according to the mathematical convention or the nautical. (See the vectorconfig subcommand)

string *series* (in)
Name of the series the vector belongs to. Determines the appearance and interpretation.

float *xcrd* (in)
X-coordinate of the point where the arrow appears

float *ycrd* (in)
Y-coordinate of the point where the arrow appears

float *ucmp* (in)
X-component or the length of the vector

float *ycentr* (in)
Y-component or the angle of the vector

$xyplot vectorconfig *series* -option *value* ...
] Set the vector drawing options for a particular series

string *series* (in)
Name of the series the vector belongs to.

The options can be one of the following:
colour
The colour of the arrow (default: black; synonym: color)
scale value
The scale factor used to convert the length of the arrow into a number of pixels (default: 1.0)
centred onoff
Logical value indicating that the xy-coordinates are to be used as the start of the arrow or as the centre (default: 0; synonym: centered)
type keyword
Interpretation of the vector components. Can be "cartesian" (default), in which case the x- and y-components are expected, "polar" (the angle 0 coincides with the positive x-axis, 90 coincides with the positive y-axis) or "nautical" (0 is "north" and 90 is "east").

$xyplot dot *series xcrd ycrd value*
Draw a dot in the plot. The size and colour is determined by the value and by the options set for the series it belongs to. (See the dotconfig subcommand)
string *series* (in)
Name of the series the dot belongs to. Determines size and colour
float *xcrd* (in)
X-coordinate of the point where the arrow appears
float *ycrd* (in)
Y-coordinate of the point where the arrow appears
float *value* (in)
Value determining size and colour

$xyplot dotconfig *series* -option *value* ...
] Set the dot drawing options for a particular series
string *series* (in)
Name of the series the dot belongs to.

The options can be one of the following:
colour
The colour of the dot if no scaling is used or the value exceeds the last limit of the classes.
scale value
The scale factor used to convert the value into the radius of the dot in pixels (default: 1.0)
radius value
The default radius of the dots, used if there is no scaling by value (in pixels; default: 3)
scalebyvalue onoff
Determines whether the dots all have the same size or a size depending on the given value (default: on).
outline onoff
Draw a black circle around the dot or not (default: on)

classes list
Set the limits and the corresponding colours. For instance:

```
    $xyplot series1 -classes {0 blue 1 green} -colour red
```

will cause a blue dot to be drawn for values smaller than 0, a green dot for values larger/equal 0 but lower than 1 and a red dot for values larger/equal 1.

3deffect onoff
Show a highlight in the dots, to mimick a 3D effect (default: off)

If there is no list of classes for the particular series, the dots are scaled by the value.

You can combine the colouring by value and the scaling by value by setting a list of classes and setting the *scalebyvalue* option on.

$xyplot contourlines *xcrd ycrd values* ?classes?
Draw contour lines for the values given on the grid. The grid is defined by the xcrd and ycrd arguments. The xcrd argument (resp. ycrd) is expected to be a matrix, implemented as a list of lists which gives the x-coordinates (resp. y-coordinates) of the grid cell corners. The function values are given at these corners. The number of rows in xcrd (resp. ycrd) is ny and each row contains nx values so that the total number of values in xcrd (resp. ycrd) is nx * ny. The classes determine which contour lines are drawn. If a value on one of the corners is missing, the contour lines in that cell will not be drawn.

Entries in the legend are drawn via the *legendisolines* subcommand.

list *xcrd* (in)
List of lists, each value is an x-coordinate for a grid cell corner

list *ycrd* (in)
List of lists, each value is an y-coordinate for a grid cell corner

list *values* (in)
List of lists, each value is the value at a grid cell corner

list *classes* (in)
List of class values or a list of lists of two elements (each inner list the class value and the colour to be used). If empty or missing, the classes are determined automatically.

Note: The class values must enclose the whole range of values. *Note:* The xcrd argument is generally made of nypoints identical rows, while each row of ycrd is made with one single value.

$xyplot contourlinesfunctionvalues *xvec yvec valuesmat* ?classes?
Draw contour lines for the values given on the grid. The grid is defined by the xvec and yvec arguments. Here, xvec (resp. yvec) is a list of x-coordinates (resp. y-coordinates). The number of values in xvec (resp. yvec) is the number of points in the x-coordinate (resp. y-coordinate). The function values are given at these corners. The classes determine which contour lines are drawn. If a value on one of the corners is missing, the contour lines in that cell will not be drawn.

Entries in the legend are drawn via the *legendisolines* subcommand.

list *xvec* (in)
List of x-coordinates in increasing order.

list *yvec* (in)
List y-coordinates in increasing order.

list *valuesmat* (in)
List of ny lists of nx values, each value is the value at a grid cell corner. The total number of values is valuesmat is nx * ny.

list *classes* (in)
List of class values or a list of lists of two elements (each inner list the class value and the colour to be used). If empty or missing, the classes are determined automatically.

Note: The class values must enclose the whole range of values.

$xyplot contourfill *xcrd ycrd values* ?classes?
Draw filled contours for the values given on the grid. (The use of this method is identical to the "contourlines" method).

Entries in the legend are drawn via the *legendshades* subcommand.

$xyplot contourbox *xcrd ycrd values* ?classes?
Draw the cells as filled quadrangles. The colour is determined from the average of the values on all four corners.

Entries in the legend are drawn via the *legendshades* subcommand.

$xyplot colorMap *colours*
Set the colours to be used with the contour methods. The argument is either a predefined colourmap (grey/gray, jet, hot or cool) or a list of colours. When selecting the colours for actually drawing the contours, the given colours will be interpolated (based on the HLS scheme).

list *colours* (in)
List of colour names or colour values or one of the predefined maps:
- grey or gray: gray colours from dark to light
- jet: rainbow colours
- hot: colours from yellow via red to darkred
- cool: colours from cyan via blue to magenta

$xyplot legendisolines *values classes*
Add the contour classes to the legend as coloured lines. The text indicates the values.

list *values* (in)
The list of values as used for the actual drawing. This argument is used only if the list of classes is empty.

list *values* (in)
The list of classes as used for the actual drawing.

$xyplot legendshades *values classes*
Add the contour classes to the legend as coloured rectangles. The text indicates the values.

list *values* (in)

The list of values as used for the actual drawing. This argument is used only if the list of classes is empty.

list *values* (in)
The list of classes as used for the actual drawing.

$xyplot grid *xcrd ycrd*
Draw the grid cells as lines connecting the (valid) grid points.

list *xcrd* (in)
List of lists, each value is an x-coordinate for a grid cell corner

list *ycrd* (in)
List of lists, each value is an y-coordinate for a grid cell corner

$xyplot xband *ymin ymax*
Draw a light grey band in the plot, ranging over the full x-axis. This can be used to indicate a "typical" range for the data.

float *ymin* (in)
Lower bound for the band

float *ymax* (in)
Upper bound for the band

$xyplot yband *xmin xmax*
Draw a light grey band in the plot, ranging over the full y-axis. This can be used to indicate a "typical" range for the data.

float *xmin* (in)
Lower bound for the band

float *xmax* (in)
Upper bound for the band

$xyplot labeldot *x y text orient*
Draw a label and a symbol in the plot. The label will appear near the symbol. The label will be drawn in grey, so as not to be too conspicuous.

You can configure the appearance of the symbol by using the data series name "labeldot": *$w dataconfig labeldot -colour red -type symbol -symbol dot*

float *x* (in)
X-coordinate of the symbol to be drawn

float *y* (in)
Y-coordinate of the symbol to be drawn

string *text* (in)
Text for the label

string *orient* (in)
Optional orientation (one of w, e, n, s) defining the position of the label with respect to the symbol. It defaults to w (so the label appears left of the symbol).

For *polar plots*:

$polarplot plot *series radius angle*
Add a data point to the polar plot.

string *series* (in)
Name of the data series the new point belongs to.

float *radius* (in)
Radial coordinate of the new point.

float *angle* (in)
Angular coordinate of the new point (in degrees).

For *wind rose diagrams*:

$windrose plot *data colour*
Draw the data contained in the *data* argument. The data are added to the existing spokes towards the outer circle.

list *data* (in)
List of data (the length should correspond to the number of sectors)

string *colour*
Colour in which the new segments will be drawn

For *3D plots*:

$plot3d plotfunc *function*
Plot a function defined over two variables **x** and **y**. The resolution is determined by the set grid sizes (see the method **gridsize** for more information).

string *function* (in)
Name of the procedure that calculates the z-value for the given x and y coordinates. The procedure has to accept two float arguments (x is first argument, y is second) and return a floating-point value.

$plot3d plotfuncont *function contours*
Plot a function defined over two variables **x** and **y** using the contour levels in **contours** to colour the surface. The resolution is determined by the set grid sizes (see the method **gridsize** for more information).

string *function* (in)
Name of the procedure that calculates the z-value for the given x and y coordinates. The procedure has to accept two float arguments (x is first argument, y is second) and return a floating-point value.

list *contours* (in)
List of values in ascending order that represent the contour levels (the boundaries between the colours in the contour map).

$plot3d gridsize *nxcells nycells*
Set the grid size in the two directions. Together they determine how many polygons will be drawn for a function plot.

int *nxcells* (in)
Number of grid cells in x direction. Has to be an integer number greater than zero.

int *nycells* (in)
Number of grid cells in y direction. Has to be an integer number greater than zero.

$plot3d plotdata *data*

Plot a matrix of data.

list *data* (in)

The data to be plotted. The data has to be provided as a nested list with 2 levels. The outer list contains rows, drawn in y-direction, and each row is a list whose elements are drawn in x-direction, for the columns. Example:

```
set data {
{1.0 2.0 3.0}
{4.0 5.0 6.0}
}
```

$plot3d interpolatedata *data contours*

Plot the data using bilinear interpolation with the contour levels in **contours** to colour the surface. The resolution is determined by the set grid sizes (see the method **gridsize** for more information).

list *data* (in)

The data to be plotted, just as for the plotdata subcommand.

list *contours* (in)

List of values in ascending order that represent the contour levels (the boundaries between the colours in the contour map).

$plot3d colour *fill border*

Configure the colour to use for polygon borders and inner area. *Note:* The "color" subcommand is a synonym.

color *fill* (in)

The colour to use for filling the polygons.

color *border* (in)

The colour to use for the border of the polygons.

$plot3d ribbon *yzpairs*

Plot a ribbon based on the pairs of yz-coordinates. The colours for the ribbon itself and the edge are taken from the colours option.

list *yzpairs* (in)

List of pairs of yz-coordinates

For 3D ribbon plots:

$plot3d plot *yzpairs*

Plot a ribbon based on the pairs of yz-coordinates. The colours for the ribbon itself and the edge are taken from the colours option.

list *yzpairs* (in)

List of pairs of yz-coordinates

For *xy plots*, *stripcharts*, *histograms*, *polar plots* and *ternary diagrams*:

$xyplot dataconfig *series* -option *value* ...

Set the value for one or more options regarding the drawing of data of a specific series.

string *series* (in)

Name of the data series whose configuration we are changing.

The following options are allowed:

colour c
see below ...

color c
The colour to be used when drawing the data series.

type enum
The drawing mode chosen for the series. This can be one of **line**, **symbol**, or **both**.

symbol enum
What kind of symbol to draw. The value of this option is ignored when the drawing mode **line** was chosen. This can be one of **plus**, **cross**, **circle**, **up** (triangle pointing up), **down** (triangle pointing down), **dot** (filled circle), **upfilled** or **downfilled** (filled triangles).

radius integer
The size of the radius of the symbol. The total width of the symbol will be 2 times the radius size. The default radius is 4.

width integer
The width of the line (if drawn) or the width of the polygon outline (if -filled).

filled enum
Whether to fill the area above or below the data line or not. Can be one of: **no**, **up** or **down** (Special effects)

fillcolour colour
Colour to use when filling the area associated with the data line.

style enum
The style to be used for histograms:
- **filled**: Fill the area under the data points with bars (default)
- **spike**: Draw vertical lines from the y-axis (lower boundary) to the data point
- **symbol**: Draw a symbol at the data point
- **plateau**: Draw a horizontal line at the height of the data point
- **stair**: Draw a continuous stair-like line connecting the data points

smooth boolean
(Used with ternary diagrams) Whether to draw the lines and filled polygons with the smooth option on (rounded corners) or not.

For *piecharts* and *spiral pies*:

$pie plot *data*
Fill a piechart.

list *data* (in)
A list of pairs (labels and values). The values determine the relative size of the circle segments. The labels are drawn beside the circle.

$pie colours *colour1 colour2* ...
Set the colours to be used.

color *colour1* (in)
The first colour.

color *colour2* (in)
The second colour, and so on.

$pie explode *segment*
Explode a segment (that is: move one segment out of the circle). If the segment is indicated as "auto", then you can click on a segment. This will be exploded instead of any previously exploded segment.
int *segment*
The segment to be exploded or "auto" if you want to do this interactively.

For *radial charts*:

$radial plot *data colour thickness*
Draw a new line in the radial chart
list *data* (in)
A list of data (one for each spoke). The values determine the distance from the centre of the line connecting the spokes.
color *colour* (in)
The colour for the line.
int *thickness* (in)
An optional argument for the thickness of the line.

$pie colours *colour1 colour2* ...
Set the colours to be used.
color *colour1* (in)
The first colour.
color *colour2* (in)
The second colour, and so on.

For *vertical barcharts*:

$barchart plot *series ydata colour* ?dir? ?brightness?
Add a data series to a barchart. The bars are tagged with a tag "data_\$series" to identify them.
string *series* (in)
Name of the series the values belong to.
list *ydata* (in)
A list of values, one for each x-axis label.
color *colour* (in)
The colour of the bars.
string *dir* (in)
If given, "top-down" or "bottom-up", to indicate the direction in which the colour changes. (If not given, a uniform colour is used).
string *brightness* (in)
If given, "bright" or "dark" (defaulting to "bright"). The colour will change to respectively white or black, depending on the direction.

$barchart config -option *value* ...

Set options for drawing the bars.

showvalues boolean
Whether to show the values or not (above the bars)

valuefont newfont
Name of the font to use for the values

valuecolour colour
Colour for the values

valueformat format
Format string to use for formatting the values

For *horizontal barcharts*:

$barchart plot *series xdata colour* ?dir? ?brightness?
Add a data series to a barchart. The bars are tagged with a tag "data_\$series" to identify them.

string *series* (in)
Name of the series the values belong to.

list *xdata* (in)
A list of values, one for each y-axis label.

color *colour* (in)
The colour of the bars.

string *dir* (in)
If given, "left-right" or "right-left", to indicate the direction in which the colour changes. (If not given, a uniform colour is used).

string *brightness* (in)
If given, "bright" or "dark" (defaulting to "bright"). The colour will change to respectively white or black, depending on the direction.

$barchart config -option *value* ...
Set options for drawing the bars.

showvalues boolean
Whether to show the values or not (to the right of the bars)

valuefont newfont
Name of the font to use for the values

valuecolour colour
Colour for the values

valueformat format
Format string to use for formatting the values

For *3D barcharts*:

$barchart plot *label yvalue colour*
Add the next bar to the barchart.

string *label* (in)
The label to be shown below the column.

float *yvalue* (in)
The value that determines the height of the column

color *colour* (in)
The colour of the column.

$barchart config -option *value* ...
Set one or more configuration parameters. The following options are supported:
usebackground boolean
Whether to draw walls to the left and to the back of the columns or not
useticklines boolean
Whether to draw ticklines on the walls or not
showvalues boolean
Whether to show the values or not
labelfont newfont
Name of the font to use for labels
labelcolour colour
Colour for the labels
valuefont newfont
Name of the font to use for the values
valuecolour colour
Colour for the values

For *3D ribbon charts*:

$ribbon line *xypairs colour*
Plot the given xy-pairs as a ribbon in the chart
list *xypairs* (in)
The pairs of x/y values to be drawn (the series is drawn as a whole)
color *colour* (in)
The colour of the ribbon.

$ribbon area *xypairs colour*
Plot the given xy-pairs as a ribbon with a filled area in front. The effect is that of a box with the data as its upper surface.
list *xypairs* (in)
The pairs of x/y values to be drawn (the series is drawn as a whole)
color *colour* (in)
The colour of the ribbon/area.

For *boxplots*:

$boxplot plot *series label values*
Add a box-and-whisker to the plot. The dataconfig command can be used to customize the box-and-whisker (see the box-and-whiskers command for the xyplot for details).
string *series* (in)
Name of the data series the box belongs to
string *label* (in)
The label along the x- or y-axis to which the data belong
list *values* (in)

List of raw values, the extent of the box and the whiskers will be determined from this list.

$boxplot violin *series label values*

Add a "violin" to the plot. The dataconfig command can be used to customize the appearance (see the violin command for the xyplot for details).

string *series* (in)
Name of the data series to be used for settings

string *label* (in)
The label along the x- or y-axis to which the data belong

list *values* (in)
List of raw values, the extent of the box and the whiskers will be determined from this list.

For *timecharts*:

$timechart period *text time_begin time_end colour*

Add a time period to the chart.

string *text* (in)
The text describing the period.

string *time_begin* (in)
Start time of the period.

string *time_end* (in)
Stop time of the period.

color *colour* (in)
The colour of the bar (defaults to black).

$timechart addperiod *time_begin time_end colour*

Add a new period to the current row in the chart. This allows you to highlight several periods in the same row. No new text is drawn.

string *time_begin* (in)
Start time of the period.

string *time_end* (in)
Stop time of the period.

color *colour* (in)
The colour of the bar (defaults to black).

$timechart milestone *text time colour*

Add a *milestone* (represented as an point-down triangle) to the chart.

string *text* (in)
The text describing the milestone.

string *time* (in)
Time at which the milestone must be positioned.

color *colour* (in)
The colour of the triangle (defaults to black).

$timechart addmilestone *time colour*

Add another *milestone* to the current row in the chart.

string *time* (in)

Time at which the milestone must be positioned.

color *colour* (in)

The colour of the triangle (defaults to black).

$timechart vertline *text time colour*

Add a vertical line (to indicate the start of the month for instance) to the chart in the specified colour.

string *text* (in)

The text appearing at the top (an abbreviation of the date/time for instance).

string *time* (in)

Time at which the line must be positioned.

color *colour* (in)

The colour of the line to be drawn (defaults to black)

$timechart hscroll *scrollbar*

Connect a horizontal scrollbar to the chart. See also the section on scrolling.

widget *scrollbar* (in)

The horizontal scrollbar that is to be connected to the chart

$timechart vscroll *scrollbar*

Connect a vertical scrollbar to the chart. See also the section on scrolling.

widget *scrollbar* (in)

The vertical scrollbar that is to be connected to the chart

For *Gantt charts*:

$ganttchart task *text time_begin time_end completed*

Add a task with its period and level of completion to the chart. Returns a list of canvas items that can be used for further manipulations, like connecting two tasks.

string *text* (in)

The text describing the task.

string *time_begin* (in)

Start time of the task.

string *time_end* (in)

Stop time of the task.

float *completed* (in)

The percentage of the task that is completed.

$ganttchart milestone *text time colour*

Add a *milestone* (represented as an point-down triangle) to the chart.

string *text* (in)

The text describing the milestone.

string *time* (in)

Time at which the milestone must be positioned.

color *colour* (in)

The colour of the triangle (defaults to black).

$ganttchart vertline *text time*

Add a vertical line (to indicate the start of the month for instance) to the chart.

string *text* (in)

The text appearing at the top (an abbreviation of the date/time for instance).

string *time* (in)

Time at which the line must be positioned.

$ganttchart connect *from to*

Add an arrow that connects the *from* task with the *to* task.

list *from* (in)

The list of items returned by the "task" command that represents the task from which the arrow starts.

string *text* (in)

The text summarising the tasks

list *args* (in)

One or more tasks (the lists returned by the "task" command). They are shifted down to make room for the summary.

list *to* (in)

The list of items returned by the "task" command that represents the task at which the arrow ends.

$ganttchart summary *text args*

Add a summary item that spans all the tasks listed. The graphical representation is a thick bar running from the leftmost task to the rightmost.

Use this command before connecting the tasks, as the arrow would not be shifted down!

string *text* (in)

The text summarising the tasks

list *args* (in)

One or more tasks (the lists returned by the "task" command). They are shifted down to make room for the summary.

$ganttchart color *keyword newcolor*

Set the colour of a part of the Gantt chart. These colours hold for all items of that type.

string *keyword* (in)

The keyword indicates which part of the Gantt chart to change:
- description - the colour of the descriptive text
- completed - the colour of the filled bar representing the completed part of a task
- left - the colour for the part that is not yet completed
- odd - the background colour for the odd entries
- even - the background colour for the even entries
- summary - the colour for the summary text
- summarybar - the colour for the bar for a summary

string *newcolor* (in)

The new colour for the chosen items.

$ganttchart font *keyword newfont*

Set the font of a part of the Gantt chart. These fonts hold for all items of that type.

string *keyword* (in)

The keyword indicates which part of the Gantt chart to change:
- description - the font used for descriptive text
- summary - the font used for summaries
- scale - the font used for the time scale

string *newfont* (in)

The new font for the chosen items.

$ganttchart hscroll *scrollbar*

Connect a horizontal scrollbar to the chart. See also the section on scrolling.

widget *scrollbar* (in)

The horizontal scrollbar that is to be connected to the chart

$ganttchart vscroll *scrollbar*

Connect a vertical scrollbar to the chart. See also the section on scrolling.

widget *scrollbar* (in)

The vertical scrollbar that is to be connected to the chart

For *isometric plots* (to be extended):

$isoplot plot rectangle *x1 y1 x2 y2 colour*

Plot the outlines of a rectangle.

float *x1* (in)

Minimum x coordinate of the rectangle to be drawn.

float *y1* (in)

Minimum y coordinate of the rectangle.

float *x2* (in)

Maximum x coordinate of the rectangle to be drawn.

float *y2* (in)

Maximum y coordinate of the rectangle.

color *colour* (in)

The colour of the rectangle.

$isoplot plot filled-rectangle *x1 y1 x2 y2 colour*

Plot a rectangle filled with the given colour.

float *x1* (in)

Minimum x coordinate of the rectangle to be drawn.

float *y1* (in)

Minimum y coordinate of the rectangle.

float *x2* (in)

Maximum x coordinate of the rectangle to be drawn.

float *y2* (in)

Maximum y coordinate of the rectangle.

color *colour* (in)

The colour of the rectangle.

$isoplot plot circle *xc yc radius colour*
Plot the outline of a circle.
float *xc* (in)
X coordinate of the circle's centre.
float *yc* (in)
Y coordinate of the circle's centre.
color *colour* (in)
The colour of the circle.

$isoplot plot filled-circle *xc yc radius colour*
Plot a circle filled with the given colour.
float *xc* (in)
X coordinate of the circle's centre.
float *yc* (in)
Y coordinate of the circle's centre.
color *colour* (in)
The colour of the circle.

For *tables* you can use the following subcommands:

$table row *items*
Draw a single row of items. The appearance of the items can be controlled explicitly via the format command.
list *items* (in)
List of text items to be drawn, one per column

$table separator
Draw a horizontal line to separate two rows
$table formatcommand *procname*
Set the procedure that controls the formatting of items. By default items are simply drawn as centered text.
string *procname* (in)
Name of the procedure to be used. Its signature is:

```
proc procname {table widget row column value} {...}
```

Use the cellconfigure subcommand to set the attributes per cell.

$table cellconfigure *args*
Set the attributes for the next cell(s) to be drawn.
list *args* (in)
Key-value pairs: -background sets the background colour of the cells, -cell sets the foreground colour, -font sets the text font, -anchor sets the position of the text within the cell and -justify controls the layout of multiline text.

For *ternary diagrams* you can use the following subcommands:

$ternary plot *series xcrd ycrd zcrd text dir*

Draw a single data point with a label. The three coordinates are scaled so that a unique point in the triangle results. A label is drawn next to it.

string *series* (in)

Name of the data series the point belongs to (used to determine colour and symbol)

float *xcrd* (in)

X-coordinate of the data point (refers to the lower-left corner).

float *ycrd* (in)

Y-coordinate of the data point (refers to the lower-right corner).

float *zcrd* (in)

Z-coordinate of the data point (refers to the top corner).

string *text* (in)

Label describing the data point.

string *dir* (in)

Optional string indicating the direction in which to plot the label (e, n, etc.)

$ternary line *series coords*

Draw a continuous line based on the given coordinates (triplets).

string *series* (in)

Name of the data series the line belongs to (used to determine colour and smoothness)

list *coords* (in)

The coordinates of the points that determine the line (note that a point is defined by three coordinates).

$ternary fill *series coords*

Draw a filled polygon based on the given coordinates (triplets).

string *series* (in)

Name of the data series the polygon belongs to (used to determine colour and smoothness)

list *coords* (in)

The coordinates of the points that determine the polygon (note that a point is defined by three coordinates).

$ternary text *xtext ytext ztext*

Draw text at the three corners of the diagram to identify the components.

string *xtext* (in)

Text to be plotted at the lower-left corner

string *ytext* (in)

Text to be plotted at the lower-right corner

string *ztext* (in)

Text to be plotted at the top corner

$ternary ticklines *colour*

Draw ticklines to facilitate reading off the diagram.

string *colour* (in)

Optional argument used as the colour of the ticklines. Defaults to grey.

For *status timeline plots* you can use the following subcommands:

$timeline plot *series item start stop color*

Draw a bar in the given colour from *start* to *stop* for the item *item*. The item is a convenient label - there is no relation to the labels along the axis. The items are drawn from bottom to top.

string *item* (in)

Name to identify the bar. See remark above.

float *start* (in)

X-coordinate (or time) at which the bar starts

float *stop* (in)

X-coordinate (or time) at which the bar stops

string *color* (in)

Colour to use for the bar. Defaults to black.

$timeline vertline *text time args*

Draw a vertical line to indicate a significant moment.

string *text* (in)

Text to identify the moment.

float *time* (in)

X-coordinate (or time) at which the line is to be drawn

list *args* (in)

Individual optional arguments that will be passed to the *create line* subcommand of the underlying canvas. This way you can set the colour or the line width of the vertical line.

For *dendrograms* there is one specific subcommand:

$dendrogram plot *series data*

Draw a dendrogram according to the options given or implied when the dendrogram command was created. The "series" name allows you to select the colour of the branches (-colour/-color) as well as the colour and font of the labels (-labelcolour/-labelcolor and -labelfont).

string *series* (in)

Name of the set of options to draw the dendrogram with.

list *data* (in)

Nested list, describing the tree structure. See the *createDendrogram* command.

Coordinate transformations

Besides the commands that deal with the plots and charts directly, there are a number of commands that can be used to convert world coordinates to pixels and vice versa. These include:

::Plotchart::viewPort *w pxmin pymin pxmax pymax*

Set the viewport for window *w*. Should be used in cooperation with **::Plotchart::worldCoordinates**.

widget *w* (in)

Name of the window (canvas widget) in question.

float *pxmin* (in)
Left-most pixel coordinate.

float *pymin* (in)
Top-most pixel coordinate (remember: the vertical pixel coordinate starts with 0 at the top!).

float *pxmax* (in)
Right-most pixel coordinate.

float *pymax* (in)
Bottom-most pixel coordinate.

::Plotchart::worldCoordinates *w xmin ymin xmax ymax*
Set the extreme world coordinates for window *w*. The world coordinates need not be in ascending order (i.e. xmin can be larger than xmax, so that a reversal of the x-axis is achieved).

widget *w* (in)
Name of the window (canvas widget) in question.

float *xmin* (in)
X-coordinate to be mapped to left side of viewport.

float *ymin* (in)
Y-coordinate to be mapped to bottom of viewport.

float *xmax* (in)
X-coordinate to be mapped to right side of viewport.

float *ymax* (in)
Y-coordinate to be mapped to top side of viewport.

::Plotchart::world3DCoordinates *w xmin ymin zmin xmax ymax zmax*
Set the extreme three-dimensional world coordinates for window *w*. The world coordinates need not be in ascending order (i.e. xmin can be larger than xmax, so that a reversal of the x-axis is achieved).

widget *w* (in)
Name of the window (canvas widget) in question.

float *xmin* (in)
X-coordinate to be mapped to front side of the 3D viewport.

float *ymin* (in)
Y-coordinate to be mapped to left side of the viewport.

float *zmin* (in)
Z-coordinate to be mapped to bottom of viewport.

float *xmax* (in)
X-coordinate to be mapped to back side of viewport.

float *ymax* (in)
Y-coordinate to be mapped to right side of viewport.

float *zmax* (in)
Z-coordinate to be mapped to top side of viewport.

::Plotchart::coordsToPixel *w x y*

Return a list of pixel coordinates valid for the given window.

widget *w* (in)
Name of the canvas alias (as returned by [\$anyplot canvas]) in question.

float *x* (in)
X-coordinate to be mapped.

float *y* (in)
Y-coordinate to be mapped.

::Plotchart::coords3DToPixel *w x y z*

Return a list of pixel coordinates valid for the given window.

widget *w* (in)
Name of the canvas alias (as returned by [\$anyplot canvas]) in question.

float *x* (in)
X-coordinate to be mapped.

float *y* (in)
Y-coordinate to be mapped.

float *y* (in)
Z-coordinate to be mapped.

::Plotchart::polarCoordinates *w radmax*

Set the extreme polar coordinates for window *w*. The angle always runs from 0 to 360 degrees and the radius starts at 0. Hence you only need to give the maximum radius. *Note:* If the viewport is not square, this procedure will not adjust the extremes, so that would result in an elliptical plot. The creation routine for a polar plot always determines a square viewport.

widget *w* (in)
Name of the canvas alias (as returned by [\$anyplot canvas]) in question.

float *radmax* (in)
Maximum radius.

::Plotchart::polarToPixel *w rad phi*

Wrapper for a call to **::Plotchart::coordsToPixel**. *Note:* This procedure has been deprecated - you should use the procedure **::Plotchart::coordsToPixel** instead.

widget *w* (in)
Name of the canvas alias (as returned by [\$anyplot canvas]) in question.

float *rad* (in)
Radius of the point.

float *phi* (in)
Angle to the positive x-axis.

::Plotchart::pixelToCoords *w x y*

Return a list of world coordinates valid for the given window.

widget *w* (in)
Name of the canvas alias (as returned by [\$anyplot canvas]) in question.

float *x* (in)
X-pixel to be mapped.

float *y* (in)

Y-pixel to be mapped.

::Plotchart::pixelToIndex *w x y*

Return the index of the pie segment containing the pixel coordinates (x,y)

widget *w* (in)

Name of the canvas alias (as returned by [\$anyplot canvas]) in question, holding a piechart.

float *x* (in)

X-pixel to be mapped.

float *y* (in)

Y-pixel to be mapped.

Furthermore there is a routine to determine "pretty" numbers for use with an axis:

::Plotchart::determineScale *xmin xmax inverted*

Determine "pretty" numbers from the given range and return a list containing the minimum, maximum and stepsize that can be used for a (linear) axis.

float *xmin* (in)

Rough minimum value for the scaling

float *xmax* (in)

Rough maximum value for the scaling.

boolean *inverted* (in)

Optional argument: if 1, then the returned list produces an inverted axis. Defaults to 0 (the axis will be from minimum to maximum)

::Plotchart::determineScaleFromList *values inverted*

Determine "pretty" numbers from the given list of values and return a list containing the minimum, maximum and stepsize that can be used for a (linear) axis.

float *values* (in)

List of values that will be examined. May contain missing values (empty strings)

boolean *inverted* (in)

Optional argument: if 1, then the returned list produces an inverted axis. Defaults to 0 (the axis will be from minimum to maximum)

Missing values

Often data that need to be plotted contain gaps - in a series of measurement data, they can occur because the equipment failed, a sample was not collected correctly or for many other reasons. The *Plotchart* handles these gaps by assuming that one or both coordinates of such data points are an empty string:

```
    #
    # Create the plot with its x- and y-axes
    #
    set s [::Plotchart::createXYPlot .c {0.0 100.0 10.0} {0.0 100.0
    20.0}]

    foreach {x y} {0.0 32.0 10.0 {} 25.0 60.0 78.0 11.0 } {
        $s plot series1 $x $y
    }
```

The effect varies according to the type of plot:

- For xy-plots, radial plots and strip charts the missing data point causes a gap in the line through the points.
- For barchats, missing values are treated as if a value of zero was given.
- For time charts and Gantt charts missing values cause errors - there is no use for them there.

Other output formats

Besides output to the canvas on screen, the module is capable, via **canvas postscript**, of producing PostScript files. One may wonder whether it is possible to extend this set of output formats and the answer is "yes". This section tries to sum up the aspects of using this module for another sort of output.

One way you can create output files in a different format, is by examining the contents of the canvas after everything has been drawn and render that contents in the right form. This is probably the easiest way, as it involves nothing more than the re-creation of all the elements in the plot that are already there.

The drawback of that method is that you need to have a display, which is not always the case if you run a CGI server or something like that.

An alternative is to emulate the canvas command. For this to work, you need to know which canvas subcommands are used and what for. Obviously, the *create* subcommand is used to create the lines, texts and other items. But also the *raise* and *lower* subcommands are used, because with these the module can influence the drawing order - important to simulate a clipping rectangle around the axes. (The routine DrawMask is responsible for this - if the output format supports proper clipping areas, then a redefinition of this routine might just solve this).

Furthermore, the module uses the *cget* subcommand to find out the sizes of the canvas. A more mundane aspect of this is that the module currently assumes that the text is 14 pixels high and that 80 pixels in width suffice for the axis' labels. No "hook" is provided to customise this.

In summary:

- Emulate the *create* subcommand to create all the items in the correct format
- Emulate the *cget* subcommand for the options -width and -height to allow the correct calculation of the rectangle's position and size
- Solve the problem of *raising* and *lowering* the items so that they are properly clipped, for instance by redefining the routine DrawMask.
- Take care of the currently fixed text size properties

Special effects

As an example of some special effects you can achieve, here is the code for a plot where the area below the data line varies in colour:

```
canvas .c  -background white -width 400 -height 200
pack .c -fill both

set s [::Plotchart::createXYPlot .c {0.0 100.0 10.0} {0.0 100.0 20.0}]

$s background gradient green top-down

$s dataconfig series1 -filled up -fillcolour white

$s plot series1  0.0 20.0
$s plot series1 10.0 20.0
$s plot series1 30.0 50.0
$s plot series1 35.0 45.0
$s plot series1 45.0 25.0
$s plot series1 75.0 55.0
$s plot series1 100.0 55.0

$s plaintext 30.0 60.0 "Peak" south
```

The trick is to fill the background with a colour that changes from green at the top to white at the bottom. Then the area above the data line is filled with a white polygon. Thus the green shading varies with the height of the line.

Room for improvement

In this version there are a lot of things that still need to be implemented:

- More robust handling of incorrect calls (right now the procedures may fail when called incorrectly):
  - The axis drawing routines can not handle inverse axes right now.
  - If the user provides an invalid date/time string, the routines simply throw an error.

Resizing

Plotchart has not been designed to create plots and charts that keep track of the data that are put in. This means that if an application needs to allow the user to resize the window holding the plot or chart, it must take care to redraw the complete plot.

The code below is a simple example of how to do that:

```
package require Plotchart

grid [canvas .c -background white] -sticky news
grid columnconfigure . 0 -weight 1
grid rowconfigure . 0 -weight 1

bind .c <Configure> {doResize}
```

```
proc doPlot {} {
    #
    # Clean up the contents (see also the note below!)
    #
    .c delete all

    #
    # (Re)draw the bar chart
    #
    set p [::Plotchart::createBarchart .c {x y z} {0 100 10} 3]
    $p plot R {10 30 40} red
    $p plot G {30 40 60} green
}

proc doResize {} {
    global redo

    #
    # To avoid redrawing the plot many times during resizing,
    # cancel the callback, until the last one is left.
    #
    if { [info exists redo] } {
        after cancel $redo
    }

    set redo [after 50 doPlot]
}
```

Please note: The code above will work fine for barcharts and many other types of plots, but as **Plotchart** keeps some private information for xy plots, more is needed in these cases. This actually requires a command "destroyPlot" to take care of such details. A next version of **Plotchart** may have that.

Alternatively, you can use the **xyplot** package which is built on top of Plotchart. This package supports zooming in and zooming out, as well as resizing the plot as a whole. Here is a small demonstration program:

```
# xyplot_demo.tcl --
#     Demonstration of the xyplot package
#

package require xyplot

set xydata1 {}
set xydata2 {}
set xydata3 {}
set xydata4 {}
```

```
for { set i 0 } { $i < 1024 } { incr i } {
    lappend xydata1 [expr {$i-1000}] [expr {$i *
    sin($i/4096.0*3.1415*2) * (sin($i/256.0*3.1415*2))}]
    lappend xydata2 [expr {$i-1000}] [expr {$i *
    sin($i/4096.0*3.1415*2) * (sin($i/256.0*3.1415*2) + 0.25 *
    sin($i/256.0*3.1415*6))}]
    lappend xydata3 [expr {$i-1000}] [expr {$i *
    sin($i/4096.0*3.1415*2) * (sin($i/256.0*3.1415*2) + 0.25 *
    sin($i/256.0*3.1415*6) + 0.0625 * sin($i/256.0*3.1415*10))}]
    lappend xydata4 [expr {$i-1000}] [expr {$i *
    sin($i/4096.0*3.1415*2) * (sin($i/256.0*3.1415*2) + 0.25 *
    sin($i/256.0*3.1415*6) + 0.0625 * sin($i/256.0*3.1415*10) +
    0.015625 * sin($i/256.0*3.1415*14))}]
}

set xyp [xyplot .xyp -xformat "%5.0f" -yformat "%5.0f" -title "XY plot
    testing" -background gray90]
pack $xyp -fill both -expand true

set s1 [$xyp add_data sf1 $xydata1 -legend "Serie 1 data" -color red]
set s2 [$xyp add_data sf2 $xydata2 -legend "Serie 2 data" -color green]
set s3 [$xyp add_data sf3 $xydata3 -legend "Serie 3 data" -color blue]
set s4 [$xyp add_data sf4 $xydata4 -legend "Serie 4 data" -color
    orange]

set xyp2 [xyplot .xyp2 -xticks 8 -yticks 4 -yformat %.2f -xformat %.0f]
pack $xyp2 -fill both -expand true

set s1 [$xyp2 add_data sf1 $xydata1]
set s2 [$xyp2 add_data sf2 $xydata2]
set s3 [$xyp2 add_data sf3 $xydata3]
set s4 [$xyp2 add_data sf4 $xydata4]
```

Zooming in is done by selecting a rectangle with the left mouse button pressed. Zooming out is done by pressing the right mouse button. If you resize the window, the canvases inside are resized too. If you zoom in, you can scroll the plot via the scrollbars that are automatically attached.

Zooming in

As the Plotchart package does not keep track of the data itself, rescaling an existing plot - for instance when zooming in - would have to be done by redefining the plot and redrawing the data. However, the canvas widget offers a way out by scaling and moving items, so that zooming in becomes a bit simpler.

Whether zooming is indeed useful, depends on the type of plot. Currently it is defined for XY-plots only. The method is called "rescale" and simply redraws the axes and scales and moves the data items so that they conform to the new axes. The drawback is that any symbols are

scaled by the same amount. The rescale method works best for plots that only have lines, not symbols.

The method works very simply:

```
$p rescale {newxmin newxmax newxstep} {newymin newymax newystep}
```

Configuration options and other commands

The commands **plotconfig** and **plotstyle** can be used to set all manner of options. The command **eraseplot** can be used to completely erase a plot or chart. The syntax of these commands is:

::Plotchart::plotconfig *charttype component property value*

Set a new value for the property of a component in a particular chart or plot type or query its current value. Changed properties only have effect for the consecutive plots, not for the ones already created. Each argument is optional.

Note: The **plotstyle** command offers a more flexible way to control the configuration options.

string *charttype* (in)

The type of chart or plot (see the configuration type that is mentioned for each create command). If not given or empty, a list of chart types is returned. If it is given, the properties for that particular type are used.

string *component* (in)

The component of the plot/chart: leftaxis, rightaxis, background, margin and so on. If not given or empty, a list of components is returned. If it is given, the properties for that particular component will be set for that particular type of chart.

string *property* (in)

The property of the component of the plot/chart: textcolor, thickness of the axis line, etc. If not given or empty, a list of properties is returned. If it is given, that particular property for that particular component will be set for that particular type of chart.

string *value* (in)

The new value for the property. If empty, the current value is returned. If the value is "default", the default value will be restored.

Note, that in some cases an empty value is useful. Use "none" in this case - it can be useful for colours and for formats.

::Plotchart::plotstyle *subcmd style args*

Manipulate the *style* in which subsequent plots will be drawn. The default style is "default", but you can define and load any number of other styles.

string *subcmd* (in)

The subcommand to be executed:
- *configure* - this subcommand allows you to set the options per chart type. It takes the same options as the **plotconfig** command.
- *current* - return the current style
- *load* - make the given style the active style for subsequent plots and charts
- *names* - return the list of currently defined styles

string *style* (in)

The name of the plot style to manipulate

list *args* (in)

The new options for the style. Each option is described by: chart type, component of the chart, property of the component and the new value for the property - see the **plotconfig** command for details.

Below is a detailed list of the components and properties:

- Axes come in a wide variety:
  - leftaxis, rightaxis, topaxis, bottomaxis for the plots with a rectangular shape.
  - xaxis, yaxis and zaxis are used for the 3D plots
  - axis, this represents the radial and tangential axes of a polar plot

- All axes have the following properties:
  - color - the colour of the line and the tickmarks
  - thickness - the width of the line of the axis itself, not the tickmarks
  - ticklength - the length of the tickmarks in pixels. A positive value is outward, a negative value is inward.
  - font - the font for the labels and the text at the axis
  - format - the format for rendering the (numerical) labels. For the time axis it is the format for a date and time.
  - textcolor - the colour for the labels and the text.
  - labeloffset - space (in pixels) between the tickmark and the actual label
  - minorticks - number of minor tickmarks between the major tickmarks
  - shownumbers - show the numbers/labels or not.
  - showaxle - show the axis line or not.

- The *margin* is important for the layout. Currently only the rectangular plots allow the margins to be set: left, right, top and bottom. The values are in pixels.
- The *text* component is meant for any text appearing via the plaintext subcommand. The properties are: textcolor, font and anchor (positioning of the text relative to the given coordinates).
- The *background* has two properties: outercolor, the colour outside of the actual plot, and innercolor, the colour inside the plot. (Note: only "outercolor" has now been implemented).
- The *mask* has one property only: draw. If set to 1, the default, white rectangles are drawn to mimick the effects of clipping - excess data are made invisible this way. Otherwise these rectangles are not drawn. This is useful to control the layout more tightly, for instance with multiple plots in one canvas.
- The *title* component has the same properties as the *text* component (but it is independent of that component). It also has a *background* property: If not set (or set to the empty string) this is the same as the outercolor property of the *background* component, otherwise it is a separate colour.
- The *legend* has three properties: background, border and position. See the legend subcommand for the meaning.
- The *bar* components is used for all barchart-like plots and has three properties: *barwidth* (relative width of the bars in relation to the items along the axis), *innermargin* (the relative width of the gaps between bars or groups of bars) and the *outline* colour.
- The *labels* component is used to describe the appearance of the labels of piecharts and "spiral"

piecharts. The properties are:
- textcolor - colour of the label text
- font - font to be used for the label text
- placement - *out* of the circle or *in* the circle
- sorted - the data are sorted in ascending order first
- shownumbers - the labels are combined with the numbers according to the format
- format - the format to be used (defaults to: "%s (%g)") if the numbers are to shown. The format command gets the label first, then the number)
- formatright - if given, the format to be used for labels and numbers appearing to the right of the pie. The format command gets the number first, then the label. (Defaults to "")

- The *slice* component has properties to control the appearance of the sections in the pie diagram:
  - outline - the colour of the line around the slices (default: black)
  - outlinewidth - width of the line around the slices (default: 1 pixel)
  - startangle - the angle w.r.t. positive x-axis where the first slice starts
  - direction - the direction in which to draw the slices (default: +, that is clockwise)

- The table charts use the general components *title* and *margin* and further more the specific components *header*, *oddrow*, *evenrow*, *cell* and *frame*:
  - *header*, *oddrow* and *evenrow* have the properties: *background*, *font*, *color*, *height* and *anchor* with obvious meanings.
  - The *cell* component defines in addition *leftspace*, *rightspace* and *topspace* for fine-grained control of the spacing inside the cell. These are not set via the *cellconfigure* subcommand however.
  - Finally the *frame* component uses *color*, *outerwidth* (for the width of the line surrounding the whole table) and *innerwidth* (for the width of lines separating columns and rows).

- The target diagram and the Taylor diagram use the *limits* components in addition to the various general components. The limits component has one property: the color of the circles and circle segments.

  For the Taylor diagram you can specify the color of the reference circles via the *reference* component.

::Plotchart::eraseplot *anyplot*

Erase the plot/chart with all resources connected to it.

string *anyplot* (in)

The plot/chart command. All canvas items associated with this command and all internal resources will be removed, including the plot/chart command itself.

See the examples in plotdemos7.tcl for its use.

Scrolling for timecharts and Gantt charts

For two types of plots automatic scrolling management has been implemented: timecharts and Gantt charts. The subcommands *hscroll* and *vscroll* associate (existing) scrollbars to the plot, in much the same way as for text and canvas widgets.

Once the association is made, the scrollbars are automatically updated if:

- You add an item with a period wider than the current one.

- You add a vertical line for a time beyond the current bounds.
- You add an extra item beyond the number that was used to create the chart.

For instance:

```
package require Plotchart

canvas .c -width 400 -height 200
scrollbar .y -orient vertical
scrollbar .x -orient horizontal

grid .c .y -sticky news
grid .x    -sticky news

source plotchart.tcl

set s [::Plotchart::createTimechart .c "1 january 2004"  "31 december
    2004" 4]

$s period "Spring" "1 march 2004" "1 june 2004" green
$s period "Summer" "1 june 2004" "1 september 2004" yellow
$s vertline "1 jan" "1 january 2004"
$s vertline "1 apr" "1 april 2004"
$s vertline "1 jul" "1 july 2004"
$s vertline "1 oct" "1 october 2004"
$s vertline "1 jan" "1 january 2005"
$s vertline "1 apr" "1 april 2005"
$s vertline "1 jul" "1 july 2005"
$s milestone "Longest day" "21 july 2004"
$s milestone "Longest day 2" "21 july 2004"
$s milestone "Longest day 3" "21 july 2004"
$s milestone "Longest day 4" "21 july 2004"
$s milestone "Longest day 5" "21 july 2004"
$s milestone "Longest day 6" "21 july 2004"
$s title "Seasons (northern hemisphere)"

$s vscroll .y
$s hscroll .x
```

The original extent of the chart is from 1 january 2004 to 31 december 2004. But because of the addition of vertical lines in 2005 and more items than was specified at the creation of the chart, both the horizontal and the vertical scrollbar will be enabled.

Specialised plots

Most of the plot and chart types described above have a fairly general use and you simply prepares the data to be plotted yourself. This section describes several plot types that are more specialised, in the sense that they have specific purposes and you pass raw data that are then processed in the plotting routines.

Currently there are the following types:

- Target diagrams are used to assess the capacity of numerical models to reproduce measurement data. They are described in detail in:

```
Jason K. Joliff et al.
    Summary diagrams for coupled hydrodynamic-ecosystem model skill
    assessment
    Journal of Marine Systems 76 (2009) 64-82
    DOI: 10.1016/j.jmarsys.2008.05.014
```

- Performance profiles are used for comparing the performance of numerical methods or implementations thereof with each other. For more information:

```
Desmond Higham and Nicholas Higham
    Matlab Guide
    SIAM, 2005, Philadephia
```

- Taylor diagrams are another graphical representation of how numerical models reproduce measurement data. A detailed description appears in https://en.wikipedia.org/wiki/Taylor_diagram

Most of the general methods for XY-plots work for these plots as well, but their creation and the methods to plot the data are very specific.

::Plotchart::createTargetDiagram *w limits scale*

Create a new target diagram with circles indicating specific limits. The x-axis represents the unbiased "root-mean-square difference" (typically varying between -1 and 1) and the y-axis represents the normalised bias.

Data points closer to the origin represent better results than data points further away.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *limits* (in)

List of radii for the circles that represent the limits (for instance: 0.5 and 0.7)

double *scale* (in)

Scale for the axes - defaults to 1, but if the model results are a poor fit, then that may be too small a value. Both axes are scaled in the same way.

$target plot *series xvalues yvalues*

The plot method takes two series of data of the same length, the first one representing the model results, the second one represent the measurements or, more general, the data that need to be reproduced.

string *series* (in)

Name of the series (it will be plotted as a symbol that is configured via the *$target dataconfig* command (see the XY-plot equivalent for an explanation)

list *xvalues* (in)

List of model results (missing values are represented as empty strings)

list *yvalues* (in)

List of measured values (missing values are represented as empty strings; only if both the x and the y values are given, is the pair used in the computations)

::Plotchart::createPerformanceProfile *w max*

Create a diagram to show the performance of various numerical methods (or solvers). The idea is to first run these methods on a set of problems and measure their performance. The smaller the number the better. Then these methods are compared via a so-called performance profile: the data are scaled and ordered, such that the best method ends up highest.

Because of the nature of the plot all data must be given at once.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

float *max* (in)

Maximum value for the x-axis (the x-axis is the scaled performance of the series).

$performance plot *series_and_data_pairs*

Plot the data for each given method. The data are identified by the series name and the appearance is controlled via prior dataconfig subcommand.

list *series_and_data_pairs* (in)

List of series names and data. All data must be given at once.

::Plotchart::createTaylorDiagram *w radius_data args*

Create a new Taylor diagram (one quadrant) with circles indicating the distance to the reference point.

The data points are given as the standard deviation and the correlation to the reference.

widget *w* (in)

Name of the *existing* canvas widget to hold the plot.

list *radius_data* (in)

A list of the maximum radius for the standard deviation and the step size. Quarter circles are drawn with the axis labels.

Currently one option is supported:

reference value

Reference value for the data points to be compared with. It appears as a dot at the x-axis. Along with this dot circle segments are drawn at the same spacing as the axis labels to indicate the distance of the data points from the reference.

$taylor plot *series stdev corr*

The plot method takes the standard deviation and the correlation to the reference as input and draws a symbol as a representation. The standard deviation serves as the distance from the origin and the correlation determines the angle.

string *series* (in)

Name of the series (it will be plotted as a symbol that is configured via the *$target dataconfig* command (see the XY-plot equivalent for an explanation)

float *stdev* (in)

Standard deviation of the data point

list *corr* (in)

Correlation coefficient with the reference dataset

::Plotchart::createHeatmap *w rowlabels columnlabels args*

Create a heatmap, i.e. a tableau of rectangles whose colours depend on some data. The row and column labels are used as identifiers when filling in the data. The number of them determines the size of the tableau.

The data are passed by row, by column or per individual cell.

widget *w* (in)

Name of the *existing* canvas widget to hold the heatmap.

list *rowlabels* (in)

List of labels to display at each row. The labels also serve as identifiers.

list *columnlabels* (in)

List of labels to display at each column. The labels also serve as identifiers.

list *args* (in)

Zero or more arguments to influence the size of the heatmap. See XY plot for more information.

$heatmap plot row *label data*

Use the given data to fill the rectangles belonging to the row "label".

string *label* (in)

String indicating which row to use.

list *data* (in)

List of numerical data to be used for the colouring of the rectangles.

$heatmap plot column *label data*

Use the given data to fill the rectangles belonging to the column "label".

string *label* (in)

String indicating which column to use.

list *data* (in)

List of numerical data to be used for the colouring of the rectangles.

$heatmap plot cell *rowlabel columnlabel value*

Use the given value to fill the rectangle belonging to the cell with the given row and column labels.

string *rowlabel* (in)

String indicating which row to use.

string *columnlabel* (in)

String indicating which column to use.

float *value* (in)

Value to be used for the colouring of the cell.

$heatmap scale values *min max*

Set the range for the values - they are mapped to a colour via linear interpolation.

float *min* (in)

Minimum value to be used.

float *max* (in)

Maximum value to be used.

$heatmap scale colours *mincolour maxcolour*

Set the colours to be used for the minimum and the maximum values. The actual colour is determined via linear interpolation of the RGB values.

string *mincolour* (in)

Colour to be used at the minimum value.

string *maxcolour* (in)

Colour to be used at the maximum value.

::Plotchart::createCircleplot *w labels args*

Create a circle plot, i.e. a circle with labels that can be connected by coloured arcs. Typical use: present the relationship between the items on the circle in a graphical way.

The connections can be drawn pair by pair.

widget *w* (in)

Name of the *existing* canvas widget to hold the circle plot.

list *rowlabels* (in)

List of labels to display along the circle. Each label is accompanied by a filled dot.

list *args* (in)

Zero or more arguments to influence the size of the circle plot. See XY plot for more information.

$circleplot connect *label1 label2 colour width*

Connect the two labels via a coloured arc of given width (the arc is actually a parabola).

string *label1* (in)

String indicating the first label to connect.

string *label2* (in)

String indicating the second label to connect.

string *colour* (in)

Colour to use for the connected arc.

string *width* (in)

Width for the connected arc.

$circleplot modify *label args*

Modify the appearance of the label and the accompanying dot.

string *label* (in)

String indicating which label to modify.

list *args* (in)

List of key-value pairs:
- *-textcolour colour* - colour of the text to be used (alternative: "-textcolor")
- *-font font* - font for the text
- *-dotcolour colour* - colour for the dot (alternative: "-dotcolor")

Adding specific plot methods

The command *plotmethod* can be used to add new methods for a particular plot or chart type. It is intended to help you develop specialised graphical displays.

::Plotchart::plotmethod *charttype methodname plotproc*

Adds a new method for the given plot or chart type. The method is implemented by the command or procedure given in the plotproc argument. The procedure will be called with two extra arguments, the name of the created plot and the canvas widget that contains (see the example below).

string *charttype* (in)
The type of plot or chart that the new method should be added to.

string *methodname* (in)
Name of the method to be used.

string *plotproc* (in)
Name of the command or procedure that implements the method.

Here is a trivial example of how to use this:

```
#
# The custom method "doodle" always adds the text "DOODLE"
# to the plot
#
proc doodle {p w x y} {
    $p plaintext $x $y "DOODLE"
}
::Plotchart::plotmethod xyplot doodle doodle

#
# Use it
pack [canvas .c]

set p [::Plotchart::createXYPlot .c {0 100 10} {0 20 5}]

$p doodle 40 10
```

Table charts

To show what you can do with table charts, here is a simple example that plots a number of random data. The colours depend on the range that the data belong to. For this the procedure *setColor* is used.

```
package require Plotchart

pack [canvas .c -bg white -height 300] -fill both -expand yes

::Plotchart::plotconfig table frame outerwidth 3
::Plotchart::plotconfig table frame color red

set t [::Plotchart::createTableChart .c {"Column 1" "Column 2" "Column
    3"} 80]


proc setColor {table widget row col value} {
```

```
    $table cellconfigure -background white -color black
    if { $value < 2.0 } {
        $table cellconfigure -background red -color white
    }
    if { $value > 6.0 } {
        $table cellconfigure -background green
    }

    return [format "%6.3f" $value]
}

# Command must already exist ...
$t formatcommand setColor

$t title "Demonstration of table charts"
$t separator

for {set i 0} {$i < 9} {incr i} {
    set row {}

    for {set j 0} {$j < 3} {incr j} {
        lappend row [expr {10.0 * rand()}]
    }

    if { $i == 3 } {
        $t separator
    }

    $t row $row
}
```

Control displays

TODO

Using date/time labels

The options -timeformat and -gmt are used to control the display of date/time labels along the x-axis for those plot types for which it makes sense. These options were implemented to take care of date/time labels for stripcharts, as you can also use custom labels (the option -xlabels) if the axis is "static". Since this is not the case for stripcharts, this was not an option (Tcllib/Tklib bug 3613718). You can also specify the -locale option to get a localised date/time label. The example below illustrates how to use the -timeformat option. The -gmt option merely suppresses the handling of daylight saving time by the [clock format] command.

```
package require Plotchart

pack [canvas .c -width 500 -bg white]

#
# Note that we need to present the x values as clock seconds
#
set start [clock scan  "0:00"]
set stop  [clock scan "10:00"]
set s [Plotchart::createStripchart .c [list $start $stop 7200] {0 10
    1} -timeformat "%H:%M"]

# With a locale like "de":
#set s [Plotchart::createStripchart .c [list $start $stop 7200] {0 10
    1} -timeformat "%H:%M" -locale de]

foreach {x y} {0 0 2 5 5 2 9 9 12 10} {
    set x [expr {$start + 3600 * $x}] ;# Convert hour to clock seconds
    $s plot a $x $y
}
```

The *plot* subcommand simply interprets the x and y data as straightforward numbers, so that you need to do the conversion from date/time to "clock seconds" yourself.

Arranging multiple plots in a canvas

The command *plotpack* allows you to copy the contents of a plot into another canvas widget. This canvas widget does not act as a composite plot, but it can be saved as a PostScript file for instance: Note: the command simply takes a snapshot of the plots/charts as they are at that moment.

::Plotchart::plotpack *w dir args*

Copy the contents of the plots/charts into another widget, in a manner similar to the *pack* geometry manager.

widget *w* (in)

The name of the canvas widget to copy the plots/charts into

string *dir* (in)

The direction of the arrangement - top, left, bottom or right

list *args* (in)

List of plots/charts to be copied.

For example:

```
    set p1 [createXYPlot ...]
    set p2 [createBarchart ...]

    ... fill the plots ...
```

```
    toplevel .t
    pack [canvas .t.c2 -width ...]

    #
    # Copy the two plots above each other in the new canvas
    #
    plotpack .t.c2 top $p1 $p2
```

A different method is to use the *-box* and *-axesbox* options when creating the plot. These control the area in the canvas where the plot or chart will be drawn.

The *-box* option takes as its value a list of four numbers:

- X-coordinate of the upper-left corner of the area that will contain the plot or chart (simply a canvas coordinate)
- Y-coordinate of the upper-left corner
- Width of the area
- Height of the area

Specifying the width and height makes it easier to reposition the area with respect to other plots.

The *-axesbox* option is meant to make aligning the axes of a plot with those of other plots easier. The option takes a list of six arguments:

- Identification of the plot with respect to which it should be positioned (the command returned by the creation command).
- The anchor position that should be used (n, nw, ...)
- X-coordinate of the upper-left corner of the area that will contain the plot or chart. This coordinates is taken relative to the *anchor position*
- Y-coordinate of the upper-left corner
- Width of the axis area
- Height of the axis area

With this option the area the axes occupy is first determined and the complete area is derived from the margins.

For example:

```
    set p2 [::Plotchart::createXYPlot .c {0 10 1} {-5 5 2.5} -axesbox
    [list $p1 ne 0 0 200 200]]
```

will create a second plot whose left axis coincides with the right axis of plot "\$p1" and the top of the axis is at the same heigt as well - because the axes are positioned at a point 0 pixels to the left and 0 pixels below the north-east corner.

Interactive use

Plotchart has several features for interactive use (cf. Notes on tags):

- The legend can be moved around by pressing mouse button 1 in the legend's box and keeping it down.
- You can use the *bindplot* and *bindlast* commands to define actions that are to be taken when the user clicks on an element of the plot or chart. (see below, see also the sample code in

plotdemos12.tcl)
- *Piecharts* can show an "exploded" segment that you can select with mouse button 1.

If you require different forms of interaction, not covered by *Plotchart* itself, you can use the tags on the various canvas elements to define other bindings.

The *bindplot* and *bindlast* are defined as follows:

$anyplot bindplot *event command args*
Register a command that will be run whenever the given event occurs in the plot.
string *event*
The event that you want to bind the command to
string *command*
Name of the command/procedure that you want to run. The following arguments are prefixed: the x- and y-coordinates of the point in the plot (the world coordinates!), so that the procedure has the signature:

```
    cmd $xworld $yworld $string1 $string2 $string3
```

assuming the argument "command" is: {cmd A B C}

$anyplot bindlast *series event command*
Register a command that will be run when the event occurs within the neighbourhood of the last point added to the given series. (You can use directly after inserting a data point. All such commands will remain active).
string *event*
The event that you want to bind the command to
list *command*
Name of the command/procedure that you want to run. The following arguments are prefixed: the x- and y-coordinates of the point in the plot (the world coordinates!), so that the procedure has the signature:

```
    cmd $xworld $yworld $string1 $string2 $string3
```

assuming the argument "command" is: {cmd A B C}

Here is an example - show the values of the data points in an annotation (from the sample code in plotdemos12.tcl):

```
#
# Procedure for showing an annotation
#
proc showAnnotation {xcoord ycoord plot w} {

    $plot balloon $xcoord $ycoord "Data point: [format "%.3f, %.3f"
    $xcoord $ycoord]" north

    after 2000 [list removeAnnotation $w]
}
```

```
#
# Procedure for erase an annotation
#
proc removeAnnotation {w} {

    # Use the tags to remove all annotations
    $w delete BalloonText
    $w delete BalloonFrame
}

#
# Create a simple plot and a label
#
pack [canvas .c -bg white] [label .l -textvariable coords]

set p [::Plotchart::createXYPlot .c {0 1000 200} {0 10 1}]

$p dataconfig series1 -type both -symbol cross

foreach x {1 2 5 10 20 50 100 200 500 1000} {
    $p plot series1 $x [expr {log($x)}]

    #
    # Show the annotation for each data point
    #
    $p bindlast series1 <Enter> [list showAnnotation $p %W]
}
```

Notes on tags

The implementation of *Plotchart* relies heavily on the canvas's ability to identify graphical objects by tags and to change the drawing order of the objects. This section documents the tags that are used.

(*Note:* the tags are not always used consistently - see the notes appearing with the various tags. This section describes the current state.)

General graphical objects:

- *mask* - Used to manipulate the opaque rectangles that ensure data outside the viewport are not shown.
- *topmask, horizmask, vertmask* - specialised tags, used for scrollable plots.
- *title* - Used for title strings.
- *BalloonText, BalloonFrame* - Used to manipulate balloon text.
- *PlainText* - Used to manipulate ordinary text without any decoration.
- *background* - Tag used for gradient and image backgrounds (and for gradient-filled bars).
- *xaxis, yaxis* - Tags used for all objects related to horizontal or vertical axes. (also: both for numerical axes and axes with labels as in barcharts). Note, however, that the *text* along the axes has no particular tag.

- *raxis* - Tag used for all objects related to a *right* axis.
- *taxis* - Tag used for all objects related to a *time* axis.
- *axis3d* - Tag used for 3D axes
- *xtickline, ytickline* - Tags used for ticklines.
- *legend, legengb, legendobj* - Tags used for the legend. The latter is used to manipulate the legend as a whole.
- *legend_series* - Tag used to control the appearance of the legend entry ("series" should be replaced by the series name).
- *object* - used as standard tag for all objects drawn with the **::Plotchart::drawobject** procedure. Tags given at object creation time are added to this tag.

XY-plots (all types of axes):

- *data* - The general tag to identify graphical objects associated with data. *data_seriesname* - The tag specific to a data series ("seriesname" should be replaced). *band* - The horizontal or vertical band drawn with the xband otr yband subcommands have this tag by the actual name). *xtext* - The text labelling the xaxis. *ytext* - The text labelling hte yaxis horizontically. *vtext* - The text labelling the yaxis vertically.

Items such as labelled dots only have the "data" tag.

Piecharts and spiral pies:

- *segment_segmentnumber* - The tag identifying the segment, the string "segmentnumber" should be replaced by the actual number. This tag is used to explode the segments.

Barcharts:

Barcharts use the same tags as xy-plots (but for gradient-filled bars the data_seriesname is not used).

Histograms and isometric plots:

Currently the only tag used is "data".

Time-charts:

As these plots are scrollable, several tags are used specific to the scrolling: vertscroll, horizscroll, below, lowest, above, timeline, tline. Each item also has a tag of the form "item_number", where "number" is to be replaced by the actual sequence number of the item.

Gantt charts:

In addition to the tags described for the time-charts, the following tags are used: description, completed, summary and summarybar.

Radial charts and polar plots:

Currently the radial lines indicating the grid have no tags. The graphical objects associated with data only have the "data" tag.

Windroses:

Only the tag *data_number* is currently used ("number" should be replaced by the sequence number of the data, starting at 0.

Contour and isoline plots:

No tags are used.

3D plots and 3D ribbon plots:

Tags are used for the axes and for the data objects:

- *data* - The general tag to identify graphical objects associated with data. *line* - The tag used for lines created with the plotline subcommand.

Charts decorated with 3D effects:

The following tags are used to identify various types of graphical objects: platform, background, d, u, ticklines.

The text associated with the bars has no tags. The ribbon lines and areas have no tags either.

Tables:

Tags used are: frame, cellbg and celltext

Special plot types (target diagrams, Taylor diagrams:

Tags used are: limits, limit_labels, reference *In addition:* To implement multiple plots and charts in a single canvas, all items also get as a tag the plot/chart they belong to. This enables Plotchart to manipulate only those items.

TODO - some private notes

I have the following wishlist:

- Isometric plots - allow new items to be implemented easily.
- A general 3D viewer - emphasis on geometry, not a ray-tracer.
- Several improvements for boxplots:
  - Height of the box scales with the logarithm of the number of points
  - Marker line to indicate a "current" value
  - Box drawn from quantiles

<

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *plotchart* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation. <

Copyright

```
Copyright (c) 2022 Arjen Markus <arjenmarkus@users.sourceforge.net>
```

shtmlview

shtmlview-doctools

Keywords: shtmlview-doctools, doctools, html

Name

shtmlview_doctools - shtmlview extension adding support for doctools markup

Synopsis

package require dtplite
package require shtmlview::shtmlview

Description

The **shtmlview::doctools** package extends **shtmlview::shtmlview** widgets with support for rendering of *doctools*-formatted documents.

Simply loading the package registers the support.

The package does not export any kind of command, class, etc.

Changelog

2022-04-07 version 0.1

- Initial implementation

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such to the author of this package. Please also report any ideas for enhancements you may have for either package and/or documentation.

Code copyright

BSD License type:

Sun Microsystems, Inc. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

In no event shall the authors or distributors be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software, its documentation, or any derivatives thereof, even if the authors have been advised of the possibility of such damage.

The authors and distributors specifically disclaim any warranties, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. This software is provided on an "as is" basis, and the authors and distributors have no obligation to provide maintenance, support, updates, enhancements, or modifications.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Copyright

```
Copyright (c) 2022, Detlef Groth <detlef(at)dgroth(dot)de>
```

shtmlview-mkdoc

Keywords: shtmlview-mkdoc, html, mkdoc

Name

shtmlview_mkdoc - shtmlview extension adding support for mkdoc markup

Synopsis

package require mkdoc::mkdoc
package require shtmlview::shtmlview

Description

The **shtmlview::mkdoc** package extends **shtmlview::shtmlview** widgets with support for rendering of *mkdoc*-formatted documents.

Simply loading the package registers the support.

The package does not export any kind of command, class, etc.

Changelog

2022-04-07 version 0.1

- Initial implementation

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such to the author of this package. Please also report any ideas for enhancements you may have for either package and/or documentation.

Code copyright

BSD License type:

Sun Microsystems, Inc. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

In no event shall the authors or distributors be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software, its documentation, or any derivatives thereof, even if the authors have been advised of the possibility of such damage.

The authors and distributors specifically disclaim any warranties, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. This software is provided on an "as is" basis, and the authors and distributors have no obligation to provide maintenance, support, updates, enhancements, or modifications.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Copyright

```
Copyright (c) 2022, Detlef Groth <detlef(at)dgroth(dot)de>
```

shtmlview

Keywords: shtmlview, html, text, widget

Name

shtmlview - Extended Tcl/Tk text widget with basic support for rendering of HTML and Markdown

Synopsis

package require Tk
package require snit
package require Markdown ;# optional Markdown support
package require img::jpeg ;# optional jpeg image support
package require shtmlview::shtmlview ?**1.1.2**?
::shtmlview::converter *extension description cmdprefix*
::shtmlview::shtmlview *pathName* ?*options*?
pathName **back**
pathName **browse** *filename* ?*args*?
pathName **dosearch** *string direction*
pathName **edittext** *cmd* ?*options*?
pathName **editView**
pathName **forward**
pathName **getFiles**
pathName **getHistory**
pathName **getEditWidget**
pathName **getTextWidget**
pathName **helptext** *cmd* ?*options*?
pathName **home**
pathName **open**
pathName **reload**
pathName **render** *text* ?*ext*?
pathName **sourceView**
pathName **url**

Description

The **shtmlview::shtmlview** package provides a pure Tcl/Tk widget of the same name to render and display basic HTML and Markdown files and string.

Some history: The widget is based on the **htmllib** library developed in the 90ties by Stephen Uhler and Clif Flynt. In 2009 Robert Heller wrapped this library into the excellent mega-widget framework **snit**. The resulting widget was however tied directly into a help system. The author of this document first just isolated the display part and then added some functions such as changing font size and a few buttons in the toolbar. Then a rudimentary display of data tables was added. Even later support for inline images and extended keybindings and Markdown support was added.

The widget is *not* a web browser. It only supports relative links in the local filesystem. It does *not*

support style sheets. It does not support http(s) links nor images. It is thought of as a last fallback to use in cases where no other possibilities exists to display HTML or Markdown markup from a Tk application.

Use cases are as a help viewer and wherever the developer has control about the used html tags.

Comments and feedbacks are welcome.

The **shtmlview::shtmlview** widget overloads the text widget and provides new commands and options. These are explained in the sections Widget commands and Widget options.

Note that the file "*shtmlview.tcl*" is not only a package but also a standalone application for the direct viewing of Markdown and HTML files. Invoke it as

```
tclsh shtmlview.tcl filename.html
```

in a terminal.

The API described in this document is not the whole API offered by the snit object **::shtmlview::shtmlview**. Instead, it is the subset of that API that is expected not to change in future versions.

Commands

::shtmlview::converter *extension description cmdprefix*

Registers a conversion command prefix (*cmdprefix*) for files having the *extension*. The *description* is a short summary of the kind of files expected to have that extension.

The result of the command is the empty string.

Whenever an shtmlview widget encounters a file with the specified *extension* it will invoke the registered command prefix with one argument, the path to the file to convert. The result of the invokation is expected to be the HTML to render and display.

See section Extensions for an example.

::shtmlview::shtmlview *pathName* ?*options*?

Creates and configures the shtmlview widget *pathName*.

The result of the command is the *pathName*.

An error is thrown if a widget for *pathName* already exists.

An error is also thrown if the parent for *pathName* does not exist.

The recognized options are explained in section Widget options.

The methods of the new widget are explained in section Widget commands.

Widget options

Use method **helptext** to configure the internal text widget.

-browsecmd cmd

Each time a HTML or Markdown page is rendered the specified *cmd* command is invoked. The actual URL is appended as first argument to the command.

-closebutton boolean
If **true** (default) a close button is shown in the toolbar. This is useful if the widget is the sole child of a toplevel widget.

-historycombo boolean
If **true**, and the **tile** package is available, then a **ttk::combobox** to show page history is added to the toolbar. Defaults to **false**. Can be only set at widget creation.

-home string
Sets the homepage filename of the widget. If not set, the first page browsed to will be automatically set as the home page.

-tablesupport boolean
If **true** (default) some table support is provided. This will have some undesired results if table tags are misused as markup tool. Simple html tables with **th** and **td** tags should however display well if no internal markup inside of those tags is used.

-toolbar boolean
If **true** (default) a toolbar will be displayed at the top. The toolbar will provide standard buttons for the methods **back**, **forward** and **home**, as well as search facilities. Defaults to **true**.

Widget commands

Each shtmlview widget created with the above command supports the following commands and options:

***pathName* back**
Displays the previous HTML and Markdown page in the browsing history if any.

***pathName* browse *filename* ?args?**
Displays the HTML or Markdown text contained in the named file. Any additional arguments, while also file names, are just added to the history stack. They can be walked to using the history keys, **f** and **b**.

pathName* dosearch *string direction
Search for and highlight the given string starting from the current index in the specified direction. The direction has to be either **forward** (default) or **backwards**.

***pathName* edittext *cmd* ?*options*?**
This command exposes the internal text editor widget for configuration. See the following example:

```
::shtmlview::shtmlview .help
.help browse index.md
.help editView
.help edittext configure -background salmon
```

***pathName* editView**
This command switches the widget from viewing to editing. In this mode the user is able to edit and change the currently loaded file. To switch to a non-editable source display instead see **sourceView** below.

```
    ::shtmlview::shtmlview .help
    .help browse index.md
    .help editView
```

***pathName* forward**
Displays the next HTML or Markdown page in the browsing history if any.

***pathName* getFiles**
This command returns a list of all visited files. Duplicates and anchor links are removed from the raw data.

***pathName* getHistory**
This command returns a list of the current history of visited files and anchors.

***pathName* getEditWidget**
This commands returns the internal pathname of the text editor widget used for editing the current document. This enables the developer to directly access it, if required or desired. This is dangerous. See also **edittext**, above.

***pathName* getTextWidget**
This commands returns the internal pathname of the internal viewer text widget. This enables the developer to directly access it, if required or desired. This is dangerous. See also **helptext** , below.

***pathName* helptext *cmd* ?*options*?**
This command exposes the internal viewer text widget for configuration. See the following example:

```
    ::shtmlview::shtmlview .help
    .help browse index.html
    .help helptext configure -background yellow
```

***pathName* home**
Displays either the page set by option **-home**, or the first page **browse** was called for.

***pathName* open**
Uses a standard file open dialog to select a document in any of the supported formats for display, and then renders that file, if any.

***pathName* reload**
Reloads and redisplays the currently shown page.

***pathName* render *text* ?ext?**
Renders the given *text* in the viewer. If an extension *ext* is specified the string is assumed to be in the associated format, and the associated converter is used. Otherwise the string is considered to be either HTML or Markdown. To be treated as HTML the *text* has to start with a recognized HTML tag. In any other case it is considered to be Markdown.

***pathName* sourceView**
This command switches the widget from viewing the current document itself to viewing the source of that document. To switch to a editable source display see **editView** above.

```
    ::shtmlview::shtmlview .help
    .help browse index.md
    .help sourceView
```

***pathName* url**
Returns the currently shown URL.

Bindings for the widget

The following keys are bound to the widget for navigation and other actions:

```
<cursor>
: Standard cursor movement in the view
```

b
Back - display previous page in history

f
Forward - display next page in history

p
Previous - Move cursor to previous search match

n
Next - Move cursor to next search match

q
Remove current page from history. Implies **f**.

r
Start search backward

s
Start search forward

Ctrl-E
Toggle edit mode

Ctrl-s
In edit mode, save page

Ctrl-r
Reload current page

Ctrl-u
Toggle source vs rendered views

Return
Follow link under cursor

TAB
Move cursor to next link on current page

Example

```
    package require shtmlview::shtmlview
    proc browsed {url} {
        puts "You browsed $url"
    }
    ::shtmlview::shtmlview .help -toolbar true -browsecmd browsed
    .help browse index.html
    pack .help -fill both -expand true -side left
    package require Markdown
    .help browser test.md
```

More examples can be found in the sources of the package.

Extensions

While the package natively only support HTML documents, and Markdown documents if the supporting **Markdown** package is available, it is possible to extend the range of supported formats by means of a plugin mechanism.

The main entry point to that system is the **shtmlview::converter** command. With it is possible to register a document format and an associated conversion command. The format is identified by its file extension, like, for example "*.md*", "*.man*", etc. The conversion command is expected to convert the content of the file given to it into HTML for display.

The packages **shtmlview::doctools** and **shtmlview::mkdoc** are examples of such plugins. The first provides support for the *doctools* format used by both *Tcllib* and *Tklib* for their manpages, while the second provides support for *mkdoc*-enhanced Tcl source files. In other words, Tcl files with embedded documentation in *mkdoc* syntax.

Enclosed below the bare Tcl code of the **shtmlview::mkdoc** package:

```
    package require shtmlview::shtmlview
    package require mkdoc::mkdoc

    ::shtmlview::converter .tcl {Tcl+mkdoc files}   ::shtmlview::mkdoc
    ::shtmlview::converter .tm  {Tcl+mkdoc modules} ::shtmlview::mkdoc

    proc ::shtmlview::mkdoc {url} {
    close [file tempfile htmltemp .html]

    mkdoc::mkdoc $url $htmltemp -html

    if {[catch {
        open $htmltemp r
    } result]} {
        return -code error "Cannot open $url: $result"
    }

    set html [read $result]
    close $result
    file delete $htmltemp
```

```
    return $html
    }

    package provide shtmlview::mkdoc 0.1
```

It is of course possible to write plugins which use an external application like **pandoc** to generate the HTML to render, instead of a Tcl package.

And it is of course also possible to register conversion commands directly from the application using this package, instead of going through a separate package.

Changelog

2022-02-25 version 0.9.2

- Fix for **tk_textCopy** and documentation update

2022-03-06 version 0.9.3

- Support for MouseWheel bindings
- Fixing hyperlinks to http(s) links
- Support for file-anchor links like **file.html#anchor**
- Support for **#** as link to the top
- Thanks to aplsimple for suggestions and bug reports

2022-03-26 version 1.0.0

- HTML 3.2 tags div, sub, sup, small, big
- Initial support for Markdown files
- Initial support for base64 encoded inline image files
- Support for JPEG images if the **img::jpeg** package is available
- Support for SVG images if either **critcl** and *librsvg-dev(el)* or terminal application **rsvg-convert** or **cairosvg** are available
- Back and forward as well for anchors
- First and last browse entry buttons for history
- History with full file path to allow directory changes
- Improved usage line and install option
- Keyboard bindings for next and previous search
- Return and tab for links
- Historycombo option
- Toolbar fix
- Browse fix for non-existing files
- Removed unused css/stylesheet and web forms code
- Thanks to pepdiz for bug-reports and suggestions

2022-04-XX version 1.1.0

- Keybinding Ctrl-u to source view for HTML and Markdown files
- Keybinding Ctrl-Shift-e to use a simple text editor for file editing
- File open dialog now remembers the last directory and the last file extension
- File close button added with option -closebutton, useful for toplevel windows
- Some backslash fixes for Markdown links
- Adding span tag for styling

- Adding render method to read HTML directly without filename
- Adding plugin structure for additional file types like Tcllib doctools, or mkdoc
- Tcl doctools support resides in its own package
- Tcl mkdoc supports resides in its own package

Todo

- Markdown rendering using tcllib package **Markdown** in case an URL ends with "*.md*" (done)
- Support for SVG images for instance using svgconvert, at least on Linux/Windows
- More tags, see tag history add 3.2 tags: http://www.martinrinehart.com/frontend-engineering/engineers/html/html-tag-history.html (done)
- Source view using Ctrl-u (done)
- Edit view using Ctrl-Shift-e
- Fixing mouse wheel issues

Thanks

Stephen Uhler, Clif Flynt and Robert Heller, they provided the majority of the code in this widget.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such to the author of this package. Please also report any ideas for enhancements you may have for either package and/or documentation.

Code copyright

BSD License type:

Sun Microsystems, Inc. The following terms apply to all files a ssociated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

In no event shall the authors or distributors be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software, its documentation, or any derivatives thereof, even if the authors have been advised of the possibility of such damage.

The authors and distributors specifically disclaim any warranties, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. This software is provided on an "as is" basis, and the authors and distributors have no obligation to provide maintenance, support, updates, enhancements, or modifications.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Copyright

```
Copyright (c) 2018-2022, Detlef Groth <detlef(at)dgroth(dot)de>
Copyright (c) 2009, Robert Heller
Copyright (c) 2000, Clif Flynt
Copyright (c) 1995-1999, Stephen Uhler
```

swaplist

swaplist

Keywords: swaplist, dialog, disjointlistbox, listbox

Name

swaplist - A dialog which allows a user to move options between two lists

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require swaplist ?**0.1**?
::swaplist::swaplist *pathName variable completeList selectedList* ?*options*?

Description

This package provides a dialog which consists of 2 listboxes, along with buttons to move items between them and reorder the right list.

::swaplist::swaplist *pathName variable completeList selectedList* ?options?

Creates a dialog which presents the user with a pair of listboxes. Items are selected by using the buttons to move them to the right list. The contents of the right list are put in the *variable* upon closure of the dialog. The command returns a boolean indicating if the user pressed OK or not. If -geometry is not specified, the dialog is centered in its parent toplevel unless its parent is. in which case the dialog is centered in the screen.

Options:

-embed
: if this flag is supplied, the procedure will create a swaplist widget named *pathName*, with the *variable* set as the listvariable for the right side listbox. This flag will also cause the -title and -geometry flags to be ignored.

-reorder
: boolean specifying if buttons allowing the user to change the order of the right listbox should appear or not. defaults to true

-title
: sets the title of the dialog window. defaults to "Configuration"

-llabel
: sets the heading above the left list. defaults to "Available:"

-rlabel
: sets the heading above the right list. defaults to "Selected:"

-lbuttontext

sets the text on the "move left" button. defaults to "<<"

-rbuttontext
sets the text on the "move right" button. defaults to ">>"

-ubuttontext
sets the text on the "move up" button. defaults to "Move Up"

-dbuttontext
sets the text on the "move down" button. defaults to "Move Down"

-geometry
sets the geometry of the dialog window.

Example

```
package require swaplist
namespace import swaplist::*

if {[swaplist .slist opts "1 2 3 4 5 6 7 8 9" "1 3 5"]} {
    puts "user chose numbers: $opts"
}
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *swaplist* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Widget

tkpiechart

canvaslabel

Keywords: canvaslabel, canvas, labeler, pie, slice

Name

canvasLabel - tkpiechart canvas label class

Synopsis

package require stooop 4.1
package require switched 2.2
package require tkpiechart 6.6
stooop::new canvasLabel *canvas* ?*options*?
switched::configure *canvasLabelObject* ?*options*?
switched::cget *canvasLabelObject option*
stooop::delete *canvasLabelObject*

Description

The canvasLabel class brings some Tk label widget functionality to the canvas text item, such as

a background and a border.

The canvasLabel is built with a bullet rectangle on the left side of the text. The relief changes according to the select state, with a traditionally sunken relief when selected.

The label has a specific tag, which can be used to retrieve the coordinates of the object or move it, thanks to the canvas facilities.

stooop::new canvasLabel *canvas* ?options?
Creates a canvasLabel object in the specified Tk canvas. The canvasLabel object identifier is returned (referred to as *canvasLabelObject* in this document).

switched::configure *canvasLabelObject* ?options?
Configures a canvasLabel object or returns all the options with their current values if no options are passed as parameters.

switched::cget *canvasLabelObject option*
Returns an option value for the specified canvasLabel object.

stooop::delete *canvasLabelObject*
Deletes the specified canvasLabel object.

Options

-anchor value
Specifies the anchor position of the rectangle and the text, relative to the positioning point. The behavior is similar to the **-anchor** option of the **canvas** *text* item, except that the rectangle is taken into account. The default is *center*.

-background color
Specifies the background color of the bullet rectangle, as in the **-fill** option of the **canvas** *rectangle* item. The default is transparent (empty string).

-bordercolor color
Specifies the border color of the rectangle, as in the **-outline** option of the **canvas** *rectangle* item. The default is black.

-borderwidth value
Specifies the border width of the rectangle, as in the **-width** option of the **canvas** *rectangle* item. By default, the width is 1 pixel, which is the minimum width.

-bulletwidth value
Specifies the width of the rectangle placed to the left of the text. Defaults to *10*.

-font value
Specifies the font of the text, as in the **-font** option of the **canvas** *text* item. The default is system dependent.

-foreground color
Specifies the color of the text, as in the **-fill** option of the **canvas** *text* item. The default is black.

-justify value
Specifies how to justify the text, as in the **-justify** option of the **canvas** *text* item. The default is *left*.

-minimumwidth value
The total label width will not go below the specified value, but may be larger if the label text requires it.

-padding value
Specifies how much space to leave between the text and the closest rectangle edge. Units are identical to those specified in the **canvas** *COORDINATES* manual section.

-scale list
List of 2 floating point numbers used to set the scaling factor in the x and y axis. Scaling is applied immediately and defaults to 1.

-select boolean
Sets the label state.

-selectrelief value
Either *flat*, *raised* or *sunken*. Specifies the 3D effect desired for the text area when the label is selected.

-stipple bitmap
Specifies the stipple pattern filling the rectangle, as in the **-stipple** option of the **canvas** *rectangle* item. There is no bitmap by default.

-text text
Specifies the string to be displayed in the text area, as in the **-text** option of the **canvas** *text* item. The default is an empty string.

-textbackground color
Specifies the color of the text area background.

-width value
Specifies a maximum line length for the text, as in the **-width** option of the **canvas** *text* item. The default is *0*.

Tags

The labeler has the following specific tag (see the **canvas** manual page *ITEM IDS AND TAGS* section for more information):

- canvasLabel(canvasLabelObject)

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tkpiechart* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```


pie

Keywords: pie, canvas, labeler, slice

Name

pie - 2D or 3D pie chart object in a canvas

Synopsis

package require stooop 4.1
package require switched 2.2

package require tkpiechart 6.6
stooop::new pie *canvas x y* ?*options*?
switched::configure *pieObject* ?*options*?
switched::cget *pieObject option*
stooop::delete *pieObject*

pie::newSlice *pieObject* ?labelText?

pie::deleteSlice *pieObject sliceObject*

pie::sizeSlice *pieObject sliceObject unitShare* ?displayedValue?

pie::labelSlice *pieObject sliceObject string*

pie::selectedSlices *pieObject*

Description

A pie object is used to visualize a set of values, usually as shares of a total. Each value is represented by a colored slice, which may have a 2 dimensional or 3 dimensional look. Each slice is associated with a label displaying the data name, and a numerical field showing the percentage taken by the slice. The labels are placed by the chosen labeler object (**-labeler** option). Each label color matches its related slice.

A pie chart is made of Tk canvas items, found in **pieBoxLabeler**, **piePeripheralLabeler** and **canvasLabel** objects, that compose the pie object. The pie constructor creates the pie itself and its background slice within the parent canvas. Once the pie object exists, slices can be created and resized. At the time the pie is created, the parent Tk **canvas** widget must exist.

Slice colors are automatically generated, using a default color list for all pies, unless another list is used (using the **-colors** option). When a 3D look is used, the slice edge is darker than its top while using the same color tone.

stooop::new pie *canvas x y* ?options?
Creates a pie object in the specified Tk canvas. The upper left corner of the pie is placed at the specified coordinates in the canvas. The pie object identifier is returned (referred to as *pieObject* in this document).

switched::configure *pieObject* ?options?
Configures a pie object or returns all the options with their current values if no options are passed as parameters.

switched::cget *pieObject option*
Returns an option value for the specified pie object.

stooop::delete *pieObject*
Deletes the specified pie object.

Overview

The pie class is part of the tkpiechart extension that allows the programmer to create and dynamically update 2D or 3D pie charts in a Tcl/Tk application. The tkpiechart package is written in Tcl only, using object oriented techniques thanks to the stooop package, included in tcllib.

Options

-autoupdate boolean
Boolean value specifying whether all the slices and their labels are redrawn when a slice size

is changed. On by default. Turn it off and invoke **pie::update** if you change many slices at once and want to improve performance.

-background color

Slices may or may not fill up the 100% of the pie. The unoccupied part of the pie is a slice that takes 100% of the pie. It is by default transparent with a black border. The color of this background slice may be set by the user using color names as in the **-background** standard option (see the Tk **options** manual page for more details). When the pie has a 3D look, the background of a slice edge is darker than the top and uses the same color tone.

-colors list

Specifies a list of colors for slices. In this case, the slice colors will successively be drawn from the list in the list order, cycling through if there are more slices than colors in the list. Colors are specified in the same format as the **-background** option.

-height value

Specifies the total height for the pie, including the room taken by the labeler labels. The pie slices are resized when labels are added or deleted (when adding or deleting slices) so that the total height remains constant. This value may be specified in any of the forms described in the **canvas** *COORDINATES* manual section.

-labeler object

Specifies a placer object for the slice labels, so that, for example, slice values may be placed next to them. If not specified, the *pieBoxLabeler* (see corresponding manual) is used, the other option being the *piePeripheralLabeler* class. Each labeler has a specific behavior which may be set via its options. The labeler object is automatically deleted when the pie object is itself deleted. The labeler cannot be changed once the pie is created.

-selectable boolean

Boolean value specifying whether slices are selectable or not. Acceptable values are those defined by the Tcl language itself for boolean values. If selectable, slices can be selected with the first mouse button, by clicking on either the slice or its label. Selection can be extended by using the classical *control* or *shift* clicks. The list of currently selected slices can be retrieved at any time using the **selectedSlices** pie class member procedure.

-title text

Title text to be placed above the pie.

-titlefont value

Font for the title text.

-titleoffset value

Distance between the bottom of the title text and the top of the pie slices. This value may be specified in any of the forms described in the sizes section below.

-thickness value

The thickness is set to 0 by default, giving the pie a simple 2D shape, much faster to display. A positive thickness value will give the pie a 3D look with matched darker colors for the slices edges. These values may be specified in any of the forms described in the *SIZES* section below.

-width value

Specifies the total width for the pie, including the room taken by the labeler labels. The pie slices are resized when labels are added or deleted (when adding or deleting slices) so that the total width remains constant. This value may be specified in any of the forms described in

the **canvas** *COORDINATES* manual section.

Member procedures

pie::newSlice *pieObject* ?labelText?
Creates a slice. A unique object identifier is returned (referred to as *sliceObject* in this document). The slice color is automatically allocated and the slice label placed using the specified labeler (using the **-labeler** option). The slice itself is placed after (clockwise) the existing slices. The slice object identifier will be used for sizing and resizing the slice.

If the label text is not specified, it will be set to "*slice n*", *n* being the number of the slice in the order of creation (first slice is number 1).

pie::deleteSlice *pieObject sliceObject*
Deletes a slice. The following slices (clockwise) if any are then moved to compensate for the empty space left by the deleted slice.

pie::sizeSlice *pieObject sliceObject unitShare* ?displayedValue?
Sizes or resizes a slice. The slice is then automatically recalculated so it occupies the proper share of the whole pie. The *unitShare* parameter is a floating point number expressed in share (between 0 and 1) of the whole pie. The following slices (clockwise) are moved to accommodate the new slice size. The slice size value next to the slice label is also updated with the new share value or *displayedValue* if specified.

pie::labelSlice *pieObject sliceObject string*
Updates a slice label. Can be invoked at any time.

pie::selectedSlices *pieObject*
Returns a list of currently selected slice objects.

Tags

The whole pie, the pie graphics (all slices), and each slice have the following specific tags:

- **pie(pieObject)**
- **pieSlices(pieObject)**
- **slice(sliceObject)**

For example, the whole pie can be moved using the **canvas move** command on the pie tag, or bindings on slices can be set using the slice tags (see the **canvas** manual page *ITEM IDS AND TAGS* section for more information).

Sizes

All sizes related to pies are stored as floating point numbers. The coordinates and sizes are specified in screen units, which are floating point numbers optionally followed by one of several letters as specified in the **canvas** *COORDINATES* manual section.

Limitations

If the number of slices is too big, identical colors will be used for some of the slices. You may set your own colors in this case.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tkpiechart* of the Tklib Trackers. Please also report any ideas

for enhancements you may have for either package and/or documentation.

Copyright

Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>

pieboxlabeler

Keywords: pieboxlabeler, canvas, labeler, pie, slice

Name

pieBoxLabeler - tkpiechart pie box style labeler class

Synopsis

stooop::new pieBoxLabeler *canvas* ?*options*?
switched::configure *pieBoxLabelerObject* ?*options*?
switched::cget *pieBoxLabelerObject option*

Description

The pie box style labeler object is used as a slice label placer for a **pie** object and is passed to the pie constructor via its **-labeler** option (see the **pie** class manual).

The labels are arranged in 2 columns below the pie graphics. Each label text is placed to the right of a rectangle, the background color of which matches its corresponding slice. The slice share value is placed to the right of the label text, separated by a semicolon. Each label is actually a canvasLabel object (see the **canvasLabel** class manual for further information).

There is no need to delete a **pieBoxLabeler** object as it is automatically handled by the pie class.

stooop::new pieBoxLabeler *canvas* ?options?
Creates a pieBoxLabeler object in the specified Tk canvas. The pieBoxLabeler object identifier is returned (referred to as *pieBoxLabelerObject* in this document).

switched::configure *pieBoxLabelerObject* ?options?
Configures a pieBoxLabeler object or returns all the options with their current values if no options are passed as parameters.

switched::cget *pieBoxLabelerObject option*
Returns an option value for the specified pieBoxLabeler object.

Options

-font value
Specifies a font for the slice labels. If not specified, the default font is system dependent.

-justify value
Specifies how to justify labels within their own column. Must be one of *left*, *center* or *right*. Defaults to *left*. For example, if justification is *right*, all column labels right edges are aligned.

-offset value
Specifies the distance between the pie graphics and the closest slice label. This value may be specified in any of the forms described in the **canvas** *COORDINATES* manual section.

Tags

The labeler has the following specific tag (see the **canvas** manual page *ITEM IDS AND TAGS* section for more information):

- pieBoxLabeler(pieBoxLabelerObject)

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tkpiechart* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```


pieperipherallabeler

Keywords: pieperipherallabeler, canvas, labeler, pie, slice

Name

piePeripheralLabeler - tkpiechart pie peripheral style labeler class

Synopsis

stooop::new piePeripheralLabeler *canvas* ?*options*?
switched::configure *piePeripheralLabelerObject* ?*options*?
switched::cget *piePeripheralLabelerObject option*

Description

The pie peripheral style labeler object is used as a slice label placer for a **pie** object and is passed to the pie constructor via its **-labeler** option (see the **pie** class manual).

The slice description text labels are arranged in 2 columns below the pie graphics, whereas the slice values are placed next to the slice and actually follow the slice as the pie is updated. Each description label text is placed to the right of a rectangle, the background color of which matches its corresponding slice. Each description label is actually a canvasLabel object.

There is no need to delete a **piePeripheralLabeler** object as it is automatically handled by the pie class.

stooop::new piePeripheralLabeler *canvas* ?options?
Creates a piePeripheralLabeler object in the specified Tk canvas. The piePeripheralLabeler object identifier is returned (refered to as *piePeripheralLabelerObject* in this document).

switched::configure *piePeripheralLabelerObject* ?options?
Configures a piePeripheralLabeler object or returns all the options with their current values if no options are passed as parameters.

switched::cget *piePeripheralLabelerObject option*
Returns an option value for the specified piePeripheralLabeler object.

Options

-font value
Specifies a font for the slice labels. If not specified, the default font is system dependent.

-justify value

Specifies how to justify labels within their own column. Must be one of *left*, *center* or *right*. Defaults to *left*. For example, if justification is *right*, all column labels right edges are aligned.

-offset value
Specifies the distance between the pie graphics and the closest slice label. This value may be specified in any of the forms described in the **canvas** *COORDINATES* manual section.

-smallfont
Specifies a font for the slice values. It is usually a small font in order to avoid values overlapping when 2 slices are very close to each other. If not specified, the description label font (**-font** option) is used.

-widestvaluetext
Specifies a string of maximum width for slice values (placed around the pie next to the slices), so that enough room is allocated for these value labels when the pie width and height are set. It defaults to 00.0. For example, it could be set to "00.00 %".

Tags
The labeler has the following specific tag (see the **canvas** manual page *ITEM IDS AND TAGS* section for more information):

- piePeripheralLabeler(piePeripheralLabelerObject)

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tkpiechart* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright

```
Copyright (c) 1995-2004 Jean-Luc Fontaine <jfontain@free.fr>
```


tooltip

tipstack
Keywords: tipstack, balloon, help, hover

Name
tipstack - Stacked tooltips

Synopsis
package require Tcl 8.5
package require msgcat 1.3
package require tooltip ?**1.7**?
package require tipstack ?**1.0.1**?
::tipstack::push *widget* ?*-index index*? *text*
::tipstack::pop *widget* ?*-index index*?
::tipstack::clear *widget* ?*-index index*?
::tipstack::def *widget text* ...
::tipstack::defsub *base widget text* ...

::tipstack::clearsub *base*

Description

This package extends the functionality of package **tooltip**. It provides a dynamic stack of tip texts per widget. This enables dynamic transient changes to the tips, for example to temporarily replace a standard explanation of a field with an error message.

Commands

::tipstack::push *widget* ?-index *index*? *text*

Push a new *text* to the tooltip for the *widget*. In case of a list widget use

```
-index ...
```

to address the particular entry to change the
tooltip for.

The result of the command is the empty string

::tipstack::pop *widget* ?-index *index*?

Pop the current tooltip for the *widget* from the stack and restore the previous text. This is a no-operation if this would leave an empty stack behind. In other words, the baseline tooltip text cannot be popped of. In case of a list widget use

```
-index ...
```

to address the particular entry to change the
tooltip for.

The result of the command is the empty string

::tipstack::clear *widget* ?-index *index*?

Clear the stack for the *widget* and restore back to the baseline. In case of a list widget use

```
-index ...
```

to address the particular entry to change the
tooltip for.

The result of the command is the empty string

::tipstack::def *widget text ...*

Perform multiple pushes for a number of independent *widget*s in a single call. This command cannot be used for list widgets, as it does not allow the passing of the necessary index information. Use with menus is not possible either.

The result of the command is the empty string

::tipstack::defsub *base widget text ...*

This command is a variant of **::tipstack::def** where all the widgets to push to are subwidgets of the *base*. This is good for mega-widgets.

Note that either each *widget* has to be specified with a proper leading dot (**.**), or the *base* has

to be specigfied with a trailing dot.

The result of the command is the empty string

::tipstack::clearsub *base*
This command is a variant of **::tipstack::clear** which clears all child widgets of the *base* text was pushed to. Use with menus is not possible.

The result of the command is the empty string

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tooltip* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright
```
Copyright (c) 2003 ActiveState Corp
```

tooltip
Keywords: tooltip, balloon, help, hover

Name
tooltip - Tooltip management

Synopsis
package require Tcl 8.5
package require tooltip ?**2.0.4**?
::tooltip::tooltip *command* ?*options*?
::tooltip::tooltip *pathName* ?*option value...*? ?**--**? *message*

Description
This package provides tooltips, i.e., small text messages that can be displayed when the mouse hovers over a widget, menu item, canvas item, listbox item, ttk::treeview item or column heading, ttk::notebook tab, or text widget tag.

Commands
::tooltip::tooltip *command* ?*options*?
Manage the tooltip package using the following subcommands.

clear ?*pattern*?
Prevents the specified widgets from showing tooltips. *pattern* is a glob pattern and defaults to matching all widgets.

configure ?*option* ?*value option value ...*??
Queries or modifies the configuration options of the tooltip. The supported options are **-backgroud**, **-foreground** and **-font**. If no *option* is specified, returns a dictionary of the option values. If one *option* is specified with no value, returns the value of that option. Otherwise, sets the given *option*s to the corresponding *value*s.

delay ?*millisecs*?

Query or set the hover delay. This is the interval that the pointer must remain over the widget before the tooltip is displayed. The delay is specified in milliseconds and must be greater than or equal to 50 ms. With no argument the current delay is returned.

fade ?*boolean*?
Enable or disable fading of the tooltip. The fading is enabled by default on Win32 and Aqua. The tooltip will fade away on Leave events instead disappearing.

disable
see below ...

off
Disable all tooltips

enable
see below ...

on
Enables tooltips for defined widgets.

::tooltip::tooltip *pathName* ?*option value*...? ?--? *message*
This command arranges for widget *pathName* to display a tooltip with a *message*.

If the specified widget is a **menu**, **canvas**, **listbox**, **ttk::treeview**, **ttk::notebook** or **text** widget then additional options are used to tie the tooltip to specific menu, canvas or listbox items, ttk::treeview items or column headings, ttk::notebook tabs, or text widget tags.

-heading *columnId*
This option is used to set a tooltip for a ttk::treeview column heading. The column does not need to already exist. You should not use the same identifiers for columns and items in a widget for which you are using tooltips as their tooltips will be mixed. The widget must be a ttk::treeview widget.

-image *image*
The specified (photo) image will be displayed to the left of the primary tooltip *message*.

-index *index*
This option is used to set a tooltip on a menu item. The index may be either the entry index or the entry label. The widget must be a menu widget but the entries do not have to exist when the tooltip is set.

-info *info*
The specified *info* text will be displayed as additional information below the primary tooltip *message*.

-items *items*
This option is used to set a tooltip for canvas, listbox or ttk::treview items. For the canvas widget, the item must already be present in the canvas and will be found with a **find withtag** lookup. For listbox and ttk::treview widgets the item(s) may be created later but the programmer is responsible for managing the link between the listbox or ttk::treview item index and the corresponding tooltip. If the listbox or ttk::treview items are re-ordered, the tooltips will need amending.

If the widget is not a canvas, listbox or ttk::treview then an error is raised.

-tab *tabId*
The **-tab** option can be used to set a tooltip for a ttk::notebook tab. The tab should already

be present when this command is called, or an error will be returned. The widget must be a ttk::notebook widget.

-tag *name*
The **-tag** option can be used to set a tooltip for a text widget tag. The tag should already be present when this command is called, or an error will be returned. The widget must be a text widget.

--
The **--** option marks the end of options. The argument following this one will be treated as *message* even if it starts with a **-**.

Example

```
# Demonstrate widget tooltip
package require tooltip
pack [label .l -text "label"]
tooltip::tooltip .l "This is a label widget"
```

```
# Demonstrate menu tooltip
package require tooltip
. configure -menu [menu .menu]
.menu add cascade -label Test -menu [menu .menu.test -tearoff 0]
.menu.test add command -label Tooltip
tooltip::tooltip .menu.test -index 0 "This is a menu tooltip"
```

```
# Demonstrate canvas item tooltip
package require tooltip
pack [canvas .c]
set item [.c create rectangle 10 10 80 80 -fill red]
tooltip::tooltip .c -item $item "Canvas item tooltip"
```

```
# Demonstrate listbox item tooltip
package require tooltip
pack [listbox .lb]
.lb insert 0 "item one"
tooltip::tooltip .lb -item 0 "Listbox item tooltip"
```

```
# Demonstrate ttk::notebook tab tooltip
package require tooltip
pack [ttk::notebook .nb]
.nb add [frame .nb.f1 -height 50] -text "First tab"
.nb add [frame .nb.f2 -height 50] -text "Second tab"
tooltip::tooltip .nb -tab 0 "Tooltip for the 1st notebook tab"
tooltip::tooltip .nb -tab 1 "Tooltip for the 2nd notebook tab"
```

```
# Demonstrate text tag tooltip
package require tooltip
pack [text .txt]
.txt tag configure TIP-1 -underline 1
tooltip::tooltip .txt -tag TIP-1 "tooltip one text"
.txt insert end "An example of a " {} "tooltip" TIP-1 " tag.\n" {}
```

Migration from version 1

Version 1.3 to 1.7 called **msgcat::mc** before a tooltip was shown, using the tooltip namespace. **msgcat::mc** requires the caller environment. Due to that, version 1.8 recorded the caller namespace and used this in the call. In version 2.0, any **msgcat::mc** support was removed. The options **-namespace**, **-msgargs** and **-infoargs** were removed. Starting with TCL 8.7, **msgcat::mc** supports oo classes and oo methods. But the oo caller environment is not present when the **msgcat::mc** was invoked on tooltip display, resulting in runtime errors. It was concluded as bad design to call **msgcat::mc** late. The caller should reinstall the tooltips on eventual message change.

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *tooltip* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Copyright


```
Copyright (c) 1996-2008, Jeffrey Hobbs
Copyright (c) 2024 Emmanuel Frecon
```


widget

widget

Keywords: widget, bundle, calendar, dateentry, dialog, megawidget, menu, panelframe, ruler, screenruler, scrolledwindow, snit, statusbar, superframe, toolbar

Name

widget - Megawidget bundle

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.2**?
package require snit
widget::validate *as* ?*options*?
widget::calendar *pathname* ?*options*?
widget::dateentry *pathname* ?*options*?
widget::dialog *pathname* ?*options*?
widget::menuentry *pathname* ?*options*?
widget::panelframe *pathname* ?*options*?

widget::ruler *pathname* ?*options*?
widget::screenruler *pathname* ?*options*?
widget::scrolledwindow *pathname* ?*options*?
widget::statusbar *pathname* ?*options*?
widget::superframe *pathname* ?*options*?
widget::toolbar *pathname* ?*options*?

Description

This package provides megawidgets based on the snit oo system (snidgets). It makes use of the Tile/Ttk themed widget set.

widget::validate *as* ?options?
commands:

Widgets

widget::calendar *pathname* ?options?
options:
widget::dateentry *pathname* ?options?
options:
widget::dialog *pathname* ?options?
options:
widget::menuentry *pathname* ?options?
options:
widget::panelframe *pathname* ?options?
options:
widget::ruler *pathname* ?options?
options:
widget::screenruler *pathname* ?options?
options:
widget::scrolledwindow *pathname* ?options?
options:
widget::statusbar *pathname* ?options?
options:
widget::superframe *pathname* ?options?
options:
widget::toolbar *pathname* ?options?
options:

Example

```
package require widget::superframe ; # or widget::all
pack [widget::superframe .f -type separator -text "SuperFrame:"]
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget* of the Tklib Trackers. Please also report any ideas for

enhancements you may have for either package and/or documentation.

Category
Widget

widget_calendar
Keywords: widget_calendar, calendar, date, megawidget, snit, widget

Name
widget_calendar - Calendar Megawidget

Synopsis
package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.0**?
package require widget::calendar ?**1.0.2**?
widget::calendar *pathname* ?*options*?

Description
This package provides a calendar megawidget (snidget).

- **widget::calendar** *pathname* ?options?

Widget options
-command
A script to evaluate when a date was selected.

-dateformat
The format of the date that is returned. Default: %m/%d/%Y.

-firstday
Set first day the week, Either sunday or monday. It defaults to monday.

-font
Select the font used in the widget. It defaults to Helvetica 9.

-highlightcolor
Selects the background color for the day that has been selected. Default: #FFCC00

-language
Specify language of the calendar contents. The language is specified by abbreviations of the languge, for example: en - english, de - german ... It defaults to en.

Supported languages: de en es fr gr he it ja sv pl pt zh fi tr nl ru crk crx-nak crx-lhe

-shadecolor
Selects the color of the parts that have a shaded background. Default: #888888

-showpast
Specify if the past shall be shown. It is a boolean value and defaults to 1.

-textvariable
Specifies the name of a variable whose value is linked to the entry widget's contents. Whenever the variable changes value, the widget's contents are updated, and vice versa.

Widget command

pathname **get** ?*what*?

Returns a part of the selected date or 'all'. The argument *what* selects the part. Valid values for *what* are: day, month, year and all. 'all' is the default and returns the complete date in the format given with -dateformat.

Default bindings

On creation of the calendar widget the following bindings are installed. When pressing a key the command is invoked and the textvariable is updated. updated.

- Home - Move to the current date
- Up - Move to week before current date
- Down - Move to week after current date
- Left - Move to day before current date
- Right - Move to day after current date
- Control-Left - Move to month before current date
- Control-Right - Move to month after current date
- Control-Up - Move to year before current date
- Control-Down - Move to year after current date

Example

```
    package require widget::calendar ; # or widget::all
    set t [widget::calendar .t]
    pack $t -fill x -expand 1
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget::calendar* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Widget

widget_dateentry

Keywords: widget_dateentry, date, dateentry, megawidget, snit, widget

Name

widget_dateentry - Date Entry Megawidget

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.0**?
package require widget::dateentry ?**0.99**?
widget::dateentry *pathname* ?*options*?
pathname **get**
pathname **set** *date*

Description

This package provides a dateentry megawidget (snidget). It is based on an ttk::entry. All widget commands of the ttk::entry are available for the dateentry.

- **widget::dateentry** *pathname* ?options?

Widget options

-command
A command prefix to evaluate when a date was selected. The command prefix is executed in the global namespace and given two arguments, the raw date in seconds, and the formatted date, as per option **-dateformat**.

-dateformat
The format of the date that is entered or returned. Default: %m/%d/%Y.

-firstday
See the widget::calendar man page.

-font
Select the font used in the widget. It defaults to Helvetica 9.

-highlightcolor
See the widget::calendar man page. See the calendar man page.

-language
See the widget::calendar man page.

-shadecolor
See the widget::calendar man page.

-showpast
See the widget::calendar man page.

-textvariable
Specifies the name of a variable whose value is linked to the entry widget's contents. Whenever the variable changes value, the widget's contents are updated, and vice versa.

Widget command

***pathname* get**
Returns the selected date.

pathname* set *date
Programmatically sets a new date. Expects that the date is in the same format as configured by option **-dateformat**.

The same effect can also be achieved by setting a linked **-textvariable**.

Default bindings

On creation of the dateentry widget the following bindings are installed. For navigation within the calendar, see its manpage.

- Button-1 - Accept and select the date and close the calendar window.
- Return, space - Accept the selected date and close the calendar window
- Escape - Close calendar window
- Control-space - Show calendar window

Example

```
    package require widget::dateentry; # or widget::all
    set t [widget::dateentry .de]
    pack $t -fill x -expand 1
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget::dateentry* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category

Widget

widget_toolbar

Keywords: widget_toolbar, megawidget, snit, toolbar, widget

Name

widget_toolbar - Toolbar Megawidget

Synopsis

package require Tcl 8.4
package require Tk 8.4
package require widget ?**3.0**?
package require widget::toolbar ?**1.0**?
widget::toolbar *pathname* ?*options*?

getframe

add ?item? ?args?

delete item1 ?item2? ?...?

itemcget symbol option

itemconfigure symbol ?args?

items ?pattern?

remove ?-destroy? item1 ?item2? ?...?

Description

This package provides a toolbar megawidget (snidget). It makes use of the Tile/Ttk themed widget set.

- **widget::toolbar** *pathname* ?options?
- getframe
- add ?item? ?args?
- delete item1 ?item2? ?...?
- itemcget symbol option
- itemconfigure symbol ?args?
- items ?pattern?
- remove ?-destroy? item1 ?item2? ?...?

Widget options
- **-ipad**
- **-pad**
- **-separator**

Item options
- **-pad**
- **-separator**
- **-sticky**
- **-weight**

Example
```
package require widget::toolbar ; # or widget::all
set t [widget::toolbar .t]
pack $t -fill x -expand 1
$t add button [button .b -text foo]
$t add separator -pad {2 4}
$t add button [button .c -text bar]
```

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget::toolbar* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

Category
Widget

widgetPlus

widgetPlus
Keywords: widgetPlus

Name
widgetPlus - Enhanced Entry, Spinbox, and Combobox Widgets with Undo/Redo and other useful features

Synopsis
package require Tcl 8.5
package require Tk 8.5
package require snit
package require widgetPlus ?**1.0**?
widgetPlus::entryPlus ?*create*? *pathName*
widgetPlus::spinboxPlus ?*create*? *pathName*
widgetPlus::ttkEntryPlus ?*create*? *pathName*
widgetPlus::ttkSpinboxPlus ?*create*? *pathName*
widgetPlus::ttkComboboxPlus ?*create*? *pathName*

widgetPlus::EnableBWidget
pathName **edit** ?*arg arg* ...?
pathName **edit canredo**
pathName **edit canundo**
pathName **edit modified** ?*boolean*?
pathName **edit redo**
pathName **edit reset**
pathName **edit separator**
pathName **edit undo**

Description

The **widgetPlus** package adds new features to the widgets **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox**, and **ttk::combobox**.

The new features are Undo and Redo operations, a **<>** event, and for the **Tk** (unthemed) widgets an **-inactiveselectbackground** option, and some minor fixes to bindings. The new facilities closely follow those of the **text** widget.

The revised **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox**, and **ttk::combobox** widgets are created by the commands **widgetPlus::entryPlus**, **widgetPlus::spinboxPlus**, **widgetPlus::ttkEntryPlus**, **widgetPlus::ttkSpinboxPlus**, and **widgetPlus::ttkComboboxPlus** respectively.

For example, the **widgetPlus::entryPlus** widget is implemented as a *Snit* **snit::widgetadaptor** that wraps a **Tk entry** widget. Its **winfo class** is **Entry**, and it uses a binding tag entryPlus that replaces the **Entry** binding tag. The other widgets are implemented in a similar way.

The package uses the namespace **::widgetPlus**, which exports (only) the five commands defined below that begin with a lower-case letter (**EnableBWidget** is not exported). These commands can be imported into the global namespace, and for simplicity, descriptive text in this manual will henceforth omit the namespace qualifier.

```
namespace import ::widgetPlus::*
```

Widget options

In addition to the usual options of the **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox**, and **ttk::combobox** widgets, the wrappers **entryPlus**, **spinboxPlus**, **ttkEntryPlus**, **ttkSpinboxPlus**, and **ttkComboboxPlus** provide the options below.

-inactiveselectbackground (inactiveSelectBackground/Foreground)
Specifies the color to use for the selection when the window does not have the input **focus**. If empty, **{}**, then no selection is shown when the window does not have the **focus**. The **-inactiveselectbackground** option has no effect on the **Ttk** (themed) widgets **ttkEntryPlus**, **ttkSpinboxPlus**, and **ttkComboboxPlus**.

-maxundo (maxUndo/MaxUndo)
Specifies the maximum number of compound undo actions on the *undo stack*. A zero or a negative value imply an unlimited *undo stack*.

-undo (undo/Undo)
Specifies a *boolean* that says whether the undo mechanism is active or not.

Commands

widgetPlus::entryPlus ?*create*? *pathName*
Create an **entryPlus** widget; or convert an existing **entry** widget into an **entryPlus**.

widgetPlus::spinboxPlus ?*create*? *pathName*
Create a **spinboxPlus** widget; or convert an existing **spinbox** widget into a **spinboxPlus**.

widgetPlus::ttkEntryPlus ?*create*? *pathName*
Create a **ttkEntryPlus** widget; or convert an existing **ttk::entry** widget into a **ttkEntryPlus**.

widgetPlus::ttkSpinboxPlus ?*create*? *pathName*
Create a **ttkSpinboxPlus** widget; or convert an existing **ttk::spinbox** widget into a **ttkSpinboxPlus**.

widgetPlus::ttkComboboxPlus ?*create*? *pathName*
Create a **ttkComboboxPlus** widget; or convert an existing **ttk::combobox** widget into a **ttkComboboxPlus**.

widgetPlus::EnableBWidget
Enable **widgetPlus** to modify **BWidget Entry** widgets and their compounds (such as the **BWidget ComboBox**). The command creates a bindtag **BwEntryPlus** to replace the default bindtag **BwEntry**. This operation is not performed when the package is loaded because it requires **BWidget** to be loaded first. This command is not exported from namespace **::widgetPlus**.

Widget commands

In addition to the usual commands of the **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox**, and **ttk::combobox** widgets, the wrappers **entryPlus**, **ttkEntryPlus**, **spinboxPlus**, **ttkSpinboxPlus**, and **ttkComboboxPlus** provide:

***pathName* edit ?*arg arg ...*?**
This command controls the undo mechanism and the *modified flag*. The exact behavior of the command depends on the option argument that follows the edit argument. The following forms of the command are currently supported:

***pathName* edit canredo**
Returns a *boolean* **true** if redo is possible, i.e. when the *redo stack* is not empty. Otherwise returns **false**.

***pathName* edit canundo**
Returns a *boolean* **true** if undo is possible, i.e. when the *undo stack* is not empty. Otherwise returns **false**.

***pathName* edit modified ?*boolean*?**
If *boolean* is not specified, returns the *modified flag* of the widget. The **insert**, **delete**, **edit undo** and **edit redo** commands, or (by using this command) the user, can set or clear the *modified flag*. If *boolean* is specified, sets the *modified flag* of the widget to *boolean*.

***pathName* edit redo**
When the **-undo** option is **true**, reapplies the last undone edits provided no other edits were done since then. Generates an error when the *redo stack* is empty. Does nothing when the **-undo** option is **false**.

***pathName* edit reset**
Clears the *undo stack* and *redo stack*.

***pathName* edit separator**
Included for completeness, but has no effect.

(N.B. An implicit separator is added to the *undo stack* for each call to a widget command (e.g. **insert** or **delete**) that changes the widget contents. These therefore separate the addition or removal of individual characters by typing, except that *cut* or *paste* operations are discrete.)

***pathName* edit undo**
Undoes the last edit action when the **-undo** option is **true**. An edit action is defined as a single **insert** or **delete** command that is recorded on the *undo stack*. Generates an error when the *undo stack* is empty. Does nothing when the **-undo** option is **false**.

Bindings

Class bindings

The class binding tag for different widgets has these default values:

- **entry** uses **Entry**
- **spinbox** uses **Spinbox**
- **ttk::entry** uses **TEntry**
- **ttk::spinbox** uses **TSpinbox**
- **ttk::combobox** uses **TCombobox**
- **entryPlus** uses entryPlus
- **spinboxPlus** uses spinboxPlus
- **ttkEntryPlus** uses TEntryPlus
- **ttkSpinboxPlus** uses TSpinboxPlus
- **ttkComboboxPlus** uses TComboboxPlus

Undo and Redo

The **<>** and **<>** virtual events are mapped to these keyboard events:

- On X11 systems (Linux®, BSD®, Solaris®, etc):
  - **<>** is mapped to
- **<>** is mapped to
- On Windows® systems:
  - **<>** is mapped to
- **<>** is mapped to
- On Aqua® systems (i.e. Apple® macOS® or OS X®):
  - **<>** is mapped to
- **<>** is mapped to

entryPlus

The entryPlus bindings are based on those of the **Entry** bindtag, with these changes:

- New bindings to **<>** and **<>**.
- An adjustment to the **<>** binding, so that it behaves the same way on X11 as on other windowing systems (i.e. it deletes the existing selection, if any exists).
- The removal of the antiquated binding.

- New bindings to and, to facilitate the implementation of the **-inactiveselectbackground** option.

spinboxPlus
The spinboxPlus bindings are based on those of the **Spinbox** bindtag, with the same changes as are made when converting **Entry** to entryPlus.

TEntryPlus
The TEntryPlus bindings are based on those of the **TEntry** bindtag, with these changes:

- New bindings to **<>** and **<>**.

TSpinboxPlus
The TSpinboxPlus bindings are based on those of the **TSpinbox** bindtag, with these changes:

- New bindings to **<>** and **<>**.

TComboboxPlus
The TComboboxPlus bindings are based on those of the **TCombobox** bindtag, with these changes:

- New bindings to **<>** and **<>**.

Virtual event <>
Each **widgetPlus** widget generates a virtual event **<>** whenever the value of its selection changes. This allows other **Tcl** code, for example the package **persistentSelection**, to monitor the selection in these widgets.

Use with megawidgets
When an **entry**, **spinbox**, **ttk::entry**, **ttk::spinbox**, or **ttk::combobox** is a component of a megawidget, it is often possible to apply **widgetPlus** to that component without modifying the megawidget code.

Combobox (by Bryan Oakley)
A **combobox** with path **$w** includes an **entry** widget **$w.entry**, which uses the **Entry** binding tag. Thus **widgetPlus** can be applied to the **combobox** after creation, using the command (see Examples)

```
entryPlus $w.entry -undo 1 -maxundo 0
```

Combobox (BWidget)
A **BWidget ComboBox** with path **$w** includes an **entry** widget **$w.e**, which uses the **BwEntry** binding tag in place of the default tag **Entry**.

Package **widgetPlus** provides a replacement bindtag **BwEntryPlus**, but this is not computed when the package is loaded because it requires **BWidget** to be loaded first. The command **::widgetPlus::EnableBWidget** must be called to provide **BwEntryPlus**, and then the **BWidget ComboBox** can be modified (see Examples)

```
::widgetPlus::EnableBWidget
entryPlus $w.e -undo 1 -maxundo 0
```

To do

- Undo and Redo do not sync the value of the *modified flag* - they always set the value to *boolean* **true**.
- Undo and Redo do not sync the position of the insertion cursor.
- The **<>** event is not generated.
- Undo/Redo separators, and the **-autoseparators** option, as found in the **text** widget, are not implemented. An implicit separator is added for each call to a widget command (e.g. **insert** or **delete**) that changes the widget contents. The **edit separator** command has no effect. Separator functionality would be less useful for widgets such as **entryPlus** than it is for a **text** widget.
- Add an *overwrite* mode, toggled by on non-macOS® systems.

Bugs

This version of **widgetPlus** is intended to be compatible with all releases of **Tk** 8.5 and 8.6, and with the branches *core-8-5-branch*, *core-8-6-branch*, and *trunk* in the source code repository for **Tk**. Any incompatibility with any of these versions, for any **Tk** windowing system, should be reported as a bug. Please report such in the category *widgetPlus* of the Tklib Trackers.

Examples

To create and map an **entryPlus** widget .ep with an unlimited Undo/Redo stack:

```
package require widgetPlus
widgetPlus::entryPlus .ep -undo 1 -maxundo 0
pack .ep
```

The same but using **namespace import**:

```
package require widgetPlus
namespace import widgetPlus::*
entryPlus .ep -undo 1 -maxundo 0
pack .ep
```

To convert an existing **entry** widget .ee into an **entryPlus** with an unlimited Undo/Redo stack:

```
entry .ee

    .
    .
    .

package require widgetPlus
widgetPlus::entryPlus .ee -undo 1 -maxundo 0
```

The same but using **namespace import**:

```
entry .ee

    .
    .
```

```
    .

package require widgetPlus
namespace import widgetPlus::*
entryPlus .ee -undo 1 -maxundo 0
```

Apply **widgetPlus** to a Bryan Oakley **combobox**:

```
package require combobox
combobox::combobox .cb

    .
    .
    .

package require widgetPlus
namespace import widgetPlus::*
entryPlus .cb.entry -undo 1 -maxundo 0
```

Apply **widgetPlus** to a **BWidget ComboBox**. The command **::widgetPlus::EnableBWidget** must be called first.

```
package require BWidget
ComboBox .cb

    .
    .
    .

package require widgetPlus
::widgetPlus::EnableBWidget
namespace import widgetPlus::*

entryPlus .cb.e -undo 1 -maxundo 0
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widgetPlus* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

widgetl

widget_listentry

Keywords: widget_listentry, data entry lists, data entry ordered list, data entry set of strings, data entry unordered list, list entry panel, set entry panel, widget

Name

widget_listentry - widget::listentry widget

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require widget::listentry ?**0.1.2**?
package require widget::validator ?**0.1**?
package require widget::scrolledwindow
package require snit
package require tooltip
package require img::png
package require msgcat
widget::listentry *pathname* ?*options*?
widgetCmd destroy
widgetCmd configure
widgetCmd configure *option value ...*
widgetCmd configure *option*
widgetCmd cget *option*
{}cmdprefix* **get**
{}cmdprefix* **set** *values*
{}cmdprefix* **text** *errvar*
{}cmdprefix* **text**
{}cmdprefix* **cookievar**

Description

This package provides a megawidget for the interactive entry of ordered and unordered lists. For a simpler and more restricted megawidget please see the package widget::listsimple.

Class API

The widget class supports a single command, for the creation of widgets.

widget::listentry *pathname* ?options?

This command creates and configures new instances of the widget.

For details on the available options please see section Widget options.

The result of the command is the pathname of the new widget.

Instance API

All widget instances supported the following methods.

widgetCmd destroy

This method destroys the widget. Any further access to the widget will generate errors.

The result of the command is the empty string.

widgetCmd configure

This method comes in three variants. This variant here returns a list containing the current configuration of the widget, i.e. the values for all options.

For details on the available options please see section Widget options.

widgetCmd configure *option value ...*
This method comes in three variants. This variant here reconfigures the widget, setting the specified options to the given values.

Note that it is not possible to change the construction-time only options.

For details on the available options please see section Widget General options.

The result of the command is the empty string.

widgetCmd configure *option*
This method comes in three variants. This variant here is an alias for the method **cget** below and identical to it.

widgetCmd cget *option*
This method returns the current value of the specified *option*.

For details on the available options please see section Widget options.

Widget options

This section explains all the options available to instances of **widget::listentry**. Please note that a few of the options can be set only at instance construction time. The majority of the options can however be set both during construction- and runtime.

Widget construction-time only options

-ordered boolean
This options tells the new widget instance whether the list it manages is ordered or not. If it is ordered the widget will show buttons with which the selected entries can be moved around, editing the order.

The default is **false**, indicating an unordered list.

-allow-duplicates boolean
This options tells the new widget instance whether we are allowed to add a string to the list multiple times, or not.

The default is **false**, indicating that duplicates are not allowed.

-values cmdprefix
This option specifies a callback for the management of a predefined list of strings a user may enter.

If specified the widget will use a combobox instead of a plain entry field and fill its list during widget construction using the data delivered by this callback. The callback will be further invoked whenever a new value is entered into the main list, to save the extended list of predefined values.

The signature of this callback is

***{*}cmdprefix* get**
In this form the callback is expected to return a list of strings. The strings are loaded into the list of the internal combobox for quick access by the user.

It will be invoked once, during widget construction, to load the list of predefined strings a user may enter.

{*}cmdprefix* set *values

In this form the callback is given a list of new strings and expected to save them somewhere for future use.

The return value of the callback is ignored.

This form is invoked whenever the user entered a new string into the main list. The order of strings in *values* represents the order used to show them in the combobox's list.

-validate cmdprefix

This option specifies a callback which is invoked after every change of the contents of the internal entry field. The signature of this callback is

{*}cmdprefix text errvar

where *text* is the string to validate, and *errvar* the name of a variable the callback can store an error message into when it detects invalid data.

The widget expects that the callback returns a boolean value, where **true** indicates that *text* is valid.

The default validation, when no callback was specified, will treat the empty string as invalid, and everything else as valid.

Please note that the widget will prevent the entry of duplicate values into the main list, regardless of the validity of the input otherwise. This is in keeping with that this widget is meant for the entry of unordered lists, essentially a set of strings.

-transform cmdprefix

This option specifies a callback which is invoked whenever a newly entered element is moved from the entry field to the main list, or the entry field is validated, as part of a check for duplicates, if such are not allowed. The callback is given the text in the entry field and has to return the text which is actually added to, or checked against the list.

The use case for this callback is essentially *input normalization*. The most simple case of such would be, for example the use of

```
string tolower
```

to impose a standard letter

case on the data. More complex example can be thought of, like rewriting of multiple syntaxes of input to a canonical form. The signature of this callback is

{*}cmdprefix text

where *text* is the string to transform.

The widget expects that the callback returns the transformed string.

The default is to not transform the entered strings.

-browse cmdprefix
If this option is specified the widget will display a "Search" button and invoke the callback provided by the option whenever the user clicks on that button. The signature of the callback is

{*}cmdprefix cookievar
where the cookie variable provides access to context information the callback may wish to preserve between invokations. The initial value stored in the variable is the empty string. After that it is whatever the callback wishes to store.

The widget expects that the callback returns a list of strings, which are then added to the list, modulo validity and duplicate checks.

By default there is no browse callback and no button shown.

Widget General options

-listvariable varname
This option specifies the variable holding the list to be manipulated by the widget. Any changes to the list outside of the widget are automatically imported into the widget. Similarly, all changes made to the list by the widget are automatically exported to this variable.

-state normal|disabled
This option specifies the status of the widget, **normal** (= active), or **disabled**. A disabled widget can not be edited.

The default is **normal**.

-height integer
This option specifies the height of the internal listbox, in lines.

The default is **20**.

-skin-add string
see below ...

-skin-remove string
see below ...

-skin-up string
see below ...

-skin-down string
see below ...

-skin-browse string
see below ...

-skin-tip-add string
see below ...

-skin-tip-remove string
see below ...

-skin-tip-up string
see below ...

-skin-tip-down string

see below ...

-skin-tip-browse string
see below ...

-skin-tip-main string
see below ...

-skin-tip-entry string
see below ...

-skin-tip-list string
see below ...

-skin-tip-empty string
see below ...

-skin-tip-duplicate string
see below ...

-skin-tip-add-none string
see below ...

-skin-tip-remove-none string
see below ...

-skin-img-add image
see below ...

-skin-img-remove image
see below ...

-skin-img-up string
see below ...

-skin-img-down string
see below ...

-skin-img-browse string
see below ...

-skin-invalid-color color
These options all modify the appearance of the widget, i.e. its skin.

All options taking a string argument influence the various labels shown, with the **-skin-tip-*** options influencing the tooltips shown on hovering the over various parts in particular.

All the strings are run through **msgcat** first, enabling text localization through message catalogs. The default values are keys into the message catalogs which are part of the package itself.

The options taking images modify the images shown on the buttons for adding and removing elements of the list. They default to the PNG images distributed with the package itself.

The single option taking a color value modifies the color used to highlight invalid data entered into the internal entry field of the widget. This option defaults to **yellow**.

Example

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget::listentry* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

widget_listsimple

Keywords: widget_listsimple, data entry lists, data entry set of strings, data entry unordered list, list entry panel, set entry panel, widget

Name

widget_listsimple - widget::listsimple widget

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require widget::listsimple ?**0.1.2**?
package require widget::validator ?**0.1**?
package require widget::scrolledwindow
package require snit
package require tooltip
package require img::png
package require msgcat
widget::listsimple *pathname* ?*options*?
widgetCmd destroy
widgetCmd configure
widgetCmd configure *option value ...*
widgetCmd configure *option*
widgetCmd cget *option*
{}cmdprefix* **get**
{}cmdprefix* **set** *values*
{}cmdprefix* **text** *errvar*

Description

This package provides a megawidget for the interactive entry of unordered lists. For a megawidget allowing the entry of ordered lists, and more, please see the package widget::listentry.

Class API

The widget class supports a single command, for the creation of widgets.

widget::listsimple *pathname* ?options?

This command creates and configures new instances of the widget.

For details on the available options please see section Widget options.

The result of the command is the pathname of the new widget.

Instance API

All widget instances supported the following methods.

widgetCmd destroy
: This method destroys the widget. Any further access to the widget will generate errors.

  The result of the command is the empty string.

widgetCmd configure
: This method comes in three variants. This variant here returns a list containing the current configuration of the widget, i.e. the values for all options.

  For details on the available options please see section Widget options.

widgetCmd configure *option value ...*
: This method comes in three variants. This variant here reconfigures the widget, setting the specified options to the given values.

  Note that it is not possible to change the construction-time only options.

  For details on the available options please see section Widget General options.

  The result of the command is the empty string.

widgetCmd configure *option*
: This method comes in three variants. This variant here is an alias for the method **cget** below and identical to it.

widgetCmd cget *option*
: This method returns the current value of the specified *option*.

  For details on the available options please see section Widget options.

Widget options

This section explains all the options available to instances of **widget::listsimple**. Please note that a few of the options can be set only at instance construction time. The majority of the options can however be set both during construction- and runtime.

Widget construction-time only options

-values cmdprefix
: This option specifies a callback for the management of a predefined list of strings a user may enter.

  If specified the widget will use a combobox instead of a plain entry field and fill its list during widget construction using the data delivered by this callback. The callback will be further invoked whenever a new value is entered into the main list, to save the extended list of predefined values.

  The signature of this callback is

  ***{*}cmdprefix* get**
  : In this form the callback is expected to return a list of strings. The strings are loaded into the list of the internal combobox for quick access by the user.

It will be invoked once, during widget construction, to load the list of predefined strings a user may enter.

{*}cmdprefix* set *values
In this form the callback is given a list of new strings and expected to save them somewhere for future use.

The return value of the callback is ignored.

This form is invoked whenever the user entered a new string into the main list. The order of strings in *values* represents the order used to show them in the combobox's list.

-validate cmdprefix
This option specifies a callback which is invoked after every change of the contents of the internal entry field. The signature of this callback is

{*}cmdprefix text errvar
where *text* is the string to validate, and *errvar* the name of a variable the callback can store an error message into when it detects invalid data.

The system expects that the callback returns a boolean value, where **true** indicates that *text* is valid.

The default validation, when no callback was specified, will treat the empty string as invalid, and everything else as valid.

Please note that the widget will prevent the entry of duplicate values into the main list, regardless of the validity of the input otherwise. This is in keeping with that this widget is meant for the entry of unordered lists, essentially a set of strings.

Widget General options

-listvariable varname
This option specifies the variable holding the list to be manipulated by the widget. Any changes to the list outside of the widget are automatically imported into the widget. Similarly, all changes made to the list by the widget are automatically exported to this variable.

-skin-add string
see below ...

-skin-remove string
see below ...

-skin-tip-add string
see below ...

-skin-tip-remove string
see below ...

-skin-tip-main string
see below ...

-skin-tip-entry string
see below ...

-skin-tip-list string
see below ...
-skin-tip-empty string
see below ...
-skin-tip-duplicate string
see below ...
-skin-tip-add-none string
see below ...
-skin-tip-remove-none string
see below ...
-skin-img-add image
see below ...
-skin-img-remove image
see below ...
-skin-invalid-color color
These options all modify the appearance of the widget, i.e. its skin.

All options taking a string argument influence the various labels shown, with the **-skin-tip-*** options influencing the tooltips shown on hovering the over various parts in particular.

All the strings are run through **msgcat** first, enabling text localization through message catalogs. The default values are keys into the message catalogs which are part of the package itself.

The options taking images modify the images shown on the buttons for adding and removing elements of the list. They default to the PNG images distributed with the package itself.

The single option taking a color value modifies the color used to highlight invalid data entered into the internal entry field of the widget. This option defaults to **yellow**.

Example

Bugs, ideas, feedback
This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget::listsimple* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

widgetv

widget_validator
Keywords: widget_validator, invalid, state management, ttk::combobox, ttk::entry, validation, widget validation

Name
widget_validator - widget::validator behaviour

Synopsis

package require Tcl 8.5
package require Tk 8.5
package require widget::validator ?**0.1**?
widget::validator attach *w color cmdprefix*
widget::validator detach *w*
widget::validator validate *w*

Description

This package provides a unified validation API for **ttk**'s entry and combobox widgets.

Please note that the validation behaviour defined in this package will not reject invalid edits. It will only highlight the entry containing invalid data and set the proper state flag.

It is the responsibility of the using package or application to decide how and when to actually reject such invalid content.

widget::validator attach *w color cmdprefix*

This method adds a validating behaviour to the widget *w*.

Whenever the content of the widget's entry field changes the specified *cmdprefix* is invoked and has to return a boolean value, where **true** means that content is ok, and **false** that the content is invalid. For more information on the command prefix see section Validation. In case of the latter the background color of the entry field is changed to *color* to indicate the invalidity.

The system does not support nesting of validators on a widget, nor the attachment of multiple validators. To change validating conditions **detach** the current validator first before attaching the new.

An error is thrown if the widget has already validating behaviour attached to it.

The result of the method is the empty string.

To achieve its aims the package overrides various configuration options of the widget the behaviour is attached to. These options are restored to their previous values on **detach**.

If other behaviours are attached the validator may be rendered temporarily (partially) non-functional. Similarly, if the validator is detached while a different behaviour is also attached its restoration of configuration settings may render the other non-functional

widget::validator detach *w*

This method removes the validating behaviour from the widget *w* and restores it to its original state.

An error is thrown if the widget has no validating behaviour attached to it.

The result of the method is the empty string.

widget::validator validate *w*

Invoking this method forces a validation of the widget *w*, assuming that it has a validator behaviour attached to it.

The result of the method is the empty string.

Validation

The command prefix for used for validation has to have the following signature:

{*}cmdprefix text

The argument is the text to validate.

The result of the callback has to be a boolean value where **true** means that *text* is ok, and **false** that *text* is invalid.

Example

```
package require Tk 8.5
package require widget::validator

set TE {}
set TC {}

ttk::entry    .e -textvariable TE
ttk::combobox .c -textvariable TC -values {fruit vegetable corn}
ttk::combobox .n -values {fruit vegetable corn}
ttk::button   .x -command ::exit -text Exit

pack .e -expand 1 -fill both -side top
pack .c -expand 1 -fill both -side top
pack .n -expand 1 -fill both -side top
pack .x -expand 1 -fill both -side top

widget::validator attach .e lightblue {apply {text {
    expr {$text ne {}}
}}}

widget::validator attach .c yellow {apply {text {
    expr {$text ni {{} hello world}}
}}}

widget::validator attach .n pink {apply {text {
    expr {$text ni {{} blub}}
}}}
```

Bugs, ideas, feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such in the category *widget::validator* of the Tklib Trackers. Please also report any ideas for enhancements you may have for either package and/or documentation.

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

license license terms

This software is copyrighted by Ajuba Solutions and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Index

2

2D geometry 65

A

addon 117, 120, 121, 127, 131, 133
alert 146
append tag 42
arc 65
area display 115, 117, 119
area-display 115
area-file 116
area-map-display 117
area-store-fs 117
area-store-mem 118
area-table-display 119
arrow 65
autoscroll 8

B

background 35
balloon 248, 250
bindtags 146, 150, 153, 157
bitmap 5
bitmap-editor 5
box 65
box display 119, 121, 123
box entry 120
box-display 119
box-entry 120
box-file 121
box-map-display 121
box-store-fs 122
box-store-mem 123
box-table-display 123
bundle 253

C

calendar 253, 255
canvas 5, 9, 12, 16, 22, 27, 31, 35, 37, 38, 39, 40, 42, 44, 56, 65, 240, 242, 246, 247
canvas_drag 9
canvas_ecircle 12
canvas_epoints 16
canvas_epolyline 22
canvas_equad 27

canvas_erectangle 31
canvas_gradient 35
canvas_highlight 37
canvas_mvg 38
canvas_pdf 39
canvas_snap 40
canvas_sqmap 40
canvas_tags 42
canvas_trlines 44
canvas_zoom 45
canvaslabel 240
cell 40
center 118, 123, 129, 134
character 113
chat 46
chatwidget 46
circle 12, 65
clock 63
color gradient 35
composite widget 46
concave 27
controlling 48
controlwidget 48
conversion 5
convex 27
cross-hairs 56
crosshair 44, 56
ctext 58
cursor 63

D

data entry lists 265, 271
data entry ordered list 265
data entry set of strings 265, 271
data entry unordered list 265, 271
date 63, 255, 256
dateentry 63, 253, 256
datefield 63
diagram 5, 65
diagram-viewer 5
dialog 105, 239, 253
diameter 118, 123, 129, 134
diamond 65
disjointlistbox 239
display 124
displaying 48
dll 107

doctools 7, 228
dragging 9
drawing 65
drum 65

E

editing 12, 16, 22, 27, 31
editor 5
ellipse 65
enter callback 37
entry 63, 105, 106, 107, 110
exe 107

F

file 116, 121, 127, 132
filesystem 117, 122, 128, 133
filesystem store 117, 122, 128, 133

G

geo/area 116, 117, 118
geo/area pixels 118
geo/box 121, 123
geo/box pixels 123
geo/point 127, 128, 129
geo/point pixels 129
geo/track 132, 133, 134
geo/track pixels 134
geobox 122
gradients 35
graphics 38, 39
grid 40

H

help 248, 250
highlighting 37
history 106
hover 248, 250
html 7, 228, 229, 231

I

i18n 113
ico 107
icon 107
image 5, 35, 40, 65
imagemagick 38, 39
input 113
insert tag 42
international 113
interpolation 65

intersection 65
invalid 274
ip address 110
ipentry 110
irc 46

K

khim 113

L

labeler 240, 242, 246, 247
leave callback 37
length 118, 123, 129, 134
line 65
list entry panel 265, 271
listbox 239
location 56
location marking 126

M

magick vector graphics 38
map 40, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135
map display 117, 120, 121, 124, 127, 131, 133
map_overview 125
mapnik 130
mark 126
mark location 126
markdown 7
mega-widget 46
megawidget 253, 255, 256, 258
memory 118, 123, 129, 134
memory store 118, 123, 129, 134
menu 253
menubar 135
method 113
mkdoc 7, 229
move 65
mvg 38

N

network 110
non-convex 27
notification 146
notifywindow 146
ntext 146
ntextBindings 150
ntextIndent 153
ntextWordBreak 157

numerical values 48

O

openstreetmap 130

P

panelframe 253
pdf 39
perimeter 118, 123, 129, 134
perimeter length 118, 123, 129, 134
persistentSelection 160
photo 40
picture 65
pie 240, 242, 246, 247
pieboxlabeler 246
pieperipherallabeler 247
pixel 5
pixels 118, 123, 129, 134
plane geometry 65
plotchart 164
plotting 65
point 65
point cloud 16
point display 127, 129
point-file 127
point-map-display 127
point-store-fs 128
point-store-mem 129
point-table-display 129
points 16
polyline 22
portable document format 39
print screen 38, 39, 40
provider 130
provider-osm 130

Q

quadrilateral 27

R

raster image 65
re_syntax 146, 153, 157
rectangle 31
regexp 146, 153, 157
remove tag 42
rubberband 44
ruler 253

S

scale widget 48
screenruler 253
scroll 8
scrollbar 8
scrolledwindow 253
serialization 38, 39
set entry panel 265, 271
shtmlview 7, 231
shtmlview-doctools 228
shtmlview-mkdoc 229
slice 240, 242, 246, 247
snapshot 40
snit 253, 255, 256, 258
spline 65
square map 40
state management 274
statusbar 253
store 117, 118, 122, 123, 128, 129, 133, 134
string 105
superframe 253
swaplist 239
syntax highlighting 58

T

tabular 115, 119, 123, 129, 131, 135
tags 42
text 58, 65, 146, 150, 153, 157, 231
tile 40
tile provider 130
tiles 130
tipstack 248
tk 146
tk_getString 105
tklib_history 106
toolbar 253, 258
tooltip 250
track display 131, 133, 135
track entry 131
track-display 131
track-entry 131
track-file 132
track-map-display 133
track-store-fs 133
track-store-mem 134
track-table-display 135
tracking 44, 56

ttk::combobox 274
ttk::entry 274

V

validation 274
vector 5, 65
vector graphics 38, 39
viewer 7
viewport 56

W

widget 46, 58, 63, 231, 253, 255, 256, 258, 265, 271
widget background 35
widget validation 274
widget_calendar 255
widget_dateentry 256
widget_listentry 265
widget_listsimple 271
widget_toolbar 258
widget_validator 274
widgetPlus 259

X

xbm 5

Z

zoom 45, 118, 123, 129, 134